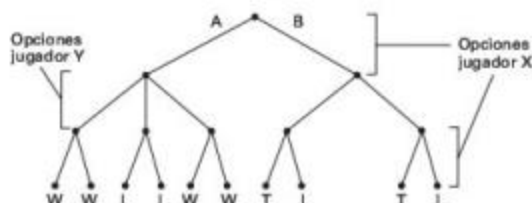


programa de seguro médico de la empresa, pero solo la base de datos A es consciente de que la lista es completa. ¿Qué podría concluir la base de datos A acerca de un empleado que no se encuentre en su lista, mientras que la base de datos B no podría llegar a esa conclusión?

19. ¿Cuáles son los componentes de un sistema de producción?
20. ¿Cuáles son las distintas formas en las que puede construirse un árbol de búsqueda?
21. ¿Qué es un agente inteligente?
22. Analice la tarea de resolver un sudoku en términos de un sistema de producción. (¿Cuáles son los estados, las producciones, etc.?)
23. a. Suponga que un árbol de búsqueda es un árbol binario y que alcanzar el objetivo requiere ocho producciones. ¿Cuál es el máximo número de nodos que podría tener el árbol en el momento de alcanzar el estado objetivo, si se construye el árbol con una búsqueda en anchura?
- b. Explique cómo se podría reducir el número total de nodos analizados durante la búsqueda realizando dos búsquedas al mismo tiempo: una que comience en el estado inicial y otra que busque hacia atrás desde el estado objetivo, hasta que las dos se encuentren. (Suponga que el árbol de búsqueda en el que se anotan los estados que vamos encontrando en la búsqueda hacia atrás es también un árbol binario y que ambas búsquedas progresan al mismo ritmo.)
24. En el texto hemos mencionado que los sistemas de producción se emplean a menudo como técnica para extraer conclusiones de hechos conocidos. Los estados del sistema son los hechos que sabemos que son ciertos en cada etapa del proceso de razonamiento y las producciones son las reglas de la lógica utilizadas para manipular los hechos conocidos. Identifique algunas reglas de la lógica que permiten extraer la conclusión "Juan es alto" a partir de los hechos "Juan es un jugador de baloncesto", "Los jugadores

de baloncesto no son bajos" y "Juan es o bajo o alto".

25. El siguiente árbol representa los posibles movimientos en un juego de competición.



En este árbol podemos ver que el jugador X tiene actualmente la opción de efectuar el movimiento A o el movimiento B. Después del movimiento del jugador X, el jugador Y puede seleccionar un movimiento y entonces el jugador X puede elegir el último movimiento de la partida. Los nodos hoja del árbol están etiquetados como W, L o T, dependiendo de si ese final representa una victoria (W), una derrota (L) o una situación de tablas (T) para el jugador X. ¿Qué debería seleccionar el jugador X, el movimiento A o el B? ¿Por qué? ¿Cómo difiere la selección de una producción en un entorno competitivo de ese mismo proceso de seleccionar una producción en un juego para una sola persona como es el puzzle de ocho piezas?

26. Analice el juego de damas como un sistema de producción y describa un heurístico que podría utilizarse para determinar cuál de dos estados está más próximo al objetivo. ¿Cómo difiere el sistema de control de este juego del utilizado en un juego unipersonal, como por ejemplo el puzzle de ocho piezas?
27. Considerando la reglas de manipulación del álgebra como producciones, los problemas relativos a la simplificación de expresiones algebraicas pueden resolverse en el contexto de un sistema de producción. Identifique un conjunto de producciones algebraicas que permitan reducir la ecuación $3/(2x - 1) = 6/(3x + 1)$ a la forma $x = 3$. ¿Podría indicar algunas reglas prácticas (es decir, reglas heurísticas) utilizadas

al realizar ese tipo de simplificaciones algebraicas?

28. Dibuje el árbol de búsqueda generado por una búsqueda en anchura que trate de resolver el puzzle de ocho piezas a partir del siguiente estado inicial, sin ayuda de ninguna información heurística.

	1	3
4	2	5
7	8	6

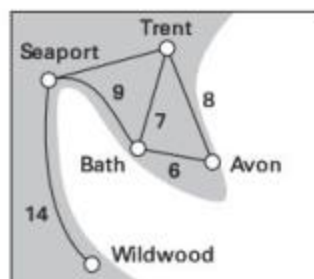
29. Dibuje el árbol de búsqueda que generará el algoritmo de búsqueda por elección del mejor de la Figura 11.10 al intentar resolver el puzzle de ocho piezas a partir del estado inicial mostrado en el Problema 28, si empleamos como heurístico el número de piezas descolocadas
30. Dibuje el árbol de búsqueda que se genera mediante el algoritmo de búsqueda por elección del mejor de la Figura 11.10 al intentar resolver el puzzle de ocho piezas a partir del siguiente estado inicial, suponiendo que el heurístico utilizado es el mismo que el desarrollado en la Sección 11.3.

1	2	3
5	7	6
4		8

31. Al resolver el puzzle de ocho piezas, ¿por qué el número de piezas descolocadas no es tan buen heurístico como el que hemos empleado en la Sección 11.3?
32. ¿Cuál es la diferencia entre la técnica de decidir qué mitad de la lista hay que usar a la hora de realizar una búsqueda binaria (Sección 5.5) y la técnica de decidir qué rama explorar al llevar a cabo una búsqueda heurística?
33. Observe que si un estado del grafo de estados de un sistema de producción tiene un valor heurístico extremadamente bajo en comparación con los otros estados y si existe una producción que va desde ese estado a sí mismo, el algoritmo de la Figura

11.10 puede verse atrapado en un bucle en el que estaría tomando en consideración este estado una y otra vez. Demuestre que si el coste de ejecutar cualquier producción en el sistema es al menos uno, entonces al calcular el coste estimado como la suma del valor heurístico más el coste de alcanzar el estado en cuestión a lo largo de la ruta que se ha recorrido, se evitaría este proceso de bucle infinito.

34. ¿Qué heurístico utilizaría una computadora a la hora de jugar una partida a las tres en raya con una persona?
35. Dibuje hasta cuatro niveles del árbol de búsqueda generado por el algoritmo de búsqueda por elección del mejor de la Figura 11.10 a la hora de averiguar la ruta entre Trent y Wildwood en el siguiente mapa. Cada nodo del árbol de búsqueda será una ciudad del mapa. Comience con el nodo correspondiente a Trent. A la hora de expandir un nodo, añada solo las ciudades que estén directamente conectadas a la ciudad que se está expandiendo. Anote en cada nodo la distancia en línea recta hasta Wildwood y utilice ese valor como heurístico. ¿Tiene algún defecto el algoritmo de búsqueda por elección del mejor con este enfoque? En caso afirmativo, ¿qué corrección hace falta?



Distancia en línea recta a Wildwood desde

Avon	10
Bath	8
Trent	15
Seaport	13

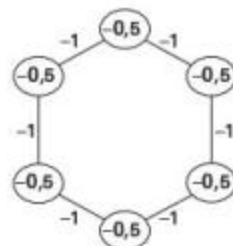
36. El algoritmo A* modifica el algoritmo de búsqueda por elección del mejor de dos formas significativas. En primer lugar, anota el coste real necesario para alcanzar cada estado. En el caso de una ruta en un mapa, el coste real es la distancia recorrida. En segundo lugar, a la hora de seleccionar el nodo que hay que expandir, elige el nodo

que tenga un valor mínimo de la suma entre el coste real y el valor heurístico. Dibuje el árbol de búsqueda del Problema 35 que resultaría al efectuar estas dos modificaciones. Anote en cada nodo la distancia recorrida hasta la ciudad, el valor heurístico para alcanzar el objetivo y su suma. ¿Cuál es la ruta encontrada desde Dearborn a Wildwood?

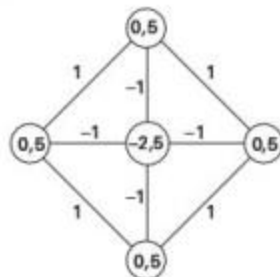
37. ¿Cuáles son las diferencias entre la inteligencia artificial fuerte y la inteligencia artificial débil?
38. En un determinado juego para dos jugadores, se tiene una pila de 5 peniques colocada sobre el suelo. Cualquier jugador puede coger 1, 2 o 3 peniques en su turno. El jugador que coja el último penique pierde. Prediga qué jugador nunca perderá.
39. Suponga que nuestro trabajo consiste en supervisar la carga de dos camiones, cada uno de los cuales puede transportar como máximo catorce toneladas. La carga es un conjunto de contenedores cuyo peso total es de veintiocho toneladas, pero cuyo peso individual varía de un contenedor a otro. El peso de cada contenedor está marcado en un lateral. ¿Qué heurístico utilizaría para repartir los contenedores entre los dos camiones?
40. ¿Cuáles de los siguientes enunciados son ejemplos de meta-razonamiento?
 - a. Hace mucho tiempo que se ha ido, así que debe de estar lejos.
 - b. Como usualmente tomo la decisión incorrecta y las dos últimas decisiones que tomé eran correctas, voy a invertir mi siguiente decisión.
 - c. Estoy cansado, así que probablemente no pienso con claridad.
 - d. Estoy cansado, así que voy a echarme una siesta.
41. ¿Es posible desarrollar un sistema de inteligencia artificial que pueda escribir un algoritmo para resolver un problema?
42. a. ¿En qué se asemejan el aprendizaje por imitación y el aprendizaje mediante entrenamiento supervisado?

b. ¿En qué se diferencian el aprendizaje por imitación y el aprendizaje mediante entrenamiento supervisado?

43. El siguiente diagrama representa una red neuronal artificial para una memoria asociativa, tal como se explica en la Sección 11.5. ¿Qué patrón se asocia con cualquier patrón en el que solo estén excitadas dos neuronas que estén separadas por solo una neurona? ¿Qué sucede si se inicializa la red con todas sus neuronas inhibidas?



44. El siguiente diagrama representa una red neuronal artificial para una memoria asociativa, tal como se explica en la Sección 11.5. ¿Qué configuración estable asocia con cualquier patrón inicial en el que al menos tres de las neuronas del perímetro estén excitadas y la neurona central esté inhibida? ¿Qué sucedería si se le proporcionara un patrón inicial en el que solo estuvieran excitadas dos neuronas del perímetro situadas en posición opuesta la una a la otra?



45. Diseñe una red neuronal artificial para una memoria asociativa (como se explica en la Sección 11.5) que conste de una matriz rectangular de neuronas que trate de derivar hacia patrones estables en los que solo esté excitada una única columna vertical de neuronas.

46. Ajuste los pesos y valores de umbral de la red neuronal artificial de la Figura 11.18 para que su salida sea 1 cuando ambas entradas coincidan (ambas sean 0 o ambas sean 1) y 0 cuando las entradas sean diferentes (una de ellas sea 0, mientras que la otra sea 1).
47. Dibuje un diagrama similar al de la Figura 11.5 que represente el proceso de simplificar la expresión algebraica $7x + 3 = 3x - 5$ para obtener la expresión $x = -2$.
48. Amplíe su respuesta al problema anterior para mostrar otras rutas que un sistema de control podría explorar a la hora de tratar de resolver el problema.
49. Dibuje un diagrama similar al de la Figura 11.5 que represente el proceso de razonamiento implicado a la hora de concluir que "Poli puede volar" a partir de los hechos iniciales "Poli es un loro", "Un loro es un pájaro" y "Todos los pájaros pueden volar".
50. A pesar de lo que se dice en el enunciado del problema anterior, algunos pájaros, como por ejemplo el avestruz o un pájaro que tenga un ala rota, no pueden volar. Sin embargo, no sería razonable construir un sistema de razonamiento deductivo en el que se enumeraran explícitamente todas las excepciones al enunciado "Todos los pájaros pueden volar". ¿Cómo decidimos entonces los seres humanos si un pájaro concreto puede o no volar?
51. Explique cómo depende del contexto el significado de la frase "He leído la nueva ley impositiva".
52. Describa cómo podría expresarse en forma de sistema de producción el problema de viajar de una ciudad a otra. ¿Cuáles son los estados? ¿Cuáles son las producciones?
53. Suponga que debemos realizar tres tareas, A, B y C, y que pueden llevarse a cabo en cualquier orden (pero no simultáneamente). Describa cómo puede expresarse en forma de sistema de producción este problema y dibuje su grafo de estados.
54. ¿Cuáles son los componentes que definen un sistema experto?
55. a. Si la notación (i, j) , donde i y j son enteros positivos, se utiliza para decir que "si la entrada situada en la posición i -ésima de la lista es mayor que la entrada de la posición j -ésima, intercambiar las dos entradas", ¿cuál de las siguientes dos secuencias será más adecuada a la hora de ordenar una lista de longitud igual a tres?
 $(1, 3) (3, 2)$
 $(1, 2) (2, 3) (1, 2)$
 b. Observe que al representar secuencias de intercambios de esta manera, las secuencias pueden descomponerse en subsecuencias que luego pueden conectarse para formar nuevas secuencias. Utilice esta técnica para describir un algoritmo genético que permita desarrollar un programa que ordene listas de longitud igual a diez.
56. Suponga que cada miembro de un grupo de robots debe equiparse con una pareja de sensores. Cada sensor puede detectar un objeto situado delante de él y tiene un alcance de dos metros. Cada robot tiene la forma de una papelera redonda y puede moverse en cualquier dirección. Diseñe una secuencia de experimentos para determinar dónde hay que colocar los sensores para obtener un robot que empuje correctamente una pelota de baloncesto a lo largo de una línea recta. ¿Cómo compararía su secuencia de experimentos con un sistema evolutivo?
57. ¿Puede un sistema de inteligencia artificial predecir la situación financiera de una bolsa de valores? En caso afirmativo, ¿cómo de complejo sería el sistema? ¿Qué distintas restricciones deberían tenerse en consideración?

Cuestiones sociales

Las siguientes cuestiones pretenden ser una guía para los problemas éticos/sociales/legales asociados con el campo de la computación. El objetivo no es responder simplemente a estas cuestiones. El lector debería considerar también por qué las ha contestado de la forma en que lo ha hecho y analizar si sus justificaciones son coherentes entre las distintas cuestiones.

1. ¿Hasta qué grado deben ser considerados responsables de la forma en que se utilicen los resultados de su trabajo los investigadores en energía nuclear, en ingeniería genética y en inteligencia artificial? ¿Son los científicos responsables de los conocimientos que adquirimos gracias a sus investigaciones? ¿Qué pasa si esos conocimientos son una consecuencia inesperada de sus investigaciones?
2. ¿Cómo diferenciaría entre la inteligencia y la inteligencia simulada? ¿Existe alguna diferencia?
3. Suponga que un sistema experto computerizado para aplicaciones médicas se labra una gran reputación dentro de la comunidad médica por proporcionar consejos correctos. ¿Hasta qué punto debería un médico permitir que ese sistema alterara sus decisiones relativas a los tratamientos de los pacientes? Si el médico aplica un tratamiento contrario al propuesto por el sistema experto y resulta que el sistema tenía razón, ¿se puede acusar de mala praxis a ese médico? En general, si un sistema experto llega a ser muy conocido dentro de un campo, ¿hasta qué grado podría amenazar, en lugar de mejorar, la capacidad de los expertos humanos para llegar a sus propias conclusiones?
4. Muchas personas argumentan que las acciones de una computadora son simplemente consecuencias del modo en que fue programada, por lo que la computadora no puede poseer libre albedrío. Por ello, una computadora no puede ser considerada responsable de sus acciones. ¿Cree que la mente humana es una computadora? ¿Están los humanos preprogramados desde su nacimiento? ¿Estamos los humanos programados por nuestro entorno? ¿Somos los humanos responsables de nuestras acciones?
5. ¿Existen caminos en los que la ciencia no debería adentrarse aún cuando sea capaz de hacerlo? Por ejemplo, si llegara a ser posible construir una máquina con capacidades de percepción y razonamiento comparables a las de los seres humanos, ¿sería apropiada la construcción de una de tales máquinas? ¿Qué problemas plantearía la existencia de una de esas máquinas? ¿Podría citar algunos de los problemas planteados por los avances en otros campos de la ciencia?
6. La historia está llena de casos en los que el trabajo de científicos y artistas se vió afectado por influencias políticas, religiosas u otras influencias sociales presentes en aquel momento. ¿En qué forma están ese tipo de cuestiones afectando a las actuales tareas de investigación científica? ¿Y qué sucede con las Ciencias de la computación en particular?
7. Muchos países actuales aceptan al menos una parte de responsabilidad a la hora de ayudar a formar de nuevo a aquellas personas cuyos trabajos han

dejado de ser necesarios a causa de los avances de la tecnología. ¿Qué debería/podría hacer la sociedad a medida que la tecnología haga que cada vez más capacidades de los seres humanos sean innecesarias?

8. Suponga que recibimos una factura generada por computadora por un importe de 0,00 euros. ¿Qué deberíamos hacer? Suponga que no hacemos nada y que 30 días después recibimos un segundo aviso de que van a cargar 0,00 euros en nuestra cuenta. ¿Qué deberíamos hacer? Suponga que no hacemos nada y que 30 días después recibimos otro aviso de que se han cargado 0,00 euros en nuestra cuenta junto con una nota que indica que a menos que la factura se pague con prontitud se emprenderán acciones legales. ¿Quién se supone que está a cargo de las cosas?
9. ¿Ha sentido la tentación alguna vez de asociar una personalidad con su computadora personal? ¿Cree que hay veces en las que la computadora parece vengativa o tozuda? ¿Alguna vez se ha enfadado con su computadora? ¿Cuál es la diferencia entre enfadarse *con* una computadora y enfadarse *como resultado de lo que la computadora hace*? ¿Cree que su computadora se ha enfadado alguna vez con usted? ¿Ha experimentado alguna vez una relación similar con otros objetos, como vehículos, televisiones o plumas de escribir?
10. Basándose en sus respuestas a la Cuestión 9, ¿hasta qué grado cree que los seres humanos deseamos asociar el comportamiento de una entidad con la presencia de inteligencia y de consciencia? ¿Hasta qué grado deberían los humanos efectuar ese tipo de asociaciones? ¿Es posible para una entidad inteligente revelar su inteligencia de alguna otra manera que a través de su comportamiento?
11. Muchos expertos piensan que la capacidad de pasar el test de Turing no implica que una máquina sea inteligente. Uno de los argumentos utilizados es que el comportamiento inteligente no implica, por sí mismo, inteligencia. A pesar de ello, la teoría de la evolución está basada en la supervivencia del más adaptado, que es un test basado en el comportamiento. ¿Implica la teoría de la evolución que el comportamiento inteligente es un predecesor de la inteligencia? ¿Implicaría la capacidad de pasar el test de Turing que las máquinas están en camino de volverse inteligentes?
12. El tratamiento médico ha avanzado hasta tal punto que numerosas partes del cuerpo humano pueden ahora sustituirse por partes artificiales o por partes obtenidas de donantes humanos. Ahora es concebible que esto pueda llegar a incluir algún día partes del cerebro. ¿Qué problemas éticos plantearían esas capacidades? Si se sustituyeran las neuronas de un paciente por neuronas artificiales de una en una, ¿seguiría ese paciente siendo la misma persona? ¿Llegaría alguna vez a notar alguna diferencia ese paciente? ¿Seguiría siendo humano ese paciente?
13. Un GPS de un automóvil proporciona una voz amigable que informa al conductor de los giros próximos y de otras acciones. En caso de que el conductor cometa un error, el GPS realizará ajustes automáticamente y proporcionará instrucciones para volver a la ruta prevista, sin mostrar ninguna emoción indebida. ¿Cree que un GPS reduce el estrés de un conductor a la

hora de conducir hasta un nuevo destino? ¿En qué formas contribuye un GPS al estrés?

14. Suponga que su teléfono inteligente proporcionara traducción de idiomas voz a voz. ¿Se sentiría cómodo utilizando esta funcionalidad? ¿Confiaría en que fuera capaz de transmitir el significado correcto de sus palabras? ¿Provocaría en usted algún tipo de preocupación?

Lecturas adicionales

Banzhaf, W., P. Nordin, R. E. Deller y F. D. Francone. *Genetic Programming: An Introduction*. San Francisco, CA: Morgan Kaufmann, 1998.

Lu, J. y J. Wu. *Multi-Agent Robotic Systems*. Boca Raton, FL: CRC Press, 2001.

Luger, G. F. *Artificial Intelligence: Structures and Strategies for Complex Problem Solving*, 5ª ed. Boston, MA: Addison-Wesley, 2005.

Mitchell, M. *An Introduction to Genetic Algorithms*. Cambridge, MA: MIT Press, 1998.

Negnevitsky, M. *Artificial Intelligence: A Guide to Intelligent Systems*, 2ª ed. Boston, MA: Addison-Wesley, 2005.

Nilsson, N. *Artificial Intelligence: A New Synthesis*. San Francisco, CA: Morgan Kaufmann, 1998.

Nolfi, S. y D. Floreano. *Evolutionary Robotics*. Cambridge, MA: MIT Press, 2000.

Rumelhart, D. E. y J. L. McClelland. *Parallel Distributed Processing*. Cambridge, MA: MIT Press, 1986.

Russell, S. y P. Norvig. *Artificial Intelligence: A Modern Approach*, 3ª ed. Upper Saddle River, NJ: Prentice-Hall, 2009.

Shapiro, L. G. y G. C. Stockman. *Computer Vision*. Englewood Cliffs, NJ: Prentice-Hall, 2001.

Shieber, S. *The Turing Test*. Cambridge, MA: MIT Press, 2004.

Weizenbaum, J. *Computer Power and Human Reason*. Nueva York: W. H. Freeman, 1979.

Teoría de la computación

En este capítulo vamos a considerar los fundamentos teóricos de las Ciencias de la computación. En cierto sentido, es el material presentado en este capítulo el que proporciona a las Ciencias de la computación la consideración de verdadera ciencia. Aunque de una naturaleza algo abstracta, este cuerpo de conocimientos tiene muchas aplicaciones de carácter eminentemente práctico. En particular, exploraremos sus implicaciones en relación con la potencia de los lenguajes de programación y veremos cómo conducen a la definición de un sistema de cifrado de clave pública ampliamente utilizado para las comunicaciones a través de Internet.

12.1 Funciones y su computabilidad

12.2 Máquinas de Turing

Fundamentos de la máquina de Turing
La tesis de Church–Turing

12.3 Lenguajes de programación universales

El lenguaje de Bare Bones
Programación en Bare Bones
La universalidad de Bare Bones

12.4 Una función no computable

El problema de la detención
La irresolubilidad del problema de la detención

12.5 Complejidad de los problemas

Medida de la complejidad de un problema
Problemas polinómicos y no polinómicos
Problemas NP

*12.6 Criptografía de clave pública

Notación modular
Criptografía de clave pública RSA

**Las secciones marcadas con asterisco se sugieren como secciones opcionales.*

En este capítulo vamos a considerar una serie de cuestiones relativas a lo que las computadoras pueden o no pueden hacer. Veremos cómo hay unas máquinas simples, denominadas máquinas de Turing, que se utilizan para identificar la frontera entre problemas resolubles mediante máquinas y problemas que no lo son. Analizaremos un problema concreto, conocido con el nombre de problema de la detención, cuya solución escapa a la capacidad de los sistemas algorítmicos y cae, por tanto, fuera de la capacidad de las computadoras tanto actuales como futuras. Además, veremos que incluso entre el conjunto de problemas resolubles mediante máquinas, existen problemas cuya solución es tan compleja que podemos considerarlos irresolubles desde el punto de vista práctico. Cerraremos el capítulo analizando cómo esos conocimientos en el área de la complejidad pueden utilizarse para construir un sistema de cifrado de clave pública.

12.1 Funciones y su computabilidad

Nuestro objetivo en este capítulo es investigar las capacidades de las computadoras. Queremos entender lo que las máquinas pueden o no pueden hacer y qué características se requieren para que las máquinas puedan desarrollar todo su potencial. Comenzaremos con el concepto de computación de funciones.

Una **función** en su sentido matemático es una correspondencia entre un conjunto de posibles valores de entrada y un conjunto de valores de salida, de modo que a cada una de las posibles entradas se le asigna una única salida. Un ejemplo sería la función que convierte las medidas en yardas a metros. A cada medida en yardas, la función le asigna un valor que es el que se obtendría si se midiera esa misma distancia en metros. Otro ejemplo, que podríamos denominar función de ordenación, asigna a cada lista de entrada compuesta por valores numéricos una lista de salida cuyas entradas son las mismas que las de la lista de entrada pero dispuestas en orden creciente. Otro ejemplo sería la función suma cuyas entradas son parejas de valores y cuyas salidas son los valores que representan la suma de cada pareja de entrada.

El proceso de determinar el valor concreto de salida que una cierta función asigna a una entrada determinada se denomina *computación de la función*. La habilidad de calcular funciones es importante, porque si somos capaces de

Teoría de funciones recursivas

Nada molesta más a la naturaleza humana que el que nos digan que no se puede hacer algo. Una vez que los investigadores comenzaron a identificar problemas que son irresolubles, en el sentido de que no disponen de solución algorítmica, otras personas comenzaron a estudiar esos problemas intentando comprender su complejidad. Hoy día, esta área de investigación es una de las partes principales de un tema conocido con el nombre de teoría de funciones recursivas, y hemos aprendido muchas cosas acerca de estos problemas superdifíciles. De hecho, al igual que los matemáticos han desarrollado sistemas de numeración que permiten revelar niveles "cuantitativos" más allá del infinito, los teóricos de las funciones recursivas han descubierto múltiples niveles de complejidad dentro de problemas que yacen mucho más allá de las capacidades de los algoritmos.

resolver los problemas es por medio de ese proceso de computar funciones. Para resolver un problema de suma debemos calcular la función suma, para ordenar una lista debemos computar la función de ordenación. Por ello, una de las tareas fundamentales de las Ciencias de la computación consiste en encontrar técnicas para computar las funciones que subyacen a los problemas que deseamos resolver.

Considere, por ejemplo, un sistema en el que las entradas y salidas de una función puedan ser predeterminadas y anotadas en una tabla. Cada vez que se necesite conocer la salida de la función, basta con buscar las entradas dadas en la tabla, donde podremos ver cuál es la salida requerida. Por tanto, la computación de esa función se reduciría al problema de buscar en la tabla. Dichos sistemas son cómodos, pero su potencia es limitada, porque muchas funciones no pueden representarse de manera completa en una tabla. Un ejemplo es el que se muestra en la Figura 12.1, que es un intento de visualizar la función que convierte las medidas en yardas en sus medidas equivalentes en metros. Puesto que no existe ningún límite a la lista de posibles parejas de entrada/salida, la tabla siempre estará obligatoriamente incompleta.

Una técnica más potente para el cálculo de funciones consiste en seguir las directrices proporcionadas por una fórmula algebraica, en lugar de tratar de visualizar en una tabla todas las posibles combinaciones de entrada/salida. Por ejemplo, podemos utilizar la fórmula algebraica

$$V = P(1 + r)^n$$

para describir cómo calcular el valor de una inversión P después de aplicar una tasa de interés anual compuesto de r durante n años.

Pero la potencia expresiva de las fórmulas algebraicas también tiene sus limitaciones. Existen funciones cuyas relaciones de entrada/salida son demasiado complejas como para poderlas describir mediante manipulaciones algebraicas. Como ejemplos podríamos citar las funciones trigonométricas como el seno o el coseno. Si nos obligan a calcular el seno de 38 grados, podemos dibujar el triángulo apropiado, medir sus lados y calcular la relación deseada, un proceso que no se puede expresar en términos de manipulaciones algebraicas del valor 38. Nuestra calculadora de bolsillo también tiene que realizar malabarismos para resolver la tarea de calcular el seno de 38 grados. En realidad, está

Figura 12.1 Un intento de mostrar la función que convierte las medidas en yardas en sus medidas equivalentes en metros.

Yardas (entrada)	Metros (salida)
1	0,9144
2	1,8288
3	2,7432
4	3,6576
5	4,5720
⋮	⋮
⋮	⋮
⋮	⋮

obligada a aplicar técnicas matemáticas bastante sofisticadas con el fin de obtener una muy buena aproximación al seno de 38 grados y es esa aproximación lo que nos da como respuesta.

Podemos ver, por tanto, que a medida que consideramos funciones de complejidad creciente, nos vemos obligados a aplicar técnicas más potentes para computarlas. La cuestión que nos planteamos es si siempre podremos encontrar un sistema para computar las funciones, independientemente de su complejidad. La respuesta a esta pregunta es no. Un sorprendente resultado de las matemáticas es que existen funciones tan complejas que no existe ningún proceso paso a paso bien definido para determinar sus salidas a partir de sus valores de entrada. Por ello, la computación de esas funciones cae más allá de las capacidades de cualquier sistema algorítmico. De esas funciones decimos que son no computables, mientras que las funciones cuyos valores de salida se pueden determinar algorítmicamente a partir de sus valores de entrada se denominan **computables**.

La distinción entre funciones computables y no computables es muy importante en las Ciencias de la computación. Puesto que las máquinas solo pueden realizar tareas descritas mediante algoritmos, el estudio de las funciones computables es el estudio de los límites de las capacidades de las máquinas. Si podemos identificar capacidades que permitan a una máquina computar el conjunto completo de funciones computables y luego construimos máquinas con estas capacidades, podremos estar seguros de que las máquinas que construyamos son lo más potentes posible. De la misma forma, si descubrimos que la solución a un problema requiere la computación de una función no computable, podemos concluir que la solución de ese problema cae más allá de las capacidades de las máquinas.

Cuestiones y ejercicios

1. Identifique algunas funciones que puedan representarse completamente en forma tabular.
2. Identifique algunas funciones cuyas salidas puedan describirse en forma de una expresión algebraica en la que se incluyan sus entradas.
3. Identifique una función que no pueda describirse en términos de una fórmula algebraica. ¿Es esa función, de todos modos, computable?
4. Los antiguos matemáticos griegos utilizaban una regla y un compás para dibujar formas geométricas. Desarrollaron técnicas para hallar el punto medio de una recta, construir un ángulo recto y dibujar un triángulo equilátero. Sin embargo, ¿podría citar algunas "computaciones" que su "sistema computacional" no pudiera realizar?

12.2 Máquinas de Turing

En un intento de comprender las capacidades y limitaciones de las máquinas, muchos investigadores han propuesto y estudiado diversos dispositivos computacionales. Uno de ellos es la máquina de Turing, que fue propuesta por Alan

Orígenes de las máquinas de Turing

Alan Turing desarrolló el concepto de máquina de Turing en la década de 1930, mucho antes de que la tecnología fuera capaz de proporcionarnos las técnicas que conocemos hoy día. De hecho, la visión de Turing era la de un ser humano realizando los cálculos con lápiz y papel. El objetivo de Turing era el de proporcionar un modelo con el que estudiar los límites de los "procesos computacionales". Esto fue poco después de la publicación en 1931 del famoso artículo de Gödel en el que se exponían las limitaciones de los sistemas computacionales, y grandes esfuerzos de investigación se dirigieron a intentar comprender esas limitaciones. En el mismo año en que Turing presentó su modelo (1936), Emil Post presentó otro modelo (ahora conocido con el nombre de sistemas de producción de Post) que ha demostrado tener las mismas capacidades que el de Turing. Como testimonio de la visión de estos primeros investigadores, sus primeros modelos de sistemas computacionales (como las máquinas de Turing y los sistemas de producción de Post) siguen sirviendo como herramientas muy valiosas en las investigaciones efectuadas en el campo de las Ciencias de la computación.

M. Turing en 1936 y continúa utilizándose hoy día para estudiar la potencia de los procesos algorítmicos.

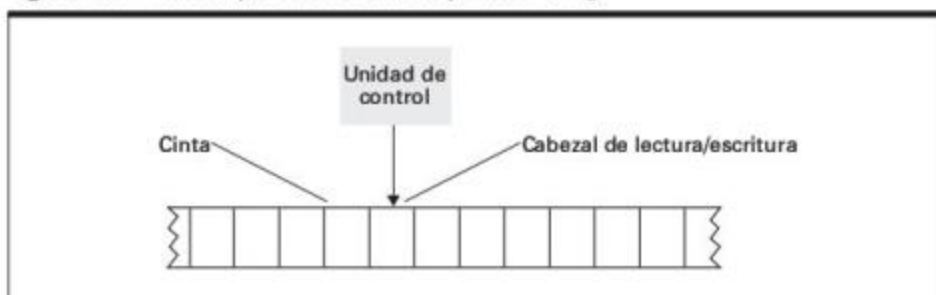
Fundamentos de la máquina de Turing

Una **máquina de Turing** está compuesta por una unidad de control que puede leer y escribir símbolos en una cinta por medio de un cabezal de lectura/escritura (Figura 12.2). La cinta se extiende indefinidamente por ambos extremos y está dividida en casillas, cada una de las cuales puede contener un símbolo de entre un conjunto finito de ellos. Este conjunto se denomina alfabeto de la máquina.

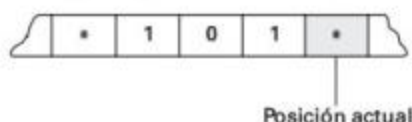
En todo momento durante los cálculos realizados con una máquina de Turing, la máquina debe estar en una de entre un número finito de condiciones, denominados estados. La computación efectuada por una máquina de Turing comienza en un estado especial denominado estado inicial y termina cuando la máquina alcanza otro estado especial conocido con el nombre de estado de detención.

La computación efectuada por una máquina de Turing consiste en una secuencia de pasos ejecutados por la unidad de control de la máquina. Cada paso consiste en observar el símbolo escrito en la casilla actual de la cinta (la que está viendo el cabezal de lectura/escritura), escribir un símbolo en esa casilla, mover posiblemente el cabezal de lectura/escritura una casilla hacia la izquierda o hacia la derecha y luego cambiar de estado. La acción concreta que hay que realizar está determinada por un programa que le dice a la unidad de control lo que hay que hacer basándose en el estado de la máquina y en el contenido de la casilla actual de la cinta.

Veamos un ejemplo de una máquina de Turing específica. Con este objetivo, vamos a representar la cinta de la máquina como una banda horizontal dividida en casillas en las que podemos anotar símbolos pertenecientes al alfabeto de la máquina. Indicaremos la posición actual del cabezal de lectura/escritura

Figure 12.2 Los componentes de una máquina de Turing.

tura de la máquina colocando una etiqueta alusiva debajo de la casilla actual de la cinta. El alfabeto para nuestro ejemplo estará compuesto por los símbolos 0, 1 y *. La cinta de nuestra máquina podría tener la siguiente apariencia:



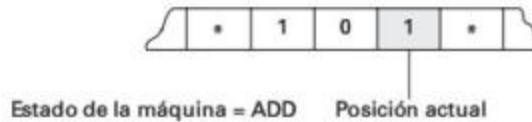
Interpretando una cadena de símbolos de la cinta como si representara una serie de números binarios separados por asteriscos, podemos reconocer que esta cinta concreta contiene el valor 5. Nuestra máquina de Turing está diseñada para incrementar en una unidad ese valor de la cinta. Para ser más precisos, vamos a suponer que en la posición inicial hay un asterisco que marca el extremo derecho de una cadena de 0s y 1s, y que la cinta debe modificar el patrón de bits situado a su izquierda, para representar el siguiente entero.

Los estados de nuestra máquina serán START (estado inicial), ADD (sumar), CARRY (acarreo), OVERFLOW (desbordamiento), RETURN (volver) y HALT (detención). Las acciones correspondientes a cada uno de estos estados y el contenido de la casilla correspondiente se describen en la tabla de la Figura 12.3. Vamos a suponer que la máquina comienza siempre en el estado START.

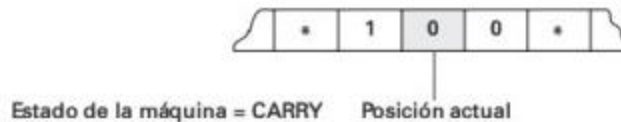
Figura 12.3 Una máquina de Turing para incrementar un valor.

Estado actual	Contenido de la casilla actual	Valor que hay que escribir	Dirección de movimiento	Estado al que pasar
START	*	*	Izquierda	ADD
ADD	0	1	Derecha	RETURN
ADD	1	0	Izquierda	CARRY
ADD	*	*	Derecha	HALT
CARRY	0	1	Derecha	RETURN
CARRY	1	0	Izquierda	CARRY
CARRY	*	1	Izquierda	OVERFLOW
OVERFLOW	(Ignorado)	*	Derecha	RETURN
RETURN	0	0	Derecha	RETURN
RETURN	1	1	Derecha	RETURN
RETURN	*	*	Sin movimiento	HALT

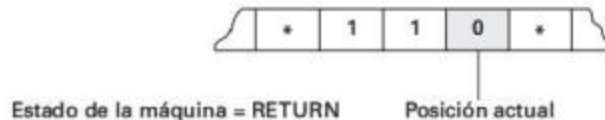
Vamos a aplicar esta máquina a la cinta que hemos mostrado anteriormente que contenía el valor 5. Observe que, cuando nos encontramos con el estado *START* con la casilla actual conteniendo * (como es nuestro caso), la tabla nos dice que hay que reescribir el *, mover el cabezal de lectura/escritura una casilla hacia la izquierda y entrar en el estado *ADD*. Habiéndose hecho esto, la máquina se puede describir como sigue:



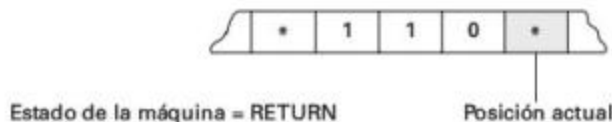
Para continuar, examinamos la tabla para ver qué es lo que tenemos que hacer cuando estamos en estado *ADD* y la casilla actual contiene un 1. La tabla dice que hay que sustituir el 1 de la casilla actual por un 0, mover el cabezal de lectura/escritura una casilla hacia la izquierda y entrar en el estado *CARRY*. Por tanto, la configuración de la máquina pasará a ser:



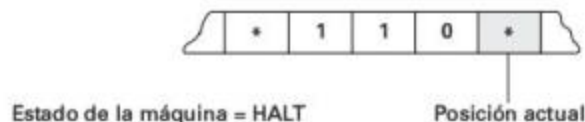
Consultamos de nuevo la tabla para ver qué hay que hacer a continuación y vemos que cuando nos encontramos en el estado *CARRY* y la casilla actual contiene un 0, tenemos que sustituir el 0 por un 1, mover el cabezal de lectura/escritura una casilla hacia la derecha y entrar en el estado *RETURN*. Después de esto, la configuración de la máquina será la siguiente:



A partir de aquí, la tabla nos dice que debemos continuar sustituyendo el 0 de la casilla actual por otro 0, mover el cabezal de lectura/escritura una casilla hacia la derecha y permanecer en el estado *RETURN*. En consecuencia, nos encontraremos nuestra máquina en la siguiente condición:



En este punto, vemos que la tabla nos dice que hay que reescribir el asterisco en la casilla actual y entrar en el estado *HALT*. La máquina se parará así en la siguiente configuración (los símbolos de la cinta representan ahora el valor 6, como deseábamos):



La tesis de Church-Turing

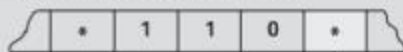
La máquina de Turing del ejemplo anterior se puede utilizar para calcular la función conocida con el nombre de función sucesora, que asigna a cada entero no negativo n que se le proporcione como valor de entrada, un valor de salida igual a $n + 1$. Simplemente necesitamos colocar el valor de entrada en formato binario en la cinta de la máquina, operar la máquina hasta que se detenga y luego leer el valor de salida de la cinta. Toda función que pueda computarse de esta forma mediante una máquina de Turing se dice que es **computable según Turing**.

La conjetura de Turing era que el conjunto de las funciones computables según Turing coincide con el conjunto de todas las funciones computables. En otras palabras, conjeturó que la potencia computacional de la máquina de Turing incluye la de cualquier sistema algorítmico o, lo que es lo mismo, que (a diferencia de otros enfoques, como los de las tablas o las fórmulas algebraicas) el concepto de máquina de Turing proporciona un contexto con el que expresar las soluciones para todas las funciones computables. Hoy día, esta conjetura se denomina a menudo **tesis de Church-Turing**, en referencia a las contribuciones hechas tanto por Alan Turing como por Alonzo Church. Desde el trabajo inicial de Turing, se han recopilado muchas pruebas para apoyar esta tesis, y hoy día la tesis de Church-Turing es ampliamente aceptada. Es decir, se considera que el conjunto de las funciones computables y el conjunto de las funciones computables según Turing coinciden.

La importancia de esta conjetura es que nos proporciona información acerca de las capacidades y limitaciones de las máquinas de computación. De forma más precisa, establece las capacidades de las máquinas de Turing como un estándar con el que comparar la potencia de otros sistemas computacionales. Si un sistema computacional es capaz de calcular todas las funciones computables según Turing, se considera que su potencia es igual a la máxima posible que cualquier sistema computacional puede ofrecer.

Cuestiones y ejercicios

1. Aplique la máquina de Turing descrita en esta sección (Figura 12.3), comenzando a partir del siguiente estado inicial:



Estado de la máquina = START

Posición actual

2. Describa una máquina de Turing que sustituya una cadena de 0s y 1s por un único 0.
3. Describa una máquina de Turing que decremente en una unidad el valor de la cinta si este es mayor que cero o deje el valor como está si es igual a cero.
4. Identifique una situación cotidiana en la que se realice algún tipo de cálculo. ¿En qué sentido es análoga esa situación a una máquina de Turing?

5. Describa una máquina de Turing que termine por detenerse para algunas entradas pero que nunca se detenga para otras.

12.3 Lenguajes de programación universales

En el Capítulo 6 hemos estudiado diversas características que podemos encontrar en los lenguajes de programación de alto nivel. En esta sección vamos a aplicar nuestros conocimientos acerca de la computabilidad para determinar cuáles de esas características son realmente necesarias. Veremos que la mayoría de las características de los lenguajes de alto nivel actuales simplemente los hacen más cómodos sin por ello contribuir a aumentar la potencia fundamental del lenguaje.

Nuestro enfoque consistirá en describir un lenguaje de programación simple de carácter imperativo que sea lo suficientemente rico como para permitirnos expresar programas que sirvan para calcular todas las funciones computables según Turing (y por tanto todas las funciones computables). Por tanto, si un futuro programador comprueba que un problema no puede resolverse utilizando este lenguaje, la razón no será un fallo del lenguaje, sino más bien que no existe ningún algoritmo para resolver el problema. Un lenguaje de programación con esta propiedad se denomina **lenguaje de programación universal**.

Puede que el lector encuentre sorprendente el hecho de que un lenguaje universal no necesite ser complejo. De hecho, el lenguaje que vamos a presentar es bastante simple. Lo llamaremos Bare Bones (N. del T.: que podría traducirse como lenguaje mínimo) porque aísla el conjunto mínimo de requisitos que es necesario imponer a un lenguaje de programación universal.

El lenguaje de Bare Bones

Comenzamos nuestra presentación de Bare Bones considerando las sentencias declarativas que podemos encontrar en otros lenguajes de programación. Estas sentencias permiten a los programadores darse el lujo de pensar en términos de estructuras de datos y de tipos de datos (tales como matrices de valores numéricos y cadenas de caracteres alfabéticos), aunque la propia máquina se limite a manipular los patrones de bits sin tener ningún conocimiento de qué es lo que esos patrones representan. Antes de presentarla a una máquina para que la ejecute, una sentencia de alto nivel que trate con estructuras y tipos de datos elaborados debe ser traducida a instrucciones de lenguaje máquina que manipulen patrones de bits, con el fin de simular las acciones solicitadas.

Por comodidad, podemos interpretar esos patrones como valores numéricos representados en notación binaria. Por tanto, todos los cálculos realizados por una computadora podrían expresarse como cálculos numéricos con enteros no negativos (como vemos todo depende de cómo se miren las cosas). Además, los lenguajes de programación se podrían simplificar exigiendo a los programadores que expresaran los algoritmos en estos términos (aunque esto obligaría al programador a realizar una tediosa labor).

Puesto que nuestro objetivo al desarrollar Bare Bones es desarrollar el lenguaje más simple posible, vamos a seguir precisamente este camino: consideraremos que todas las variables de Bare Bones representan patrones de bits que, por comodidad, interpretaremos como enteros no negativos en notación binaria. Por tanto, si una variable tiene actualmente asignado el patrón 10 diremos que contiene el valor dos, mientras que si tiene asignado el patrón 101 diremos que contiene el valor cinco.

Utilizando este convenio, todas las variables de un programa Bare Bones son del mismo tipo, por lo que el lenguaje no necesita sentencias declarativas para describir los nombres de las variables y sus propiedades asociadas. Al emplear Bare Bones, un programador puede simplemente comenzar a utilizar un nuevo nombre de variable en el momento que sea necesario, asumiendo que hace referencia a un patrón de bits interpretado como un entero no negativo.

Por supuesto, un traductor para nuestro lenguaje Bare Bones deberá ser capaz de distinguir los nombres de variables de otros términos. Esto se hace diseñando la sintaxis de Bare Bones de modo que el papel de cualquier término pueda identificarse a partir de la propia sintaxis. Con este objeto, decidimos que los nombres de las variables deben comenzar con una letra del alfabeto inglés, que puede ir seguida por cualquier combinación de letras y dígitos (0 a 9). Por tanto, las cadenas XYZ, B747, abcdefghi y X5Y pueden utilizarse como nombres de variable, mientras que 2G5, %o y x. y no.

Consideremos ahora las sentencias procedimentales de Bare Bones. Existen tres sentencias de asignación y una estructura de control que representa un bucle. El lenguaje es un lenguaje de formato libre, por lo que cada sentencia termina con un punto y coma, lo que facilita al traductor la tarea de separar las sentencias que aparezcan en la misma línea. Sin embargo, nosotros adoptaremos la política de escribir una única sentencia por línea con el fin de mejorar la legibilidad de los programas.

Cada una de las tres sentencias de asignación solicita que se modifique el contenido de la variable identificada en la sentencia. La primera de ellas nos permite asociar el valor cero con una variable. Su sintaxis es

```
clear nombre;
```

donde *nombre* puede ser cualquier nombre de variable.

Las otras sentencias de asignación son esencialmente opuestas entre sí:

```
incr nombre;
```

y

```
decr nombre;
```

De nuevo, *nombre* representa cualquier nombre de variable. La primera de estas sentencias hace que el valor asociado con la variable identificada se incremente en una unidad. Por tanto, si la variable Y tuviera asignado el valor cinco antes de ejecutar la sentencia

```
incr Y;
```

entonces el valor asignado a Y después de ejecutada la sentencia sería seis.

Por el contrario, la sentencia *decr* se utiliza para decrementar en una unidad el valor asociado con la variable identificada. Una excepción es cuando la variable ya tiene asociado el valor cero, en cuyo caso la sentencia no modifica

el valor. Por tanto, si el valor asociado con Y es cinco antes de ejecutar la sentencia

```
decr Y;
```

entonces el valor asociado con Y después de ejecutar la sentencia será cuatro. Sin embargo, si el valor de Y fuera cero antes de ejecutar la sentencia, ese valor continuaría siendo cero después de ejecutarla.

Bare Bones solo proporciona una estructura de control representada por una pareja de sentencias `while`-`end`. La secuencia de sentencias

```
while nombre not 0 do;
.
.
.
end;
```

(donde *nombre* representa cualquier nombre de variable) hace que cualquier sentencia o secuencia de sentencias colocada entre las sentencias `while` y `end` se repita mientras que el valor de la variable *nombre* sea distinto de cero. Para ser más precisos, cuando nos encontramos una estructura `while`-`end` durante la ejecución del programa, se comprueba en primer lugar si el valor de la variable identificada es cero. Si lo es, nos saltamos la estructura y la ejecución continúa en la instrucción que se encuentra después de la instrucción `end`. Sin embargo, si el valor de la variable es distinto de cero, se ejecuta la secuencia de sentencias contenida dentro de la estructura `while`-`end` y se devuelve el control a la sentencia `while`, donde se vuelve a comparar el valor con cero. Observe que será parcialmente responsabilidad del programador toda la complejidad asociada con el control del bucle, ya que la sentencia deberá hacer explícitamente que se modifique el valor de la variable dentro del cuerpo del bucle con el fin de evitar un bucle infinito. Por ejemplo, la secuencia

```
incr X;
while X not 0 do;
  incr Z;
end;
```

da como resultado un proceso infinito, porque una vez alcanzada la sentencia `while`, el valor asociado con X nunca puede ser cero, mientras que la secuencia

```
clear Z;
while X not 0 do;
  incr Z;
  decr X;
end;
```

llegará a terminar, con el efecto de transferir a la variable Z el valor inicialmente asociado con X.

Observe que las sentencias `while` y `end` deben aparecer emparejadas, y que la sentencia `while` debe aparecer primero. Sin embargo, una pareja de sentencias `while`-`end` puede aparecer dentro de las sentencias ejecutadas por otra pareja de sentencias `while`-`end`. En dicho caso, el emparejamiento de sentencias `while` y `end` se lleva a cabo analizando el programa en su forma escrita, de principio a fin, y asociando cada sentencia `end` con la sentencia `while` prece-

dente más próxima que todavía no haya sido emparejada. Aunque no es sintácticamente necesario, utilizaremos a menudo el sangrado para mejorar la legibilidad de tales estructuras.

Como ejemplo final, la secuencia de sentencias de la Figura 12.4 hace que se asigne a Z el producto de los valores asociados con X e Y , aunque tiene el efecto colateral de destruir cualquier valor distinto de cero que pudiera haber estado asociado con X . (La estructura `while-end` controlada por la variable W tiene el efecto de restaurar el valor original de Y .)

Programación en Bare Bones

Recuerde que nuestro objetivo al presentar el lenguaje Bare Bones es investigar qué cosas son posibles; no qué cosas resultan prácticas. Bare Bones sería terrible de utilizar en el entorno de una aplicación real. Por otro lado, pronto veremos que este lenguaje tan simple satisface completamente nuestro objetivo de disponer de un lenguaje de programación universal mínimo. Por ahora, simplemente vamos a mostrar cómo puede utilizarse Bare Bones para expresar algunas operaciones elementales.

En primer lugar, observemos que combinando las sentencias de asignación, cualquier valor (cualquier entero no negativo) se puede asociar con una variable dada. Por ejemplo, la siguiente secuencia asigna el valor 3 a la variable X asignándole en primer lugar el valor cero y luego incrementando su valor tres veces:

```
clear X;
incr X;
incr X;
incr X;
```

Otra actividad muy común en los programas consiste en copiar datos de una ubicación a otra. En términos de Bare Bones, esto significa que tenemos que poder asignar el valor de una variable a otra variable. Esto se puede hacer poniendo a cero primero la variable de destino y luego incrementándola un número apropiado de veces. De hecho, ya hemos observado que la secuencia

Figura 12.4 Un programa en Bare Bones para el cálculo de $X \times Y$.

```
clear Z;
while X not 0 do;
  clear W;
  while Y not 0 do;
    incr Z;
    incr W;
    decr Y;
  end;
  while W not 0 do;
    incr Y;
    decr W;
  end;
  decr X;
end;
```



```

clear Z;
while X not 0 do;
  incr Z;
  decr X;
end;

```

transfiere a Z el valor asociado con X. Sin embargo, esta secuencia tiene el efecto colateral de destruir el valor original de X. Para corregir esto, podemos introducir una variable auxiliar a la que transferiremos primero el valor en cuestión desde su ubicación inicial. Luego, emplearemos esta variable auxiliar como origen de datos mediante el que restaurar la variable original, al mismo tiempo que colocamos el valor en cuestión en la variable de destino deseada. De esta manera, el movimiento de Hoy a Mañana puede realizarse mediante la secuencia mostrada en la Figura 12.5.

Adoptamos la sintaxis

```
copy nombre1 to nombre2;
```

(donde *nombre1* y *nombre2* representan nombres de variable) como notación abreviada para una estructura de sentencias de la forma mostrada en la Figura 12.5. Así, aunque el propio Bare Bones no tiene una sentencia de copia explícita, a menudo escribiremos programas como si la tuviera, asumiendo que para convertir esos programas informales en verdaderos programas Bare Bones es preciso sustituir las sentencias *copy* por sus estructuras equivalentes *while-end*, utilizando una variable auxiliar cuyo nombre no interfiera con ningún otro nombre utilizado en algún punto del programa.

La universalidad de Bare Bones

Apliquemos ahora la tesis de Church-Turing para confirmar nuestra afirmación de que Bare Bones es un lenguaje de programación universal. En primer lugar, observemos que cualquier programa escrito en Bare Bones puede considerarse como algo que está controlando el cálculo de una función. La entrada de la función está compuesta por los valores asignados a las variables antes de la ejecución del programa y la salida de la función está compuesta por los valores de las variables una vez que el programa termina. Para calcular la función, simplemente ejecutamos el programa comenzando con las asignaciones de variables adecuadas y luego observamos los valores de las variables una vez que el programa ha terminado.

Figura 12.5 Una implementación Bare Bones de la instrucción “copy Hoy to Mañana”.

```

clear Aux;
clear Mañana;
while Hoy not 0 do;
  incr Aux;
  decr Hoy;
end;
while Aux not 0 do;
  incr Hoy;
  incr Mañana;
  decr Aux;
end;

```

Bajo estas condiciones, el programa

```
incr X;
```

controla la computación de la misma función (la función sucesora) que hemos computado mediante el ejemplo de la máquina de Turing en la Sección 12.2. De hecho, incrementa el valor asociado con X en una unidad. De la misma forma, si interpretamos las variables X e Y como entradas y la variable Z como salida, el programa

```
copy Y to Z;
while X not 0 do;
  incr Z;
  decr X;
end;
```

controla el cálculo de la función suma.

Los investigadores han demostrado que el lenguaje de programación Bare Bones puede utilizarse para expresar algoritmos para calcular todas las funciones computables según Turing. Combinando esto con la tesis de Church-Turing, vemos que cualquier función computable podrá calcularse mediante un programa escrito en Bare Bones. Por tanto, Bare Bones es un lenguaje de programación universal en el sentido de que si existe un algoritmo para resolver un problema, entonces ese problema puede resolverse mediante algún programa escrito en Bare Bones. A su vez, eso implica que Bare Bones puede servir, desde el punto de vista teórico, como un lenguaje de programación de propósito general.

Decimos *desde el punto de vista teórico* porque un lenguaje así no sería desde luego tan cómodo como los lenguajes de alto nivel presentados en el Capítulo 6. Sin embargo, cada uno de esos lenguajes contiene, esencialmente, las características de Bare Bones como núcleo fundamental. De hecho, es ese núcleo fundamental lo que garantiza la universalidad de cada uno de esos lenguajes; todas las demás características de los distintos lenguajes se incluyen por comodidad.

Aunque no resultan prácticos en un entorno de programación de aplicaciones, los lenguajes como Bare Bones sí que tienen utilidad en las Ciencias teóricas de la computación. Por ejemplo, en el Apéndice E utilizaremos Bare Bones como herramienta para resolver la cuestión relativa a la equivalencia de las estructuras iterativas y recursivas, cuestión que hemos planteado en el Capítulo 5. Allí veremos que nuestra sospecha de que son equivalentes estaba, de hecho, justificada.

Cuestiones y ejercicios

1. Demuestre que la sentencia `invert X`; (cuya acción es convertir el valor de X a 0 si su valor inicial es distinto de cero y a 1 si su valor inicial es cero) se puede simular mediante un segmento de programa Bare Bones.
2. Demuestre que incluso nuestro lenguaje Bare Bones tan simple contiene más sentencias de las necesarias, demostrando que la sentencia `clear` puede sustituirse por combinaciones de otras sentencias del lenguaje.

3. Demuestre que la estructura if-then-else se puede simularse utilizando Bare Bones. Es decir, escriba una secuencia de programa en Bare Bones que simule la acción de la sentencia

```
if X not 0 then S1 else S2;
```

donde S1 y S2 representan secuencias de sentencias arbitrarias.

4. Demuestre que cada una de las sentencias de Bare Bones puede expresarse en términos del lenguaje máquina del Apéndice C. (Por tanto, Bare Bones puede emplearse como lenguaje de programación para esa máquina.)
5. ¿Cómo podríamos tratar los números negativos en Bare Bones?
6. Describa la función computada mediante el siguiente programa Bare Bones, asumiendo que la entrada de la función está representada por X y su salida por Z:

```
clear Z;
while X not 0 do;
  incr Z;
  incr Z;
  decr X;
end;
```

12.4 Una función no computable

Ahora vamos a identificar una función que no es computable según Turing y por tanto, según la tesis de Church-Turing, tampoco será computable en sentido general. Por tanto, es una función cuyo cálculo cae más allá de las capacidades de las computadoras.

El problema de la detención

La función no computable de la que vamos a hablar está asociada con un problema conocido como **problema de la detención**, que (en un sentido informal) es el problema de tratar de predecir de antemano si un programa terminará (o se detendrá) si se inicia bajo ciertas condiciones. Por ejemplo, considere el siguiente programa simple en Bare Bones

```
while X not 0 do;
  incr X;
end;
```

Si ejecutamos este programa con un valor inicial de X igual a cero, el bucle no se ejecutará y la ejecución del programa terminará rápidamente. Sin embargo, si ejecutamos el programa con cualquier otro valor inicial de X, el bucle se ejecutará para siempre, por lo que obtendremos un proceso que no termina.

En este caso, entonces, es fácil concluir que la ejecución del programa se detendrá únicamente si se inicia asignando a X el valor cero. Sin embargo, cuando pasamos a estudiar problemas más complejos, la tarea de predecir el

comportamiento de un programa se vuelve más complicada. De hecho, en algunos casos, la tarea es imposible como veremos. Pero primero necesitamos formalizar nuestra terminología y centrar nuestro análisis de forma más precisa.

Nuestro ejemplo demuestra que el que un programa llegue a detenerse puede depender de los valores iniciales de sus variables. Por tanto, si esperamos poder predecir si la ejecución de un programa se detendrá, necesitamos ser muy precisos en relación a esos valores iniciales. La elección que estamos a punto de hacer con respecto a esos valores podrá parecerle extraña al lector a primera vista, pero no debe desesperarse. Nuestro objetivo es aprovecharnos de una técnica denominada **auto-referencia**, que es la idea de un objeto que se hace referencia a sí mismo. Esta estratagema ha conducido en muchas ocasiones a la obtención de resultados sorprendentes en matemáticas, que van desde curiosidades de tipo informal como la frase "Este enunciado es falso" a esa paradoja, mucho más seria, representada por la cuestión "¿El conjunto de todos los conjuntos se contiene a sí mismo?". Lo que estamos a punto de hacer, por tanto, es definir el escenario para una línea de razonamiento similar a "Si lo hace, entonces no lo hace; pero si no lo hace, entonces lo hace".

En nuestro caso, la auto-referencia se conseguirá asignando a las variables de un programa un valor inicial que representará al propio programa. Con este objetivo, observe que cada programa Bare Bones puede codificarse como un único patrón de bits de gran longitud en un formato en el que se utilice un carácter por byte aplicando el código ASCII, pudiendo después interpretarse ese patrón de bits como la representación binaria de un número entero no negativo (de magnitud bastante grande). Es ese valor entero el que asignaremos como valor inicial de las variables del programa.

Consideremos lo que sucedería si hiciéramos esto en el caso del programa sencillo

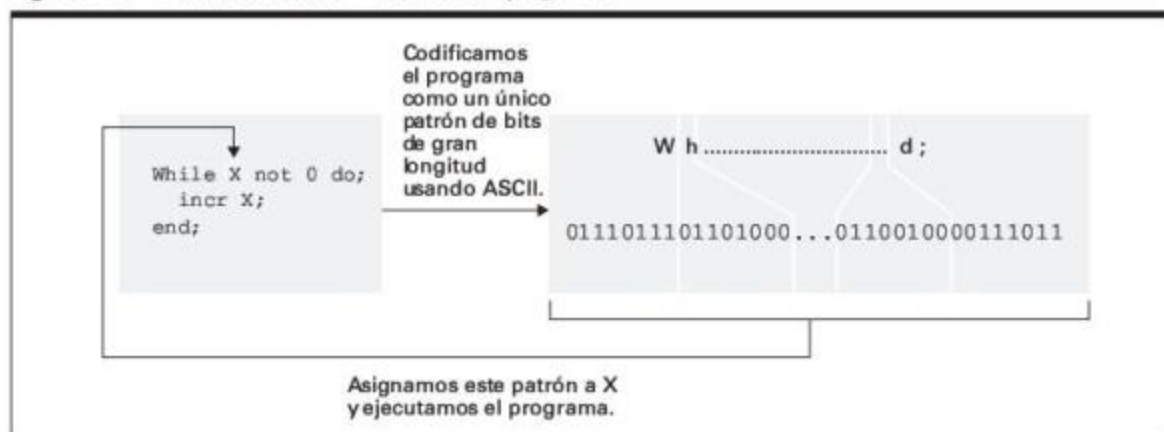
```
while X not 0 do;
  incr X;
end;
```

Deseamos saber lo que sucedería si iniciáramos este programa después de asignar a X el valor entero que represente al propio programa (Figura 12.6). En este caso, la respuesta es inmediata: puesto que X tiene un valor distinto de cero, el programa quedará atrapado en el bucle y nunca terminará. Por otro lado, si hiciéramos un experimento similar con el programa

```
clear X;
while X not 0 do;
  incr X;
end;
```

el programa terminaría, porque la variable X tendría el valor cero en el momento de alcanzar la estructura while-end, independientemente de cuál fuera su valor inicial.

Establezcamos entonces la siguiente definición: un programa Bare Bones es **auto-terminante** si al ejecutar el programa después de inicializar todas sus variables con la propia representación codificada del programa nos da un proceso que finaliza. Informalmente, un programa será auto-terminante si su eje-

Figure 12.6 Prueba de auto-terminación de un programa.

cución termina cuando se inicializa utilizando el propio programa como entrada. Esta es la auto-referencia que habíamos prometido.

Observe que el que un programa sea auto-terminante probablemente no tiene nada que ver con el propósito para el que fue escrito. Se trata simplemente de una propiedad que cada programa en Bare Bones poseerá o no poseerá. Es decir, cada programa Bare Bones será auto-terminante o no lo será.

Ahora podemos describir el problema de la detención de una forma precisa: será el problema de determinar si los programas Bare Bones son o no auto-terminantes. En breve veremos que no existe ningún algoritmo para responder a esta cuestión de carácter general. Es decir, no existe ningún algoritmo que, al darle como entrada cualquier programa Bare Bones sea capaz de determinar si ese programa es o no auto-terminante. Por tanto, la solución al problema de la detención cae más allá de las capacidades de la computadora.

El hecho de que hayamos resuelto aparentemente el problema de la detención en los ejemplos anteriores y ahora digamos que el problema de la detención es irresoluble puede sonar contradictorio, así que hagamos una pausa para clarificar los conceptos. Las observaciones que hemos utilizado en los ejemplos eran aplicables a esos casos particulares y no serían aplicables a todas las situaciones. Lo que el problema de la detención exige es un único algoritmo genérico que pueda aplicarse a cualquier programa Bare Bones para determinar si es auto-terminante. Nuestra capacidad de aplicar ciertos conocimientos aislados para determinar si un programa concreto es auto-terminante, no implica de ninguna forma la existencia de una única solución genérica que pueda aplicarse en todos los casos. En resumen, podemos construir una máquina que pueda resolver el problema de la detención en un caso concreto, pero no podemos construir una única máquina que pudiéramos emplear para resolver el problema de la detención en cualquier situación que surja.

La irresolubilidad del problema de la detención

Ahora queremos demostrar que resolver el problema de la detención cae más allá de las capacidades de las máquinas. Nuestro enfoque consistirá en demostrar que el resolver el problema requeriría un algoritmo para calcular una fun-

ción no computable. Las entradas de la función en cuestión son versiones codificadas de programas Bare Bones; sus salidas están limitadas a los valores 0 y 1. Para ser más precisos, definiremos la función de modo que una entrada que represente un programa auto-terminante genere el valor de salida 1, mientras que una entrada que represente un programa que no sea auto-terminante genere el valor de salida 0. En aras de la concesión, nos referiremos a esta función como *función de detención*.

Nuestra tarea consiste en demostrar que la función de detención no es computable. El enfoque que adoptaremos es una técnica conocida con el nombre de "prueba por contradicción". En resumen, demostraremos que un enunciado es falso demostrando que no puede ser verdadero. Demostramos, por tanto, que el enunciado "La función de detención es computable" no puede ser cierto. Nuestro argumento completo se resume en la Figura 12.7.

Si la función de detención es computable, entonces (como Bare Bones es un lenguaje de programación universal) debe existir un programa Bare Bones que la calcule. En otras palabras, existe un programa Bare Bones que terminará con su salida igual a 1 si su entrada es la versión codificada de un programa auto-terminante, y en caso contrario terminará con su salida igual a 0.

Para aplicar este programa no necesitamos identificar qué variable utilizamos como entrada, sino que basta simplemente con inicializar todas las variables del programa con la representación codificada del programa que hay que probar. Esto se debe a que una variable que no sea una variable de entrada será, inherentemente, una variable cuyo valor inicial no afecta al valor final de salida. Podemos entonces concluir que si la función de detención es computable, entonces existirá un programa Bare Bones que termine con su salida igual a 1 si se inicializan todas las variables con la versión codificada de un programa auto-terminante y que termine con su salida igual a 0 en caso contrario.

Asumiendo que la variable de salida del programa es *X* (si no fuera así podemos simplemente renombrar las variables), podríamos modificar el programa añadiendo las sentencias

```
while X not 0 do;  
end;
```

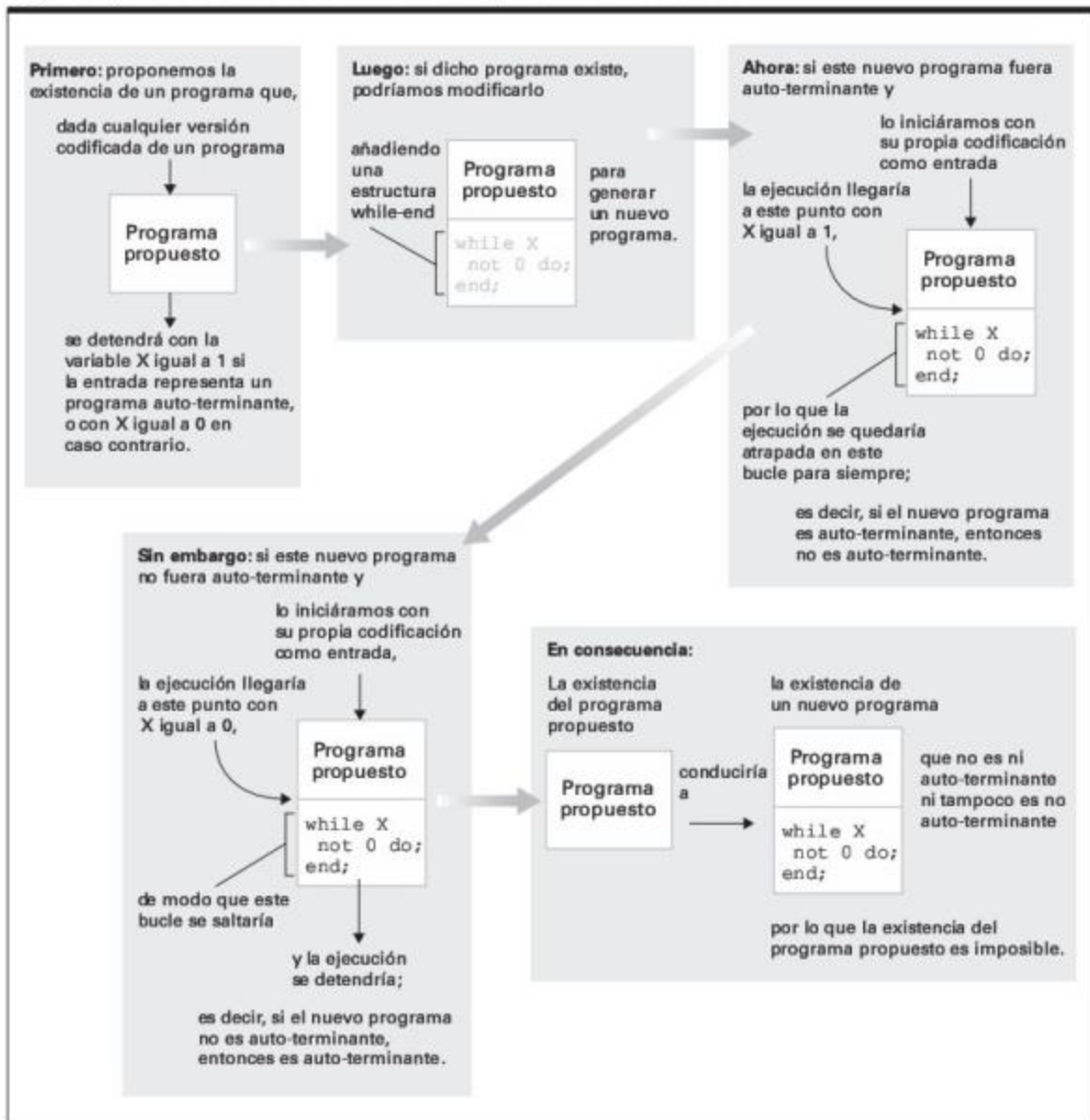
al final, obteniendo así un nuevo programa. Este nuevo programa deberá ser auto-terminante o no. Sin embargo, estamos a punto de ver que no puede ser ninguna de las dos cosas.

En particular, si este nuevo programa fuera auto-terminante y lo ejecutáramos después de inicializar sus variables con la propia representación codificada del programa, entonces cuando su ejecución alcanzara la sentencia *while* que hemos añadido, la variable *X* contendría un 1. (Hasta este punto el nuevo programa es idéntico al programa original que generaba un 1 si su entrada era la representación de un programa auto-terminante.) Llegados a este punto, la ejecución del programa quedaría atrapada para siempre en la estructura *while-end*, porque no hemos incluido ninguna sentencia para decrementar *X* dentro del bucle. Pero esto contradice nuestra suposición de que el nuevo programa es auto-terminante. Por tanto, tenemos que concluir que el nuevo programa no es auto-terminante.

Sin embargo, si el nuevo programa fuera no auto-terminante y lo ejecutáramos después de inicializar sus variables con la representación codificada del

propio programa, llegaría a la sentencia `while` que hemos añadido con un valor 0 asignado a la variable `X`. (Esto ocurre porque las sentencias que preceden a la sentencia `while` constituyen el programa original, que genera una salida igual a 0 cuando su entrada representa un programa que no es auto-terminante.) En este caso, nos saltaríamos el bucle de la estructura `while-end` y el programa se detendría. Pero esta es precisamente la propiedad que caracteriza a un programa auto-terminante, así que estamos obligados a concluir que el nuevo programa es auto-terminante, al igual que nos vimos forzados anteriormente a concluir que el programa no es auto-terminante.

Figure 12.7 Demostración de la irresolubilidad del problema de la detención.



En resumen, vemos que nos encontramos en la situación imposible de un programa que por un lado debe ser auto-terminante o no serlo, y que por otro lado no puede ser ninguna de las dos cosas. En consecuencia, la suposición que nos ha conducido a este dilema tiene que ser falsa.

Podemos concluir que la función de detención no es computable, y como la solución al problema de la detención depende de la computación de dicha función, llegamos a la conclusión de que resolver el problema de la detención cae más allá de las capacidades de cualquier sistema algorítmico. Este tipo de problemas se denominan **problemas irresolubles**.

Para terminar, conviene relacionar lo que acabamos de ver con las ideas del Capítulo 11. Allí, una de las principales cuestiones subyacentes era si la potencia de las máquinas de computación incluía las capacidades requeridas para la propia inteligencia. Recuerde que las máquinas solo pueden resolver problemas que tengan solución algorítmica, y ahora acabamos de ver que existen problemas que no tienen solución algorítmica. La cuestión es, entonces, si la mente humana abarca algo más que la ejecución de procesos algorítmicos. Si no es así, entonces los límites que acabamos de identificar aquí serán también los límites del pensamiento humano. No hace falta decir que esta cuestión es altamente debatida y en ocasiones ese debate llega a ser bastante emocional. Por ejemplo, si la mente humana no es más que una máquina programada, entonces nos veríamos forzados a concluir que los seres humanos no poseen libre albedrío.

Cuestiones y ejercicios

1. ¿Es auto-terminante el siguiente programa Bare Bones? Explique su respuesta.

```
incr X;  
decr Y;
```

2. ¿Es auto-terminante el siguiente programa Bare Bones? Explique su respuesta.

```
copy X to Y;  
incr Y;  
incr Y;  
while X not 0 do;  
  decr X;  
  decr X;  
  decr Y;  
  decr Y;  
end;  
decr Y;  
while Y not 0 do;  
end;
```

3. ¿Qué error hay en el siguiente escenario?

En un cierto pueblo, todo el mundo es propietario de su casa. El pintor del pueblo afirma que él pinta todas aquellas casas que no son pintadas por sus propios dueños y que solo pinta las casas que no son pintadas por sus propios dueños.

(Pista: ¿quién pinta la casa del pintor?)

12.5 Complejidad de los problemas

En la Sección 12.4 hemos investigado la resolubilidad de los problemas. En esta sección, estamos interesados en la cuestión de si un problema resoluble tiene una solución práctica. Veremos que algunos problemas que son teóricamente resolubles son tan complejos que los podemos considerar como irresolubles desde el punto de vista práctico.

Medida de la complejidad de un problema

Comenzaremos volviendo a nuestro estudio de la eficiencia de los algoritmos que iniciamos en la Sección 5.6. Allí utilizábamos la notación zeta-mayúscula para clasificar los algoritmos de acuerdo con el tiempo requerido para ejecutarlos. Vimos allí que el algoritmo de ordenación por inserción pertenece a la clase $\Theta(N^2)$, el algoritmo de búsqueda secuencial pertenece a la clase $\Theta(n)$ y que el algoritmo de búsqueda binaria es de la clase $\Theta(\lg n)$. Utilizaremos ahora este sistema de clasificación como ayuda para identificar la complejidad de los problemas. Nuestro objetivo es desarrollar un sistema de clasificación que nos diga qué problemas son más complejos que otros y, en último término, qué problemas son tan complejos que su solución es imposible desde el punto de vista práctico.

La razón de que nuestro presente análisis se base en nuestro conocimiento de la eficiencia de los algoritmos es que queremos medir la complejidad de un problema en términos de la complejidad de su solución. Consideraremos que un problema simple es aquel que dispone de una solución simple y que un problema complejo es aquel que no dispone de una solución simple. Observe que el hecho de que un problema tenga una solución difícil no implica necesariamente que el propio problema sea complejo. Después de todo, un problema dispone de múltiples soluciones, y alguna de ellas tenderá a ser compleja. Por tanto, para concluir que un problema es complejo tenemos que demostrar que ninguna de sus soluciones es simple.

En las Ciencias de la computación, los problemas que nos interesan son aquellos que son resolubles mediante máquinas. Las soluciones a estos problemas se expresan en forma de algoritmos. Por tanto, la complejidad de un problema estará determinada por las propiedades de los algoritmos que permiten resolver dicho problema. Para ser más precisos, consideraremos que la complejidad de un problema es igual a la complejidad del algoritmo más simple que permite resolverlo.

Pero, ¿cómo medimos la complejidad de un algoritmo? Lamentablemente, el término *complejidad* tiene diferentes interpretaciones. Una de esas inter-

pretaciones trata con la cantidad de toma de decisiones y el grado de ramificación que contenga el algoritmo. Desde ese punto de vista, un algoritmo complejo sería aquel que incluyera un conjunto de instrucciones muy entrelazado y complicado. Esta interpretación puede ser compatible con el punto de vista de un ingeniero de software, que está interesado en los problemas relativos al descubrimiento y representación de algoritmos, pero no captura el concepto de complejidad desde el punto de vista de una máquina. Una máquina no toma decisiones realmente a la hora de seleccionar la siguiente instrucción que tiene que ejecutar, sino que simplemente sigue su ciclo de máquina una y otra vez, ejecutando cada vez la instrucción que le indique el contador de programa. En consecuencia, una máquina puede ejecutar un conjunto de instrucciones muy entrelazado con la misma facilidad que puede ejecutar una lista de instrucciones en un simple orden secuencial. Esta interpretación de la complejidad tiende, por tanto, a medir la dificultad inherente a la representación de un algoritmo, más que la complejidad del propio algoritmo.

Una interpretación que refleja con más precisión la complejidad de un algoritmo desde el punto de vista de una máquina consiste en medir el número de pasos que hay que realizar a la hora de ejecutar el algoritmo. Observe que esto no es lo mismo que el número de instrucciones que aparecen en el programa escrito. Un bucle cuyo cuerpo esté compuesto de una única instrucción, pero cuyo control solicite la ejecución del bucle 100 veces es equivalente a 100 instrucciones secuenciales desde el punto de vista del número de instrucciones ejecutadas. Por tanto, esa rutina del bucle se consideraría más compleja que una lista de 50 instrucciones escritas individualmente, aún cuando esta lista parezca más larga en forma escrita. Lo importante aquí es que este significado de la palabra *complejidad* está relacionado en último término con el tiempo que una máquina tarda en ejecutar una solución y no con el tamaño del programa que representa dicha solución.

Podemos por tanto considerar que un problema es complejo si todas sus soluciones requieren una gran cantidad de tiempo de ejecución. Esta definición de la complejidad se suele denominar **complejidad temporal**. Ya nos hemos topado indirectamente con el concepto de complejidad temporal al estudiar el tema de la complejidad de los algoritmos en la Sección 5.6. Después de todo, el estudio de la eficiencia de un algoritmo es el estudio de la complejidad temporal de ese algoritmo, los dos conceptos son simplemente inversos el uno del otro. Es decir, "más eficiente" es lo mismo que "menos complejo". Por tanto, en términos de complejidad temporal, el algoritmo de búsqueda secuencial (que vimos que es del orden $\Theta(n)$) es una solución más compleja del problema de buscar en una lista que el algoritmo de búsqueda binaria (que vimos que es del orden de $\Theta(\lg n)$).

Apliquemos ahora nuestros conocimientos sobre la complejidad de los algoritmos para obtener un medio de identificar la complejidad de los problemas. Definimos la complejidad (temporal) de un problema como $\Theta(f(n))$, donde $f(n)$ es alguna expresión matemática dependiente de n , si existe un algoritmo para resolver el problema cuya complejidad temporal sea del orden $\Theta(f(n))$ y ningún otro algoritmo para resolver el problema tiene una complejidad temporal menor. Es decir, la complejidad (temporal) de un problema se define como la complejidad (temporal) de su mejor solución. Lamentablemente, encontrar la mejor solución a un problema y saber que es la mejor solu-

ción suele ser a menudo bastante complicado. En dichas situaciones, se utiliza una variante de la notación zeta-mayúscula, denominada **notación O mayúscula**, para representar lo que se conoce acerca de la complejidad de un problema. De forma más precisa, si $f(n)$ es una expresión matemática dependiente de n y si un problema puede ser resuelto por un algoritmo en $\Theta(f(n))$, diremos entonces que el problema es del orden $O(f(n))$. Por tanto, decir que un problema pertenece a la clase $O(f(n))$ implica que tiene una solución cuya complejidad es del orden $\Theta(f(n))$ pero que posiblemente podría tener una solución mejor.

Nuestro análisis de los algoritmos de búsqueda y ordenación nos dice que el problema de buscar en una lista de longitud n (cuando lo único que sabemos es que la lista ha sido previamente ordenada) es del orden $O(\lg n)$, ya que el algoritmo de búsqueda binaria permite resolver el problema. Además, los investigadores han demostrado que el problema de búsqueda es en realidad de orden $\Theta(\lg n)$, por lo que la búsqueda binaria representa una solución óptima para dicho problema. Por el contrario, sabemos que el problema de ordenar una lista de longitud n (cuando no sabemos nada acerca de la distribución original de los valores que la componen) es del orden $O(n^2)$, porque el algoritmo de ordenación por inserción permite resolver ese problema. Sin embargo, se sabe que el problema de la ordenación es de orden $\Theta(n \lg n)$, lo que nos dice que el algoritmo de ordenación por inserción no es una solución óptima (en términos de la complejidad temporal).

Un ejemplo de una mejor solución al problema de ordenación es el algoritmo de ordenación por combinación. El enfoque de ese algoritmo consiste en ordenar pequeñas partes de la lista con el fin de obtener partes ordenadas más grandes, que luego pueden combinarse para obtener otras partes ordenadas aún mayores. Cada proceso de combinación aplica el algoritmo de combinación que ya presentamos al hablar de los archivos secuenciales (Figura 9.15). Por comodidad, lo volvemos a mostrar en la Figura 12.8, esta vez en el contexto de la combinación de dos listas. El algoritmo completo (recursivo) de ordenación

Figura 12.8 Un procedimiento CombinarListas para combinar dos listas.

```
procedure CombinarListas (ListaEntradaA, ListaEntradaB, ListaSalida)
if (ambas listas de entrada están vacías) then (Parar con ListaSalida vacía)
if (ListaEntradaA está vacía)
    then (Declarar que la lista se ha terminado)
    else (Declarar como entrada actual su primera entrada)
if (ListaEntradaB está vacía)
    then (Declarar que la lista se ha terminado)
    else (Declarar como entrada actual su primera entrada)
while (ninguna de las listas de entrada se ha terminado) do
    (Poner la entrada actual "más pequeña" en ListaSalida;
     if (esa entrada actual es la última entrada en su correspondiente lista de entrada)
         then (Declarar que la lista de entrada se ha terminado)
         else (Declarar la siguiente entrada de esa lista de entrada como la entrada actual de la lista)
    )
Comenzando con la entrada actual de la lista de entrada que no se haya terminado,
copiar las entradas restantes a ListaSalida.
```


por combinación se incluye como un procedimiento denominado OrdenaciónComb en la Figura 12.9. Al pedirle que ordene una lista, este procedimiento comprueba primero si la lista tiene menos de dos entradas. En caso afirmativo, la tarea del procedimiento estará completada. En caso contrario, el procedimiento divide la lista en dos fragmentos, pide a otras copias del procedimiento OrdenaciónComb que ordene esos fragmentos y luego combina esos fragmentos ordenados para obtener la versión final ordenada de la lista.

Para analizar la complejidad de este algoritmo, consideraremos en primer lugar el número de comparaciones que hay que realizar entre las entradas de las listas a la hora de combinar una lista de longitud r con una lista de longitud s . El proceso de combinación se lleva a cabo comparando repetidamente una entrada de una lista con una entrada de la otra lista y colocando la "más pequeña" de las dos entradas en la lista de salida. Por tanto, cada vez que se efectúa una comparación, el número de entradas que queda por considerarse reduce en una unidad. Puesto que solo hay $r + s$ entradas al principio, podemos concluir que el proceso de combinar dos listas requerirá no más de $r + s$ comparaciones.

Consideremos ahora el algoritmo completo de ordenación por combinación. Este algoritmo afronta la tarea de ordenar una lista de longitud n de tal forma que el problema de ordenación inicial se reduce a otros problemas más pequeños, en cada uno de los cuales tenemos que ordenar una lista de longitud aproximadamente igual a $n/2$. Estos dos problemas se reducen a su vez a un total de cuatro problemas de ordenación de una lista de longitud aproximadamente igual a $n/4$. Este proceso de división puede resumirse mediante la estructura de árbol de la Figura 12.10, donde cada nodo del árbol representa un único problema dentro del proceso recursivo y las ramas situadas por debajo de un nodo representan los problemas más pequeños derivados del problema padre. Por tanto, podemos averiguar el número total de comparaciones que se realizan a lo largo de todo el proceso de ordenación sumando el número de comparaciones que tienen lugar en los distintos nodos del árbol.

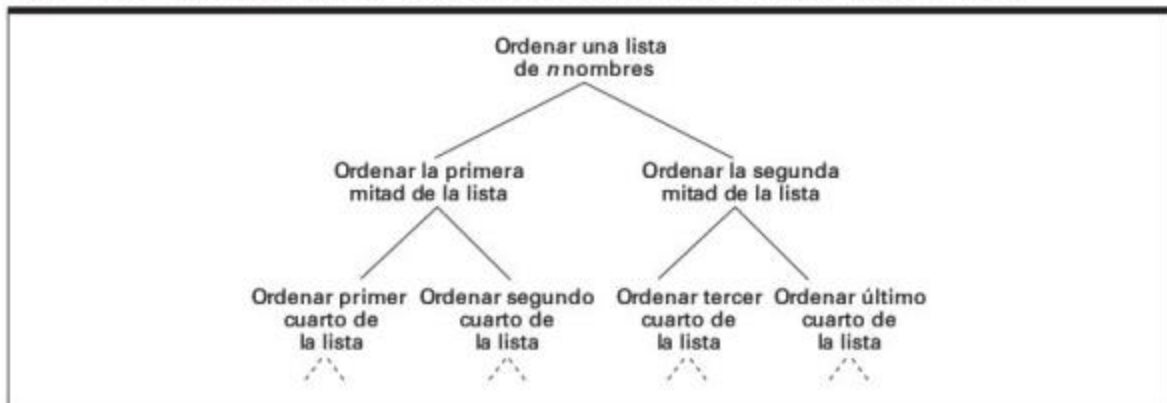
Determinemos primero el número de comparaciones realizadas en cada nivel del árbol. Observe que cada uno de los nodos que aparece en un nivel determinado del árbol está encargado de ordenar una parte diferente de la lista general. Esa ordenación se lleva a cabo mediante el proceso de combinación y requiere, por tanto, un número de comparaciones que no es superior al de entradas existentes en el segmento de lista, como ya hemos demostrado. Por tanto, cada nivel del árbol requiere un número de comparaciones que no es mayor que el número total de entradas de todos los segmentos de lista y, como

Figura 12.9 El algoritmo de ordenación por combinación implementado como un procedimiento OrdenaciónComb.

```

procedure OrdenacionComb (Lista)
if (Lista tiene más de una entrada)
  then (Aplicar el procedimiento OrdenacionComb para ordenar la primera mitad de Lista;
        Aplicar el procedimiento OrdenacionComb para ordenar la segunda mitad de Lista;
        Aplicar el procedimiento CombinarListas para combinar la primera y segunda
          mitades de Lista, para generar una versión ordenada de Lista
  )

```


Figura 12.10 La jerarquía de problemas generada por el algoritmo de ordenación por combinación.

los segmentos de un determinado nivel del árbol representan partes disjuntas de la lista original, ese total no será mayor que la longitud de la lista original. En consecuencia, cada nivel del árbol requiere no más de n comparaciones. (Por supuesto, el nivel inferior implica ordenar la lista de longitud menor que dos, lo que no requiere ninguna comparación en absoluto.)

Ahora vamos a calcular el número de niveles del árbol. Para ello, observe que el proceso de dividir los problemas en otros problemas más pequeños continúa hasta obtener listas de longitud inferior a dos. Por tanto, el número de niveles del árbol estará determinado por el número de veces que, comenzando con el valor inicial n , podamos dividir entre dos repetidamente hasta que el resultado no sea mayor que uno, lo que nos da un resultado de $\lg n$. De forma más precisa, no hay más de $\lceil \lg n \rceil$ niveles del árbol que requieren comparaciones, donde la notación $\lceil \lg n \rceil$ representa el valor de $\lg n$ redondeado al siguiente entero superior.

Finalmente, el número total de comparaciones realizadas por el algoritmo de ordenación por combinación a la hora de ordenar una lista de longitud n se obtiene multiplicando el número de comparaciones realizadas en cada nivel del árbol por el número de niveles en el que se realizan comparaciones. Podemos concluir que ese valor no es superior a $n \lceil \lg n \rceil$. Puesto que el grafo de $n \lceil \lg n \rceil$ tiene la misma forma general que el grafo de $n \lg n$, podemos concluir que el algoritmo de ordenación por combinación pertenece a $O(n \lg n)$. Combinando esto con el hecho de que los investigadores nos dicen que el problema de ordenación tiene una complejidad igual a $\Theta(n \lg n)$, querrá decir que el algoritmo de ordenación por combinación representa una solución óptima al problema de ordenación.

Problemas polinómicos y no polinómicos

Suponga que $f(n)$ y $g(n)$ son expresiones matemáticas. Decir que $g(n)$ está acotada por $f(n)$ significa que, a medida que aplicamos esas expresiones a valores cada vez mayores de n , el valor de $f(n)$ termina por ser mayor que el de $g(n)$ y continúa siendo mayor que $g(n)$ para todos los valores n superiores. En otras palabras, decir que $g(n)$ está acotada por $f(n)$ significa que la gráfica de $f(n)$

Complejidad espacial

Una alternativa a medir la complejidad en términos del tiempo es la de medir en su lugar los requisitos de espacio de almacenamiento, lo que da como resultado una medida que se conoce con el nombre de **complejidad espacial**. Es decir, la complejidad espacial de un problema está determinada por la cantidad de espacio de almacenamiento requerida para solucionar el problema. En el texto hemos visto que la complejidad temporal de ordenar una lista con n entradas es $O(n \lg n)$. La complejidad espacial del mismo problema no es superior a $O(n + 1) = O(n)$. Después de todo, ordenar una lista con n entradas utilizando la ordenación por inserción requiere reservar espacio para la propia lista más el espacio necesario para almacenar una única entrada de una forma temporal. Por tanto, si nos pidieran almacenar listas cada vez más largas nos encontraríamos con que el tiempo requerido se iría incrementando más rápidamente que el espacio necesario. Este es, de hecho, un fenómeno bastante común. Puesto que el utilizar el espacio requiere un cierto tiempo, la complejidad espacial de un problema nunca crece más rápidamente que su complejidad temporal.

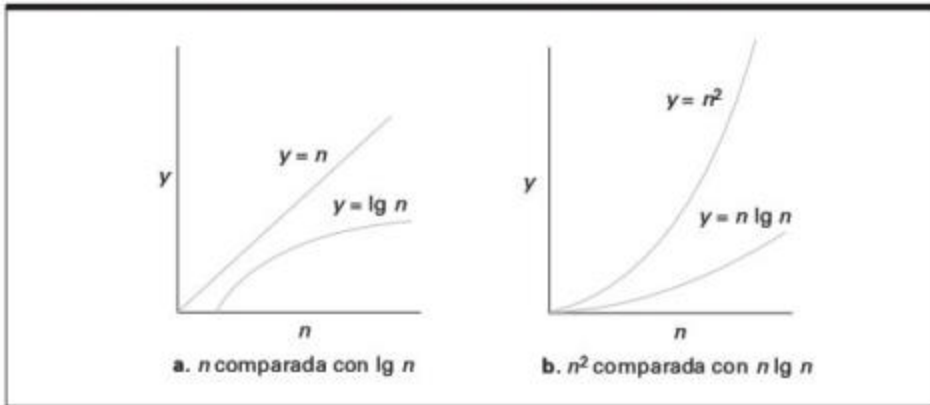
A menudo se pueden adoptar compromisos entre la complejidad temporal y la espacial. En algunas aplicaciones, resulta ventajoso realizar ciertos cálculos de antemano y almacenar los resultados en una tabla, de la que se los pueda extraer rápidamente cuando sea necesario. Esa técnica de “búsqueda en tablas” reduce el tiempo requerido para obtener un resultado cuando se necesita, a expensas del espacio adicional ocupado por la tabla. Por otro lado, a menudo se emplea también la compresión de datos para reducir los requisitos de almacenamiento, a expensas del tiempo adicional requerido para comprimir y descomprimir los datos.

estará situada por encima de la gráfica de $g(n)$ para valores “grandes” de n . Por ejemplo, la expresión $\lg n$ está acotada por la expresión n (Figura 12.11a) y $n \lg n$ está acotada por n^2 (Figura 12.11b).

Decimos que un problema es un **problema polinómico** si pertenece a la clase $O(f(n))$, en la que la expresión $f(n)$ es un polinomio ella misma o está acotada por un polinomio. El conjunto de todos los problemas polinómicos se denomina **P**. Observe que nuestros anteriores análisis nos dicen que los problemas de buscar en una lista y de ordenar una lista pertenecen a **P**.

Decir que un problema es un problema polinómico es un enunciado del tiempo necesario para resolver el problema. A menudo, decimos que un problema perteneciente a **P** puede resolverse en tiempo polinómico o que el problema tiene una solución de tiempo polinómico.

Identificar los problemas pertenecientes a **P** tiene una importancia crucial en las Ciencias de la computación, porque esa cuestión está estrechamente relacionada con otra serie de cuestiones relativas a si los problemas tienen soluciones prácticas. De hecho, los problemas que no pertenecen a la clase **P** se caracterizan por tener tiempos de ejecución extremadamente largos, incluso para entradas de tamaño moderado. Por ejemplo, considere un problema cuya solución requiera 2^n pasos. La expresión exponencial 2^n no está acotada por ningún polinomio, si $f(n)$ es un polinomio, entonces al ir incrementando el valor de n , nos encontramos con que los valores de 2^n terminarán por ser mayores que los de $f(n)$. Esto significa que un algoritmo de complejidad $\Theta(2^n)$ será generalmente menos eficiente (y requerirá por tanto más tiempo) que un

Figura 12.11 Gráficas de las expresiones matemáticas n , $\lg n$, $n \lg n$ y n^2 .

algoritmo de complejidad $\Theta(f(n))$. Cuando la complejidad de un algoritmo se identifica mediante una expresión exponencial, se dice que el algoritmo requiere un tiempo exponencial.

Como ejemplo concreto, considere el problema de enumerar todos los posibles subcomités que pueden formarse a partir de un grupo formado por n personas. Puesto que hay $2^n - 1$ subcomités posibles (permitimos que un subcomité esté compuesto por el conjunto completo, pero no consideramos el subconjunto vacío como un subcomité), cualquier algoritmo que resuelva este problema deberá tener al menos $2^n - 1$ pasos y, por tanto, su complejidad será por lo menos de ese orden. Pero, la expresión $2^n - 1$, al ser exponencial, no está acotada por ningún polinomio. Por tanto, cualquier solución a este problema es enormemente costosa en términos de tiempo a medida que se incrementa el tamaño del grupo a partir del cual se forman los comités.

A diferencia del problema de los subcomités, cuya complejidad es grande simplemente debido al tamaño de su salida, existen problemas cuya complejidad es grande aún cuando la salida final sea simplemente una respuesta de tipo sí o no. Un ejemplo sería la capacidad de responder cuestiones acerca de la verdad o falsedad de enunciados relativos a la suma de números reales. Por ejemplo, podemos reconocer fácilmente que la respuesta a la cuestión "¿Es verdad que existe un número real, que al sumarse consigo mismo da el valor 6?" es sí, mientras que la respuesta a la pregunta "¿Es verdad que existe un número real distinto de cero que, al sumarse consigo mismo da 0?" es no. Sin embargo, a medida que esas cuestiones se vuelven más complejas, nuestra capacidad de responderlas comienza a desvanecerse. Si nos tuviéramos que enfrentar con muchas de esas preguntas nos podríamos sentir tentados a recurrir a un programa de computadora en busca de ayuda. Lamentablemente, se ha demostrado que la capacidad de responder a estas cuestiones requiere un tiempo exponencial, por lo que incluso una computadora terminará por no poder dar una respuesta rápida, a medida que las cuestiones vayan haciéndose más complejas.

El hecho de que los problemas teóricamente resolubles pero que no pertenecen a P tengan una complejidad temporal tan enorme nos lleva a concluir que esos problemas son esencialmente irresolubles desde un punto de vista

práctico. Los expertos en Ciencias de la computación denominan a estos problemas **intratables**. A su vez, la clase P ha llegado a representar una importante frontera que diferencia a los problemas intratables de aquellos que pueden tener soluciones prácticas. Por tanto, la comprensión de la clase P ha llegado a ser un objetivo de gran importancia dentro de las Ciencias de la computación.

Problemas NP

Consideremos ahora el **problema del viajante** relativo a un viajante que debe visitar a cada uno de sus clientes que viven en ciudades distintas, sin superar el presupuesto que tiene asignado para el viaje. Su problema entonces es encontrar una ruta (que comience en su ciudad de residencia, que conecte todas las ciudades necesarias y que vuelva a su ciudad de residencia) cuya longitud total no exceda del número de máximo de kilómetros que puede viajar.

La solución tradicional a este problema es considerar todas las rutas potenciales de manera sistemática, comparando la longitud de cada ruta con el límite de kilómetros hasta encontrar una ruta aceptable o hasta haber analizado todas las posibilidades. Sin embargo, este enfoque no permite obtener una solución de tiempo polinómico. A medida que se incrementa el número de ciudades, la cantidad de rutas que hay que probar crece más rápidamente que cualquier polinomio. Por tanto, resolver el problema del viajante de esta forma no es práctico en aquellos casos en los que el número de ciudades implicadas sea grande.

En conclusión, para resolver el problema del viajante en una cantidad de tiempo razonable necesitamos encontrar un algoritmo más rápido. Nuestro deseo se ve alimentado por la observación de que si existe una ruta satisfactoria y por causalidad la seleccionamos en primer lugar, nuestro algoritmo actual terminará rápidamente. En particular, la siguiente lista de instrucciones puede ejecutarse rápidamente y tendría el potencial de resolver el problema:

```
Seleccionar una de las rutas posibles y calcular su
distancia total.
If (esta distancia no es mayor que el número máximo de
    kilómetros)
    then (declarar que la búsqueda ha tenido éxito)
    else (no declarar nada)
```

Sin embargo, este conjunto de instrucciones no es un algoritmo en el sentido técnico de la palabra. Su primera instrucción es ambigua, ya que no especifica qué ruta hay seleccionar y tampoco especifica cómo hay que tomar la decisión. En lugar de ello, la instrucción depende de la creatividad que tenga el mecanismo que esté ejecutando el programa a la hora de tomar la decisión por sí mismo. A este tipo de instrucciones las llamamos no deterministas y a un "algoritmo" que contenga tal clase de instrucciones le denominamos **algoritmo no determinista**.

Observe que cuando el número de ciudades aumenta, el tiempo requerido para ejecutar el algoritmo no determinista anterior crece de forma relativamente lenta. El proceso de seleccionar una ruta consiste simplemente en gene-

rar una lista de las ciudades, lo cual puede llevarse a cabo en un tiempo proporcional al número de ciudades existente. Además, el tiempo requerido para calcular la distancia total a lo largo de la ruta seleccionada también es proporcional al número de ciudades que hay que visitar, y el tiempo requerido para comparar este total con el número máximo de kilómetros es independiente del número máximo de ciudades. Por ello, el tiempo requerido para ejecutar ese algoritmo no determinista estará acotado por un polinomio. Por tanto, es posible resolver el problema del viajante en un tiempo polinómico utilizando un algoritmo no determinista.

Por supuesto, esta solución no determinista no es una solución totalmente satisfactoria. Depende de que acertemos a la hora de elegir la ruta. Pero su existencia es suficiente para sugerir que quizá exista una solución determinista al problema del viajante capaz de ejecutarse en tiempo polinómico. El que esto sea o no cierto sigue siendo una cuestión aún no resuelta. De hecho, el problema del viajante es uno de entre muchos problemas que se sabe que dispone de soluciones no deterministas capaces de ejecutarse en tiempo polinómico, pero para los cuales no se conoce aún ninguna solución determinista que se ejecute en tiempo polinómico. La llamativa eficiencia de las soluciones no deterministas a estos problemas hace que algunos expertos tengan la esperanza de poder encontrar algún día soluciones deterministas eficientes, aunque la mayoría creen que estos problemas son simplemente demasiado complejos como para resolverlos utilizando algoritmos deterministas eficientes.

Un problema que pueda resolverse en tiempo polinómico mediante un algoritmo no determinista se denomina **problema polinómico no determinista** o **problema NP** (*Nondeterministic Polynomial*). Habitualmente se denomina NP a la clase formada por todos los problemas de tipo NP. Observe que todos los problemas pertenecientes a P también pertenecen a NP, porque podemos añadir una instrucción no determinista a cualquier algoritmo (determinista) sin que el rendimiento de este se vea afectado.

El que todos los problemas NP pertenezcan también a P sigue siendo una cuestión no resuelta como demuestra el problema del viajante. Este es quizá el problema actualmente no resuelto más conocido de todo el campo de las Ciencias de la computación. Su solución podría tener consecuencias muy importantes. Por ejemplo, en la siguiente sección veremos que se han diseñado sistemas de cifrado cuya integridad depende de la enorme cantidad de tiempo requerido para resolver problemas similares al problema del viajante. Si resultara que existen soluciones eficientes a dicho problema, esos sistemas de cifrado se verían comprometidos.

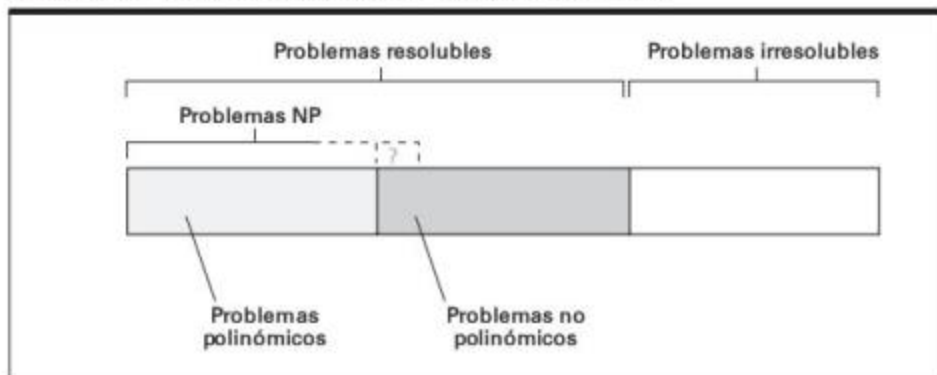
Los esfuerzos para resolver la cuestión de si la clase NP coincide, de hecho, con la clase P ha conducido al descubrimiento de una clase de problemas dentro de la clase NP a los que se conoce con el nombre de **problemas NP-completos**. Estos problemas tienen la propiedad de que una solución en tiempo polinómico para cualquiera de ellos proporcionaría también una solución en tiempo polinómico para todos los demás problemas de la clase NP. Es decir, si se pudiera encontrar un algoritmo (determinista) que resolviera uno de los problemas NP-completos en tiempo polinómico, entonces dicho algoritmo podría ampliarse para resolver cualquier otro problema de la clase NP en tiempo polinómico. A su vez, eso implicaría que la clase NP coincide con la clase P. El problema del viajante es un ejemplo de problema NP-completo.

En resumen, hemos visto que los problemas pueden clasificarse como resolubles (que tienen una solución algorítmica) o irresolubles (que no tienen una solución algorítmica), como se ilustra en la Figura 12.12. Además, dentro de la clase de problemas resolubles existen dos subclases. Una de ellas es el conjunto de problemas polinómicos, que contiene aquellos problemas que disponen de soluciones prácticas. El segundo es el conjunto de los problemas no polinómicos, cuyas soluciones solo son prácticas para entradas relativamente pequeñas o para entradas cuidadosamente seleccionadas. Finalmente, tenemos también los misteriosos problemas NP, que hasta ahora se han resistido a todos los intentos de clasificarlos de manera precisa.

Cuestiones y ejercicios

1. Suponga que un problema puede resolverse mediante un algoritmo de orden $\Theta(2^n)$. ¿Qué podemos concluir acerca de la complejidad del problema?
2. Suponga que un problema puede resolverse mediante un algoritmo del orden $\Theta(n^2)$, así como mediante otro algoritmo de orden $\Theta(2^n)$. ¿Alguno de los algoritmos será siempre más eficiente que el otro?
3. Enumere todos los subcomités que se pueden formar a partir de un comité compuesto por los dos miembros Alicia y Benito. Enumere todos los subcomités que pueden formarse a partir del comité formado por Alicia, Benito y Carol. ¿Qué sucede con los subcomités que pueden formarse a partir de un comité formado por Alicia, Benito, Carol y David?
4. Proporcione un ejemplo de problema polinómico. Proporcione un ejemplo de problema no polinómico. Proporcione un ejemplo de un problema NP para el que todavía no se haya podido demostrar que es un problema polinómico.
5. Si la complejidad del algoritmo X es mayor que la del algoritmo Y, ¿es el algoritmo X necesariamente más difícil de entender que el algoritmo Y? Explique su respuesta.

Figura 12.12 Resumen gráfico de la clasificación de problemas.



Determinista y no determinista

En muchos casos, existe una línea muy fina entre un “algoritmo” determinista y otro no determinista. Sin embargo, la distinción es bastante clara e importante. Un algoritmo determinista no depende de las características creativas del mecanismo que ejecuta el algoritmo, mientras que un “algoritmo” no determinista sí que podría depender de ellas. Por ejemplo, compare la instrucción

Ir hasta la siguiente intersección y girar a la derecha o a la izquierda.

con la instrucción

Ir hasta la siguiente intersección y girar a la derecha o a la izquierda dependiendo de lo que le diga la persona que está de pie en la esquina.

En ambos casos, la acción que lleve a cabo la persona que está siguiendo las instrucciones no está determinada antes de ejecutar realmente la instrucción. Sin embargo, la primera instrucción requiere que la persona que está siguiendo esas instrucciones tome una decisión basándose en su propio criterio y es, por tanto, no determinista. La segunda instrucción no impone ese requisito a la persona que está siguiendo las instrucciones, a esa persona se le dice lo que tiene que hacer en cada etapa. Si varias personas diferentes siguen la primera instrucción, unas girarán hacia la derecha, mientras que otras lo harán hacia la izquierda. Si varias personas siguen la segunda instrucción y reciben la misma información, todas ellas girarán en la misma dirección. Aquí radica una diferencia importante entre los “algoritmos” deterministas y no deterministas. Si un algoritmo determinista se ejecuta repetidamente con los mismos datos de entrada, cada vez se ejecutarán las mismas acciones. Sin embargo, un “algoritmo” no determinista podría dar lugar a acciones diferentes si se repite en condiciones idénticas.

12.6 Criptografía de clave pública

En algunos casos, el hecho de que un problema sea difícil de resolver ha resultado ser una ventaja en lugar de un inconveniente. Un caso especialmente interesante es el de encontrar los factores de un cierto entero, un problema para el que todavía no se ha descubierto una solución eficiente, si es que existe una. Por ejemplo, utilizando únicamente lápiz y papel podríamos comprobar que la tarea de determinar los factores de valores relativamente pequeños, como 2173, es bastante tediosa, y si el número en cuestión fuera tan grande que su representación requiriera varios centenares de dígitos, la tarea sería intratable incluso si aplicáramos tecnología moderna y usáramos las mejores técnicas de factorización actualmente conocidas.

La imposibilidad de determinar una forma eficiente de determinar los factores de números enteros de gran tamaño ha traído de cabeza durante mucho tiempo a numerosos matemáticos, pero en el campo de la criptografía se ha utilizado este hecho para diseñar un método muy popular de cifrado y descifrado de mensajes. Este método se conoce con el nombre de **algoritmo RSA**, un nombre elegido en honor a sus inventores Ron Rivest, Adi Shamir y Len

Adleman. Se trata de un medio de cifrar mensajes utilizando un conjunto de valores conocido con el nombre de **claves de cifrado** y de descifrar dichos mensajes empleando otro conjunto de valores conocido con el nombre de **claves de descifrado**. Las personas que conocen las claves de cifrado pueden cifrar los mensajes, pero no pueden descifrarlos. La única persona que puede descifrar los mensajes es aquella que conozca las claves de descifrado. Por tanto, las claves de cifrado se pueden distribuir a todo el mundo sin problemas y sin comprometer la seguridad del sistema.

Estos sistemas criptográficos se conocen como sistemas de **cifrado de clave pública**, un término que refleja el hecho de que las claves utilizadas para cifrar los mensajes pueden ser de conocimiento público sin que sufra por ello la seguridad del sistema. De hecho, las claves de cifrado se denominan a menudo **claves públicas**, mientras que las claves de descifrado se denominan **claves privadas** (Figura 12.13).

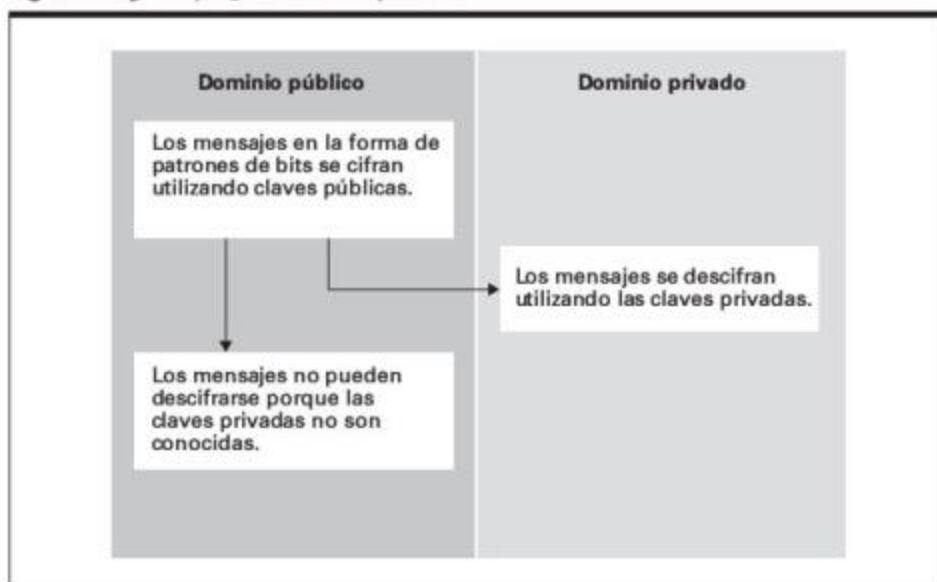
Notación modular

Para describir el sistema de cifrado de clave pública RSA es conveniente utilizar la notación $x \pmod{m}$, que se lee " x módulo m " o simplemente " $x \bmod m$ ", para representar el resto obtenido al dividir el valor x entre m . Así, $9 \pmod{7}$ es 2 porque $9 \div 7$ nos da un resto igual a 2. De forma similar, $24 \pmod{7}$ es 3 porque $24 \div 7$ nos da un resto de 3, y $14 \pmod{7}$ es 0 porque $14 \div 7$ da un resto igual a 0. Observe que $x \pmod{m}$ es igual a x si x es un entero comprendido en el rango que va de 0 a $m - 1$. Por ejemplo, $4 \pmod{9}$ es igual a 4.

Las matemáticas nos dicen que si p y q son números primos y m es un entero comprendido entre 0 y pq (el producto de p por q) entonces

$$1 = m^{k(p-1)(q-1)} \pmod{pq}$$

Figura 12.13 Criptografía de clave pública.



para cualquier entero positivo k . Aunque no vamos a demostrar aquí esta igualdad, sí que es conveniente ver un ejemplo con el fin de clarificar qué es lo que queremos decir. Supongamos, por tanto, que p y q son los números primos 3 y 5, respectivamente, y que m es el entero 4. Entonces, esa igualdad afirma que para cualquier entero positivo k , el valor $m^{k(p-1)(q-1)}$ dividido entre 15 (el producto de 3 por 5) nos dará como resto el valor 1. En particular, si $k = 1$, entonces

$$m^{k(p-1)(q-1)} = 4^{1(3-1)(5-1)} = 4^8 = 65.536$$

que al dividirlo entre 15 nos da un resto de 1, tal como establece la igualdad. Además, si $k = 2$, entonces

$$m^{k(p-1)(q-1)} = 4^{2(3-1)(5-1)} = 4^{16} = 4.294.967.296$$

que al dividirlo entre 15 de nuevo nos da como resto 1. De hecho, siempre obtendremos el resto 1, independientemente del entero positivo que seleccionemos para k .

Criptografía de clave pública RSA

Ahora estamos preparados para construir y analizar un sistema de cifrado de clave pública basado en el algoritmo RSA. Primero seleccionamos dos números primos diferentes p y q , cuyo producto representaremos mediante n . A continuación, seleccionamos otros dos enteros positivos e y d tales que $e \times d = k(p-1)(q-1) + 1$, para algún entero positivo k . Llamamos a estos valores e y d porque formarán parte del proceso de cifrado (e) y descifrado (d), respectivamente. (El hecho de que se puedan seleccionar valores e y d que satisfagan la ecuación anterior se deduce también de las correspondientes teorías matemáticas, aunque aquí no lo vamos a demostrar.)

Así, hemos seleccionado cinco valores: p , q , n , e y d . Los valores e y n serán las claves de cifrado. Los valores d y n serán las claves de descifrado. Los valores p y q solo se emplean para construir el sistema de cifrado.

Consideremos un ejemplo específico para clarificar el proceso. Suponga que elegimos 7 y 13 como valores de p y q . Luego $n = 7 \times 13 = 91$. Además, podemos utilizar los valores 5 y 29 para e y d , porque $5 \times 29 = 145 = 144 + 1 = 2(7-1)(13-1) + 1 = 2(p-1)(q-1) + 1$ tal como se requiere. Por tanto, las claves de cifrado serán $n = 91$ y $e = 5$, y las claves de descifrado serán $n = 91$ y $d = 29$. Distribuiremos las claves de cifrado a cualquiera que desee enviarnos mensajes, pero conservaremos para nosotros las claves de descifrado (así como los valores de p y q).

Veamos ahora cómo se cifran los mensajes. Con este objetivo, suponga que un mensaje está codificado en forma de patrón de bits (quizá utilizando ASCII o Unicode) y suponga que el valor de este patrón, cuando se interpreta como representación binaria, es menor que n . (Si no es menor que n , partiríamos el mensaje en segmentos más pequeños y cifraríamos cada segmento individualmente.)

Supongamos que nuestro mensaje, al ser interpretado como representación binaria, representa el valor m . Entonces, la versión cifrada del mensaje será la representación binaria del valor $c = m^e \pmod{n}$. Es decir, el mensaje cifrado será la representación binaria del resto que se obtiene al dividir m^e entre n .

En particular, si alguien quisiera cifrar el mensaje 10111 utilizando las claves de cifrado $n = 91$ y $e = 5$ que hemos seleccionado en el ejemplo anterior, primero reconocería que 10111 es la representación binaria de 23, luego calcularía $23^5 = 23^5 = 6.436.343$, y por último dividiría este valor entre $n = 91$, obteniendo como resto 4. La versión cifrada del mensaje sería por tanto 100, que es la representación binaria de 4.

Para descifrar un mensaje que represente el valor c en notación binaria, calculamos el valor $c^d \pmod{n}$. Es decir, calculamos el valor c^d , dividimos el resultado entre n y nos quedamos con el resto. De hecho, este resto será el valor m del mensaje original porque

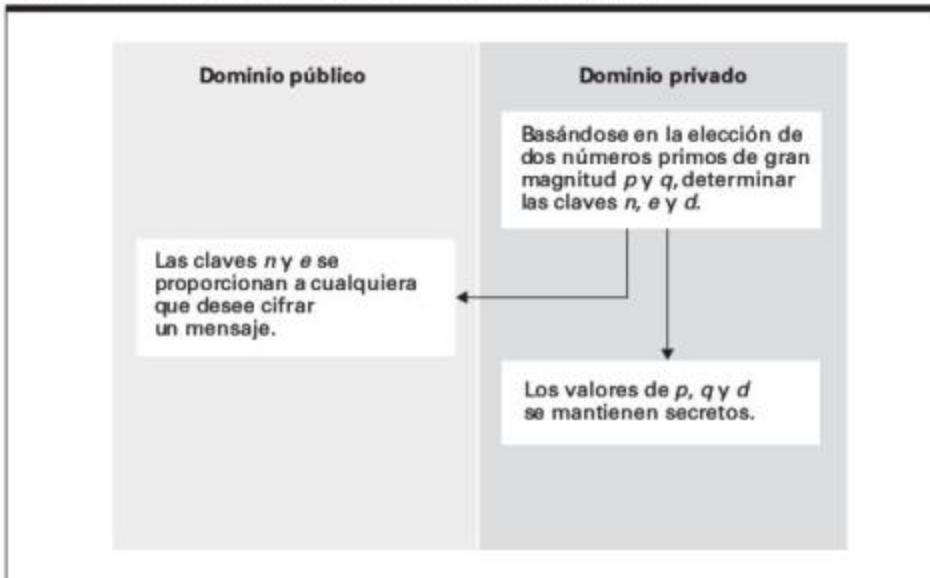
$$\begin{aligned} c^d \pmod{n} &= m^{e \times d} \pmod{n} \\ &= m^{k(p-1)(q-1) + 1} \pmod{n} \\ &= m \times m^{k(p-1)(q-1)} \pmod{n} \\ &= m \pmod{n} \\ &= m \end{aligned}$$

Aquí hemos usado el hecho de que $m^{k(p-1)(q-1)} \pmod{n} = m^{k(p-1)(q-1)} \pmod{pq} = 1$ y también el hecho de que $m \pmod{n} = m$ (porque $m < n$), como hemos indicado anteriormente.

Continuando con el ejemplo anterior, si recibiéramos el mensaje 100, reconoceríamos que es la representación binaria del valor 4, calcularíamos el valor $4^d = 4^{29} = 288.230.376.151.711.744$, y lo dividiríamos entre $n = 91$ obteniendo como resto el valor 23, que genera el mensaje original 10111 en notación binaria.

En resumen, un sistema de cifrado de clave pública RSA se construye seleccionando dos números enteros primos, p y q , a partir de los cuales generamos los valores n , e y d . Los valores n y e se utilizan para cifrar mensajes y son, por tanto, las claves públicas. Los valores n y d se usan para descifrar los mensajes y son las claves privadas (Figura 12.14). La belleza del sistema radica en que el saber cómo cifrar los mensajes no permite a nadie descifrar esos mismos mensajes. Por tanto, las claves de cifrado n y e pueden ser distribuidas sin ninguna restricción. Si alguien con intenciones maliciosas obtuviera esas claves, seguiría sin ser capaz de descifrar los mensajes interceptados; solo la persona que conozca las claves de descifrado podrá descifrar los mensajes.

La seguridad de este sistema está basada en la suposición de que conocer las claves de cifrado n y e no permite calcular las claves de descifrado n y d . Sin embargo, existen algoritmos que hacen precisamente eso. Una solución consistiría en factorizar el valor n para descubrir los valores p y q , y después determinar d hallando un valor k tal que $k(p-1)(q-1) + 1$ sea divisible por e (el

Figura 12.14 Diseño de un sistema de cifrado de clave pública RSA.

cociente sería entonces d). Sin embargo, el primer paso de este proceso puede requerir mucho tiempo, especialmente si los valores de p y q se eligen muy grandes. De hecho, si p y q son tan grandes que sus representaciones binarias requieren cientos de dígitos, entonces los mejores algoritmos de factorización conocidos necesitarían años para poder calcular a partir de n los valores de p y q . Por ello, el contenido de un mensaje cifrado seguiría siendo seguro mucho después de que su importancia se hubiera ya desvanecido.

Hasta la fecha, nadie ha encontrado una forma eficiente de descifrar mensajes basados en la criptografía RSA sin conocer las claves de descifrado, de modo que el cifrado de clave pública basado en el algoritmo RSA se utiliza ampliamente para conseguir confidencialidad a la hora de comunicarse a través de Internet.

Cuestiones y ejercicios

1. Calcule los factores de 66.043. (No pierda mucho tiempo con este problema. Lo importante es que se dé cuenta de que puede hacer falta mucho tiempo.)
2. Utilizando las claves públicas $n = 91$ y $e = 5$, cifre el mensaje 101.
3. Utilizando las claves privadas $n = 91$ y $d = 29$, descifre el mensaje 10.
4. Calcule el valor apropiado para las claves de descifrado n y d en un sistema de criptografía de clave pública RSA basándose en los números primos $p = 7$ y $q = 19$, y utilizando como clave de cifrado $e = 5$.

Problemas de repaso

1. Demuestre cómo puede simularse mediante Bare Bones una estructura de la forma

```
while X equals 0 do;
.
.
.
end;
```

2. Escriba un programa en Bare Bones que almacene un 1 en la variable Z si la variable X es menor o igual que la variable Y, y que almacene un 0 en la variable Z en caso contrario.

3. ¿Cuáles son las principales características de los algoritmos no deterministas?

4. En cada uno de los siguientes casos escriba una secuencia de programa en Bare Bones que realice la actividad indicada.

- a. Asignar 0 a Z si el valor de X es par; en caso contrario, asignar 1 a Z.
- b. Calcular la suma de los enteros comprendidos entre 0 y X.

5. Escriba una rutina en Bare Bones que divida el valor de X entre el valor de Y. Descarte cualquier resto; es decir, 1 dividido entre 2 da 0, y 5 dividido entre 3 da 1.

6. Describa la función computada por el siguiente programa Bare Bones, suponiendo que las entradas de la función están representadas por X e Y y que su salida está representada por Z:

```
copy X to Z;
copy Y to Aux;
while Aux not 0 do;
  decr Z;
  decr Aux;
end;
```

7. Describa la función computada por el siguiente programa Bare Bones, suponiendo que las entradas de la función están representadas por X e Y y que su salida está representada por Z:

```
clear Z;
copy X to Aux1;
copy Y to Aux2;
```

```
while Aux1 not 0 do;
  while Aux2 not 0 do;
    decr Z;
    decr Aux2;
  end;
  decr Aux1;
end;
```

8. Escriba un programa Bare Bones que compute la operación OR exclusivo de las variables X e Y, y que deje el resultado en la variable Z. Puede suponer que X e Y solo pueden comenzar con valores enteros iguales a 0 o a 1.

9. Demuestre que si permitimos etiquetar las sentencias de un programa Bare Bones con valores enteros y sustituimos la estructura de bucle `while` con la bifurcación condicional representada por la forma

```
if nombre not 0 goto etiqueta;
```

donde *nombre* es cualquier variable y *etiqueta* es un valor entero utilizado en algún otro lugar para etiquetar una sentencia, entonces el nuevo lenguaje continuará siendo un lenguaje de programación universal.

10. En este capítulo hemos visto cómo puede simularse en Bare Bones la sentencia

```
copy nombre1 to nombre2;
```

Demuestre cómo podría seguir simulándose esa sentencia si sustituimos la estructura de bucle `while` de Bare Bones con un bucle de post-comprobación expresado en la forma

```
repeat ... until (nombre equals 0)
```

11. Demuestre que el lenguaje Bare Bones continuaría siendo un lenguaje universal si sustituimos la sentencia `while` por un bucle de post-comprobación expresado en la forma

```
repeat ... until (nombre equals 0)
```

12. ¿Cuándo rechazaría una máquina de Turing la salida?

13. Diseñe una máquina de Turing que escriba ceros en todas las casillas situadas a la

izquierda de la casilla actual, hasta alcanzar una casilla que contenga un asterisco.

14. Suponga que un patrón de 0s y 1s de la cinta de una máquina de Turing está delimitado mediante asteriscos por ambos extremos. Diseñe una máquina de Turing que rote este patrón una celda hacia la izquierda, asumiendo que cuando la máquina comienza a operar, la casilla actual es el asterisco situado en el extremo derecho del patrón.
15. Diseñe una máquina de Turing que invierta el patrón de 0s y 1s que se encuentre entre la casilla actual (que contiene un asterisco) y el primer asterisco de la izquierda.

16. ¿Qué son los problemas irresolubles?

17. ¿Es auto-terminante el siguiente programa Bare Bones? Explique su respuesta.

```
copy X to Y;
incr Y;
incr Y;
while X not 0 do;
    decr X;
    decr X;
    decr Y;
    decr Y;
end;
decr Y;
while Y not 0 do;
    incr X;
    decr Y;
end;
while X not 0 do;
end;
```

18. ¿Es auto-terminante el siguiente programa Bare Bones? Explique su respuesta.

```
Numero ← 1;
while (Numero < 60) do
    (Numero ← Numero + 2)
```

19. ¿Es auto-terminante el siguiente programa Bare Bones? Explique su respuesta.

```
Z ← 1;
X ← 2;
while (X < 50) do
    (Z ← Z + X;
     X ← X + 2)
```

20. Analice la validez de la siguiente pareja de enunciados:

El siguiente enunciado es verdadero.
El enunciado anterior es falso.

21. Explique la diferencia entre un algoritmo determinista y un algoritmo no determinista.

22. Suponga que se encuentra en un país en el que cada persona es franca o es mentirosa. (Una persona franca siempre dice la verdad, mientras que un mentiroso siempre miente.) ¿Qué pregunta podría hacer a una persona para detectar si es una persona franca o es mentirosa?

23. ¿Cuáles son las principales cuestiones acerca de la complejidad de un problema?

24. Resuma las principales características de la criptografía de clave pública.

25. Suponga que necesitamos averiguar si alguna de las personas de un cierto grupo cumple años en una fecha dada. Una solución sería preguntar a cada una de las personas de una en una. Si adoptamos esta solución, ¿qué suceso sería el que nos dijera que existe tal persona? ¿Qué suceso nos diría que no existe tal persona? Ahora suponga que queremos averiguar si al menos uno de los enteros positivos tiene una propiedad concreta y suponga que aplicamos la misma solución, consistente en ir probando de forma sistemática todos los enteros, de uno en uno. Si fuera cierto que algún entero tiene esa propiedad, ¿cómo lo averiguaríamos? Sin embargo, si ningún entero tuviera esa propiedad, ¿cómo lo averiguaríamos? ¿Cree que la tarea de comprobar si una conjetura es cierta es necesariamente simétrica de la tarea de comprobar si esa conjetura es falsa?

26. ¿Es de tipo polinómico el problema de buscar un cierto valor concreto en una lista? Explique su respuesta.

27. Diseñe un algoritmo para determinar si un cierto entero positivo es primo. ¿Es eficiente su solución? ¿Es una solución de tipo polinómico o no polinómico?

28. ¿Una solución polinómica a un problema es siempre mejor que una solución exponencial? Explique su respuesta.

29. ¿El hecho de que un problema tenga una solución polinómica significa que siempre puede resolverse en un tiempo razonable? Explique su respuesta.
30. Suponga que asignamos al programador Carlos el problema de dividir un grupo (formado por un número par de personas) en dos subgrupos disjuntos de igual tamaño, de manera que la diferencia entre la suma de las edades de cada uno de los subgrupos sea lo mayor posible. Carlos propone la solución de formar todos los posibles pares de subgrupos, calcular la diferencia entre la edad total de cada una de esos pares de subgrupos y seleccionar el par que tenga la máxima diferencia. Por el contrario, la programadora María propone ordenar primero el grupo original por edad y luego dividirlo en dos subgrupos formando uno de los subgrupos a partir de la mitad más joven del grupo ordenado y formando el otro subgrupo a partir de la otra mitad. ¿Cuál es la complejidad de cada una de estas soluciones? ¿Cuál es la complejidad del propio problema, polinómica, NP o no polinómica?
31. ¿Por qué la solución consistente en generar todas las posibles disposiciones de una lista y luego seleccionar aquella que tiene la disposición deseada no es una solución satisfactoria para ordenar una lista?
32. Suponga que un juego de lotería está basado en seleccionar correctamente cuatro valores enteros, cada uno de los cuales está comprendido en el rango que va de 1 hasta 50. Suponga además que el bote llega a ser tan grande que sería beneficioso comprar un billete de lotería para cada una de las posibles combinaciones. Si se tarda un segundo en comprar un único billete, ¿cuánto se tardaría en comprar un billete para cada combinación? ¿Cómo cambiaría el tiempo necesario si esa lotería requiriera seleccionar cinco números en lugar de cuatro? ¿Qué tiene que ver este problema con el material presentado en este capítulo?
33. ¿Es determinista el siguiente algoritmo? Explique su respuesta.

```
procedure cualquierNum (Num)
if (Num == 5)
  then (responder "no")
  else (Num = Num + 1
        proporcionar este número
        como respuesta)
```

34. ¿Es determinista el siguiente algoritmo? Explique su respuesta.

Tomar un número aleatorio como entrada.

Si el número aleatorio es mayor que 10, entonces sumarle 20.

En caso contrario restarle 30.

Tomar otro número aleatorio para compararlo con el anterior

Si el primer número es mayor entonces intercambiar el valor con el otro número.

Escribir los valores de ambos números.

35. Identifique los puntos no deterministas en el siguiente algoritmo:

Seleccionar tres números entre 1 y 100.

if (la suma de los números seleccionados es mayor que 150)

then (responder "sí")

else (seleccionar uno de los números elegidos y proporcionar ese número como respuesta.)

36. ¿Qué complejidad temporal tiene el siguiente algoritmo, polinómica o no polinómica? Explique su respuesta.

```
procedure misterio (ListaDeNumeros)
  Seleccionar un conjunto de numeros
  de ListaDeNumeros.
  if (los números de dicho conjunto
      suman 125)
    then (responder "sí")
    else (no dar una respuesta)
```

37. ¿Cuáles de los siguientes problemas pertenecen a la clase P?
- Un problema con complejidad n^2
 - Un problema con complejidad $2n$
 - Un problema con complejidad $n^2 + 2n$
 - Un problema con complejidad $n!$

38. Resuma la diferencia entre afirmar que un problema es de tipo polinómico y afirmar que es un problema polinómico no determinista.
39. ¿Qué es un problema intratable? Explique su respuesta.
40. ¿Cuál es la complejidad temporal de cada uno de los siguientes algoritmos?
- Multiplicación de matrices.
 - Ordenación por burbuja.
 - Búsqueda lineal.
 - Búsqueda binaria.
41. Suponga que tenemos que resolver el problema del viajante para el caso de que existan 15 ciudades y que cada pareja de ciudades está conectada por una carretera distinta. ¿Cuántas rutas diferentes a través de todas las ciudades existirán? ¿Cuánto se tardaría en calcular la longitud de todas esas rutas, suponiendo que la longitud de una ruta puede calcularse en un microsegundo?
42. ¿Cuántas comparaciones entre nombres se realizarán si aplicamos el algoritmo de ordenación por combinación (Figuras 12.9 y 12.8) a la lista formada por los nombres Alicia, Benito, Carol y David? ¿Cuántas se requerirán si la lista está formada por Alicia, Benito, Carol, David y Elena?
43. Explique el uso de una función computable de Turing.
44. Diseñe un algoritmo para determinar soluciones enteras para las ecuaciones de la forma $x^2 + y^2 = n$, donde n es un entero positivo dado. Determine la complejidad temporal del algoritmo.
45. Otro problema que pertenece a la categoría de los problemas NP-completos es el denominado **problema de la mochila**, que es el problema de hallar qué números de una lista tienen una suma igual a un cierto valor concreto. Por ejemplo, los números 257, 388 y 782 son las entradas de la lista
 642 257 771 388 391 782 304
 cuya suma es 1427. Determine las entradas cuya suma es igual a 1723. ¿Qué algoritmo ha aplicado? ¿Cuál es la complejidad de ese algoritmo?
46. Identifique las similitudes entre el problema del viajante y el problema de la mochila (véase el Problema 45).
47. El siguiente algoritmo para ordenar listas se denomina ordenación por inserción en la Figura 5.11. ¿Cuántas comparaciones entre entradas de la lista requiere el algoritmo de ordenación por inserción cuando se aplica a una lista de n entradas?
- procedure** OrdenaciónInserta (Lista)
 $N \leftarrow 2$;
while (el valor de N no exceda la longitud de Lista) **do**
 (Seleccionar la entrada N -ésima de Lista como la entrada pivote;
 Mover la entrada pivote a una ubicación temporal dejando un hueco en Lista;
while (haya un nombre por encima del hueco y dicho nombre sea mayor que la entrada pivote)
do (mover el nombre al hueco dejando encima suyo un hueco)
 Mover la entrada pivote al hueco de Lista;
 $N \leftarrow N + 1$
)
48. Utilice el sistema de cifrado de clave pública RSA para cifrar el mensaje 110 utilizando las claves públicas $n = 91$ y $e = 5$.
49. ¿Cuál es la fortaleza del algoritmo RSA?
50. Suponga que sabemos que las claves públicas de un sistema de cifrado de clave pública basado en el algoritmo RSA son $n = 119$ y $e = 5$. ¿Cuáles serán las claves privadas?
51. ¿Cuándo es auto-terminante un programa Bare Bones?
52. ¿Qué podemos deducir si el entero positivo n no tiene ningún factor entero comprendido en el rango que va de 2 a la raíz cuadrada de n ? ¿Qué nos dice esto acerca de la tarea de calcular los factores de un entero positivo?

Cuestiones sociales

Las siguientes cuestiones pretenden ser una guía para los problemas éticos/sociales/legales asociados con el campo de la computación. El objetivo no es responder simplemente a estas cuestiones. El lector debería considerar también por qué las ha contestado de la forma en que lo ha hecho y analizar si sus justificaciones son coherentes entre las distintas cuestiones.

1. Suponga que el mejor algoritmo para resolver un problema requiere 100 años para ejecutarse. ¿Consideraría que ese problema es tratable? ¿Por qué?
2. ¿Deberían los ciudadanos tener el derecho de cifrar sus mensajes de tal forma que se evite su monitorización por parte de organismos gubernamentales? ¿Cree que su respuesta permite imponer "apropiadamente" el cumplimiento de las leyes? ¿Quién debería decidir lo que es una imposición "apropiada" del cumplimiento de las leyes?
3. Si la mente humana es un dispositivo algorítmico, ¿qué consecuencias tiene la tesis de Turing en lo que respecta a la humanidad? ¿Hasta qué grado cree que las máquinas de Turing engloban las capacidades computacionales de la mente humana?
4. Ya hemos visto que existen diferentes modelos computacionales (tablas finitas, fórmulas algebraicas, máquina de Turing, etc.) que tienen diferentes capacidades de computación. ¿Existen diferencias de las capacidades computacionales de los distintos organismos? ¿Existen diferencias en las capacidades computacionales de los distintos seres humanos? En caso afirmativo, ¿deberían los humanos que tienen mayores capacidades poder utilizarlas para conseguir un estilo de vida más acomodado?
5. Hoy día existen sitios web que proporcionan mapas de la mayoría de las ciudades. Estos sitios ofrecen ayuda a la hora de encontrar direcciones concretas, así como capacidades de ampliación para visualizar la disposición de pequeñas partes de una ciudad. Partiendo de esta realidad, considere la siguiente secuencia de suposiciones ficticias. Suponga que esos sitios con mapas incorporaran fotografías de satélite que dispusieran de similares capacidades de ampliación. Suponga que esas capacidades de ampliación se aumentaran para proporcionar una imagen más detallada de los edificios individuales y del paisaje circundante. Suponga que esos servicios de imágenes se mejoraran todavía más incluyendo vídeo en tiempo real. Suponga que esas imágenes de vídeo se mejoraran con tecnología infrarroja. En ese punto, otras personas podrían vernos dentro de nuestra propia casa, 24 horas al día. ¿En qué punto de esta progresión se habrá violado por primera vez nuestro derecho a la intimidad? ¿En qué punto de esta progresión cree que habremos ido más allá de las capacidades de la tecnología actual de los satélites espía? ¿Hasta qué punto es ficticio este escenario?
6. Suponga que una empresa desarrolla y patenta un sistema de cifrado. ¿Debería el gobierno de la nación donde tiene sede esa empresa tener derecho a utilizar el sistema como le venga en gana en nombre de la seguridad nacional? ¿Debería el gobierno de la nación en la que la empresa tiene su sede tener derecho a restringir el uso comercial de ese sistema por parte de

la empresa en nombre de la seguridad nacional? ¿Y qué pasa si la empresa es una multinacional?

7. Suponga que adquiere un producto cuya estructura interna está cifrada. ¿Tiene usted derecho a descifrar la estructura subyacente? En caso afirmativo, ¿tendría derecho a utilizar esa información de manera comercial? ¿Y de manera no comercial? ¿Qué pasa si el cifrado se realizó utilizando un sistema de cifrado secreto y usted descubre el secreto? ¿Tendría derecho a compartir ese secreto?
8. Hace bastantes años, el filósofo John Dewey (1859–1952) introdujo el término “tecnología responsable”. Proporcione algunos ejemplos de lo que usted consideraría “tecnología responsable”. Basándose en esos ejemplos, formule su propia definición de “tecnología responsable”. ¿Cree que la sociedad ha practicado una “tecnología responsable” a lo largo de los últimos 100 años? ¿Debería llevarse a cabo algún tiempo de acción para garantizar que lo haga? En caso afirmativo, ¿qué acciones serían esas?; en caso contrario, ¿por qué no?

Lecturas adicionales

Garey, M. R. y D. S. Johnson. *Computers and Intractability*. Nueva York: W. H. Freeman, 1979.

Hamburger, H. y D. Richards. *Logic and Language Models for Computer Science*. Englewood Cliffs, NJ: Prentice-Hall, 2002.

Hofstadter, D. R. *Gödel, Escher, Bach: An Eternal Golden Braid*. St. Paul, MN: Vintage, 1980.

Hopcroft, J. E., R. Motwani y J. D. Ullman. *Introduction to Automata Theory, Languages, and Computation*, 3ª ed. Boston, MA: Addison-Wesley, 2007.

Lewis, H. R. y C. H. Papadimitriou. *Elements of the Theory of Computation*, 2ª ed. Englewood Cliffs, NJ: Prentice-Hall, 1998.

Rich E. Automata. *Computability, and Complexity: Theory and Application*. Upper Saddle River, NJ: Prentice-Hall, 2008.

Sipser, M. *Introduction to the Theory of Computation*. Boston: PWS, 1996.

Smith, C. y E. Kinber. *Theory of Computing: A Gentle Introduction*. Englewood Cliffs, NJ: Prentice-Hall, 2001.

Sudkamp, T. A. *Languages and Machines: An Introduction to the Theory of Computer Science*, 3ª ed. Boston, MA: Addison-Wesley, 2006.

Apéndices



A ASCII

**B Circuitos para manipular representaciones
en complemento a dos**

C Un lenguaje máquina simple

D Lenguajes de programación de alto nivel

**E Equivalencia de las estructuras iterativas
y recursivas**

F Respuestas a las Cuestiones y ejercicios

ap é n d i c e

A

ASCII

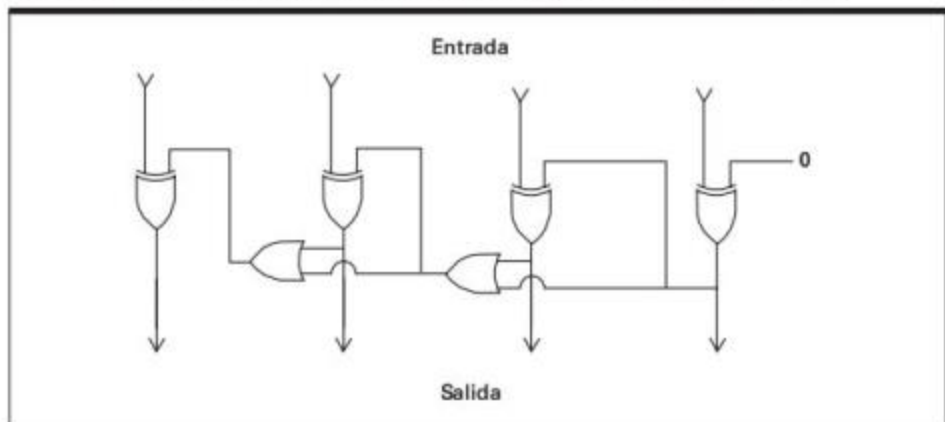
La siguiente tabla proporciona una lista parcial del código ASCII. A cada patrón de bits se le ha añadido un 0 a la izquierda con el fin de obtener el patrón de 8 bits comúnmente utilizado hoy día. En la tercera columna se especifica el valor hexadecimal de cada patrón de 8 bits.

Símbolo	ASCII	Hex	Símbolo	ASCII	Hex	Símbolo	ASCII	Hex
avance de línea	00001010	0A	>	00111110	3E	^	01011110	5E
retorno de carro	00001011	0B	?	00111111	3F	_	01011111	5F
espacio	00100000	20	@	01000000	40	`	01100000	60
!	00100001	21	A	01000001	41	a	01100001	61
"	00100010	22	B	01000010	42	b	01100010	62
#	00100011	23	C	01000011	43	c	01100011	63
\$	00100100	24	D	01000100	44	d	01100100	64
%	00100101	25	E	01000101	45	e	01100101	65
&	00100110	26	F	01000110	46	f	01100110	66
'	00100111	27	G	01000111	47	g	01100111	67
(	00101000	28	H	01001000	48	h	01101000	68
)	00101001	29	I	01001001	49	i	01101001	69
*	00101010	2A	J	01001010	4A	j	01101010	6A
+	00101011	2B	K	01001011	4B	k	01101011	6B
,	00101100	2C	L	01001100	4C	l	01101100	6C
-	00101101	2D	M	01001101	4D	m	01101101	6D
.	00101110	2E	N	01001110	4E	n	01101110	6E
/	00111111	2F	O	01001111	4F	o	01101111	6F
0	00110000	30	P	01010000	50	p	01110000	70
1	00110001	31	Q	01010001	51	q	01110001	71
2	00110010	32	R	01010010	52	r	01110010	72
3	00110011	33	S	01010011	53	s	01110011	73
4	00110100	34	T	01010100	54	t	01110100	74
5	00110101	35	U	01010101	55	u	01110101	75
6	00110110	36	V	01010110	56	v	01110110	76
7	00110111	37	W	01010111	57	w	01110111	77
8	00111000	38	X	01011000	58	x	01111000	78
9	00111001	39	Y	01011001	59	y	01111001	79
:	00111010	3A	Z	01011010	5A	z	01111010	7A
;	00111011	3B	[	01011011	5B	{	01111011	7B
<	00111100	3C	\	01011100	5C		01111100	7C
=	00111101	3D	]	01011101	5D	}	01111101	7D

Circuitos para manipular representaciones en complemento a dos

Este apéndice presenta una serie de circuitos para invertir y sumar valores representados en notación de complemento a dos. Comenzaremos con el circuito de la Figura B.1 que convierte una representación en complemento a dos de cuatro bits a la representación correspondiente al negado de dicho valor. Por ejemplo, dada la representación en complemento a dos de 3, el circuito genera la representación de -3 . El circuito lleva a cabo la tarea siguiendo el mismo algoritmo que se presenta en el texto. Es decir, copia el patrón de derecha a izquierda hasta copiar el primer 1 y luego complementa cada uno de los bits restantes a medida que los pasa de la entrada a la salida. Puesto que una de las entradas de la puerta XOR situada más a la derecha está fija a 0, esta puerta se limitará a pasar su otra entrada a la salida. Sin embargo, esta salida también se pasa hacia la izquierda, como una de las entradas de la siguiente puerta XOR. Si esta salida es 1, la siguiente puerta XOR complementará su bit de entrada al pasarlo a la salida. Además, este 1 también pasará hacia la izquierda a través de la puerta OR, con el fin de afectar a la siguiente puerta. De esta manera, el primer 1 que se copie a la salida pasará también hacia la izquierda, haciendo que

Figura B.1 Un circuito que calcula el inverso de un patrón en complemento a dos.



todos los bits restantes queden complementados a medida que pasan hacia la salida.

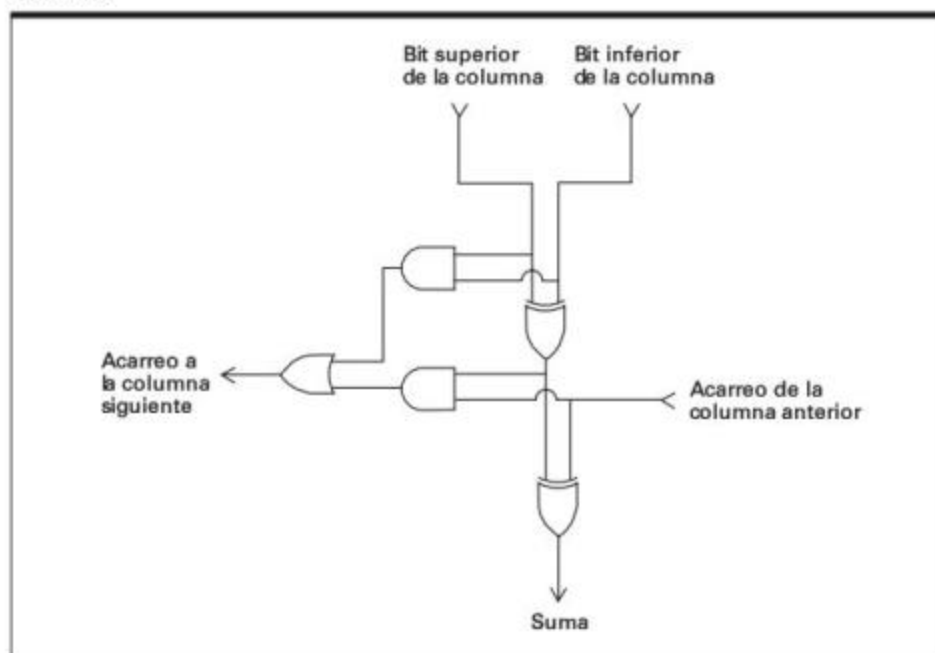
A continuación, consideramos el proceso de sumar dos valores representados en notación de complemento a dos. En particular, a la hora de resolver el problema

$$\begin{array}{r} + 0110 \\ + 1011 \\ \hline \end{array}$$

iremos columna a columna de derecha a izquierda ejecutando el mismo algoritmo para cada columna. Así, una vez que obtengamos un circuito para sumar una columna en este tipo de problema, podremos construir un circuito que sume varias columnas repitiendo simplemente el circuito de una sola columna.

El algoritmo para sumar una única columna en un problema de suma de varias columnas consiste en sumar los dos valores de la columna actual, añadirle a esa suma cualquier posible acarreo de la columna anterior, escribir el bit menos significativo de esta suma en la respuesta y transferir cualquier acarreo que se haya producido hacia la siguiente columna. El circuito de la Figura B.2 sigue precisamente este algoritmo. La puerta XOR superior determina la suma de los dos bits de entrada. La puerta XOR inferior añade a esta suma el valor del acarreo procedente de la columna anterior. Las dos puertas AND junto con la puerta OR pasan hacia la izquierda cualquier acarreo que se haya producido en el proceso. En particular, se generará un acarreo de salida igual a 1 si los dos bits de entrada originales de esta columna eran 1 o si la suma de estos bits es 1 y el acarreo de entrada también es 1.

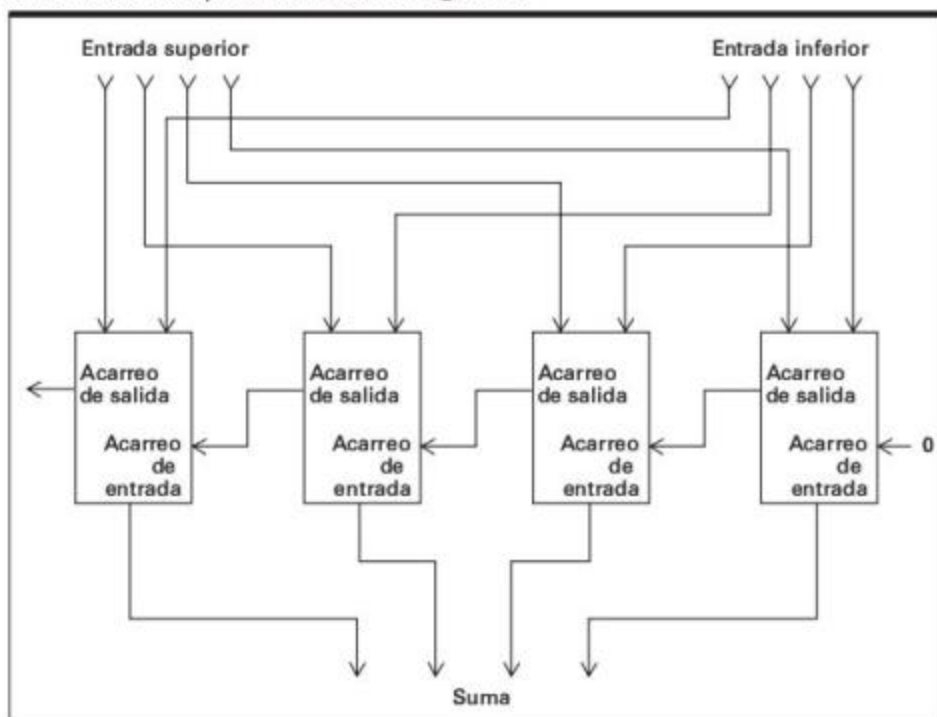
Figura B.2 Un circuito para sumar una única columna en un problema de suma de varias columnas.



La Figura B.3 muestra cómo pueden utilizarse copias de este circuito de una única columna para componer un circuito que calcule la suma de dos valores representados en un sistema de complemento a dos de cuatro bits. Cada rectángulo representa una copia del circuito de suma de una única columna. Observe que el valor de acarreo que se le entrega al rectángulo situado más a la derecha es siempre 0, porque no existe ningún acarreo de ninguna columna anterior. De forma similar, el acarreo generado por el rectángulo generado más a la izquierda se ignora.

El circuito de la Figura B.3 se conoce con el nombre de *sumador en cascada* porque la información de acarreo debe propagarse, o transmitirse en cascada, desde la columna situada más a la derecha hasta la columna situada más a la izquierda. Aunque su composición es bastante simple, estos circuitos son más lentos a la hora de realizar su función que otras versiones más inteligentes, como por ejemplo el sumador de acarreo anticipado, que minimiza esta propagación entre columnas. Por tanto, el circuito de la Figura B.3, aunque resulta suficiente para nuestros propósitos, no es el circuito utilizado en las máquinas de hoy en día.

Figura B.3 Un circuito para sumar dos valores en notación de complemento a dos utilizando cuatro copias del circuito de la Figura B.2





a p é n d i c e

C

Un lenguaje máquina simple

En este apéndice presentamos un lenguaje máquina simple pero representativo. Comenzaremos explicando la arquitectura de la propia máquina.

La arquitectura de la máquina

La máquina tiene 16 registros de propósito general numerados de 0 a F (en hexadecimal). Cada registro tiene una longitud de un byte (ocho bits). Para identificar los registros dentro de las instrucciones, a cada registro se le asigna el patrón de cuatro bits distintivo que representa su número de registro. Así, el registro 0 se identifica mediante 0000 (0 en hexadecimal), mientras que el registro 4 se identifica mediante 0100 (4 en hexadecimal).

Hay 256 celdas en la memoria principal de la máquina. Cada celda tiene asignada una dirección unívoca consistente en un número entero comprendido en el rango que va de 0 a 255. Por tanto, una dirección puede representarse mediante un patrón de ocho bits que va de 00000000 a 11111111 (o un valor hexadecimal en el rango comprendido entre 00 y FF).

Se supone que los valores en punto flotante están almacenados en el formato de ocho bits que se ha explicado en la Sección 1.7 y resumido en la Figura 1.26.

El lenguaje de la máquina

Cada instrucción del lenguaje máquina tiene dos bytes de longitud. Los primeros 4 bits proporcionan el código de operación, mientras que los 12 bits restantes forman el campo de operando. La siguiente tabla enumera las instrucciones en notación hexadecimal, proporcionando una breve descripción de cada una de ellas. Se utilizan las letras R, S y T, en lugar de dígitos hexadecimales, en aquellos campos que representen un identificador de registro que varía dependiendo de la aplicación concreta de la instrucción. Las letras X e Y se emplean, en lugar de dígitos hexadecimales, en aquellos campos variables que no representan un registro.

Código operando	Operando	Descripción
1	RXY	Cargar (LOAD) el registro R con el patrón de bits que se encuentra en la celda de memoria cuya dirección es XY. <i>Ejemplo:</i> 14A3 haría que el contenido de la celda de memoria ubicada en la dirección A3 se almacenara en el registro 4.
2	RXY	Cargar (LOAD) el registro R con el patrón de bits XY. <i>Ejemplo:</i> 20A3 haría que se almacenara el valor A3 en el registro 0.
3	RXY	Almacenar (STORE) en la celda de memoria cuya dirección es XY el patrón de bits contenido en el registro R. <i>Ejemplo:</i> 35B1 haría que el contenido del registro 5 se almacenara en la celda de memoria cuya dirección es B1.
4	ORS	Copiar (MOVE) al registro S el patrón de bits contenido en el registro R. <i>Ejemplo:</i> 40A4 haría que el contenido del registro A se copiara en el registro 4.
5	RST	Sumar (ADD) los patrones de bits de los registros S y T como si fueran representaciones en complemento a dos y almacenar el resultado en el registro R. <i>Ejemplo:</i> 5726 haría que se sumaran los valores binarios de los registros 2 y 6, y que el resultado de la suma se almacenara en el registro 7.
6	RST	Sumar (ADD) los patrones de bits de los registros S y T como si fueran representaciones en notación de punto flotante y almacenar el resultado en el registro R. <i>Ejemplo:</i> 634E haría que se sumaran los valores de los registros 4 y E como valores en punto flotante y que el resultado se almacenara en el registro 3.
7	RST	Combinar mediante OR los patrones de bits de los registros S y T y almacenar el resultado en el registro R. <i>Ejemplo:</i> 7CB4 haría que el resultado de combinar mediante OR el contenido de los registros B y 4 se almacenara en el registro C.
8	RST	Combinar mediante AND los patrones de bits de los registros S y T y almacenar el resultado en el registro R. <i>Ejemplo:</i> 8045 haría que el resultado de combinar mediante AND el contenido de los registros 4 y 5 se almacenara en el registro 0.
9	RST	Combinar mediante EXCLUSIVE OR los patrones de bits de los registros S y T y almacenar el resultado en el registro R. <i>Ejemplo:</i> 95F3 haría que el resultado de combinar mediante la operación EXCLUSIVE OR el contenido de los registros F y 3 se almacenara en el registro 5.
A	R0X	Rotar (ROTATE) X bits hacia la derecha el patrón de bits contenido en el registro R. Los bits que salen por el extremo de menor peso se vuelven a insertar en el extremo de mayor peso. <i>Ejemplo:</i> A403 haría que el contenido del registro 4 se rotara 3 bits hacia la derecha, de forma circular.
B	RXY	Saltar (JUMP) a la instrucción ubicada en la celda de memoria correspondiente a la dirección XY si el patrón de bits contenido en el registro R es igual al patrón de bits contenido en el registro número 0. En caso contrario, continuar con la secuencia normal de ejecución. (El salto se implementa copiando XY en el contador de programa durante la fase de ejecución.) <i>Ejemplo:</i> B43C compararía primero el contenido del registro 4 con el del registro 0. Si los dos fueran iguales, colocaría el patrón 3C en el contador de programa, de modo que la siguiente instrucción que se ejecute sea la que está ubicada en la siguiente dirección de memoria. En caso contrario, no se haría nada y la ejecución del programa continuaría con su secuencia normal.
C	000	Detener (HALT) la ejecución. <i>Ejemplo:</i> C000 haría que se detuviera la ejecución del programa.



apéndice

D

Lenguajes de programación de alto nivel

Este apéndice proporciona un breve resumen sobre cada uno de los lenguajes utilizados como ejemplos en el Capítulo 6.

Ada

El lenguaje Ada, denominado así en honor de Augusta Ada Byron (1815–1851), que fue una defensora de Charles Babbage y la hija del poeta Lord Byron, fue desarrollado por iniciativa del Departamento de Defensa de Estados Unidos en un intento de obtener un único lenguaje de propósito general que satisficiera todas sus necesidades de desarrollo de software. Uno de los principales objetivos durante el diseño de Ada fue incorporar características para programar sistemas de computadora en tiempo real que pudieran utilizarse como parte de máquinas de mayor tamaño, como por ejemplo sistemas de guiado de misiles, sistemas de control medioambiental dentro de edificios y sistemas de control para automóviles y pequeños electrodomésticos. Ada contiene por tanto características para expresar actividades en entornos de procesamiento paralelo, además de una serie de técnicas de utilidad para la gestión de casos especiales (denominados excepciones) que pueden surgir dentro del entorno de aplicación. Aunque originalmente se desarrolló como un lenguaje imperativo, las versiones más recientes de Ada han adoptado el paradigma de la orientación a objetos.

El diseño del lenguaje Ada ha puesto siempre el énfasis en aquellas características que permiten un desarrollo eficiente de software fiable, una característica que queda ilustrada por el hecho de que todo el software de control interno del avión Boeing 777 fue escrito en Ada. Esta es también una de las principales razones por las que se utilizó Ada como punto de partida a la hora de desarrollar el lenguaje SPARK, como se ha indicado en el Capítulo 5.

C

El lenguaje C fue desarrollado por Dennis Ritchie en los Bell Laboratories a principios de la década de 1970. Aunque diseñado originalmente como un lenguaje para el desarrollo de software de sistemas, C ha conseguido una gran

popularidad en toda la comunidad de programadores y ha sido estandarizado por el instituto ANSI.

C se concibió originalmente como un lenguaje situado un escalón por encima del lenguaje máquina. En consecuencia, su sintaxis es muy seca, si la comparamos con otros lenguajes de alto nivel que emplean palabras en inglés completas para expresar algunas de las primitivas que en C se representan mediante símbolos especiales. Esta sequedad permite una representación eficiente de algoritmos complejos, lo cual es una de las principales razones de la popularidad del lenguaje C. (A menudo, una representación concisa es más legible que otra muy larga.)

C++

El lenguaje C++ fue desarrollado por Bjarne Stroustrup en los Bell Laboratories como una versión ampliada del lenguaje C. El objetivo era obtener un lenguaje compatible con el paradigma de la orientación a objetos. Actualmente, C++ no es solo uno de los principales lenguajes orientados a objetos, sino que se ha utilizado como punto de partida para el desarrollo de otros dos de los principales lenguajes orientados a objetos: Java y C#.

C#

El lenguaje C# fue desarrollado por Microsoft como una herramienta para el entorno .NET, que es un sistema muy completo para el desarrollo de software de aplicación para máquinas que ejecuten el software de sistemas de Microsoft. El lenguaje C# es muy similar a C++ y Java. De hecho, la razón de que Microsoft introdujera C# como un lenguaje diferente, no es porque se trate de un lenguaje verdaderamente nuevo, sino que como lenguaje diferente que es, Microsoft podía personalizar características específicas del lenguaje sin preocuparse de cumplir con los estándares que afectaban a otros lenguajes y sin preocuparse tampoco por los derechos de propiedad intelectual de otras empresas. Por tanto, la novedad de C# radica en su papel como lenguaje principal para el desarrollo de software que utilice el entorno .NET. Con el respaldo de Microsoft, C# y el entorno .NET prometen ser actores relevantes en el campo del desarrollo software durante los años venideros.

Fortran

FORTRAN es el acrónimo de FORMula TRANslator (traductor de fórmulas). Este lenguaje fue uno de los primeros lenguajes de alto nivel que se desarrollaron (fue presentado en 1957) y fue también uno de los primeros lenguajes en obtener una amplia aceptación dentro de la comunidad de los profesionales de la computación. A lo largo de los años, su descripción oficial ha sufrido numerosas ampliaciones, lo que significa que el lenguaje FORTRAN actual es muy diferente del original. De hecho, estudiando la evolución de FORTRAN podemos ver los efectos que las sucesivas investigaciones han ido teniendo en el diseño de los lenguajes de programación. Aunque originalmente se diseñó como lenguaje imperativo, las versiones más recientes de FORTRAN incluyen ahora muchas características de orientación a objetos. FORTRAN continúa siendo un lenguaje muy popular dentro de la comunidad científica. En particu-

lar, muchos paquetes estadísticos y de análisis numérico están escritos en FORTRAN y muy probablemente sigan estándolo en el futuro.

Java

Java es un lenguaje orientado a objetos desarrollado por Sun Microsystems a principios de la década de 1990. Sus diseñadores tomaron prestadas numerosas ideas de C y C++. El entusiasmo por Java se debe no al propio lenguaje, sino a su implementación universal y al impresionante número de plantillas prediseñadas que hay disponibles en el entorno de programación Java. Lo de implementación universal significa que un programa escrito en Java puede ejecutarse de forma eficiente en un amplio rango de máquinas, mientras que la disponibilidad de plantillas significa que puede desarrollarse software complejo con relativa facilidad. Por ejemplo, las plantillas tipo applet y servlet permiten simplificar el desarrollo de software para la World Wide Web.



ap é n d i c e

E

Equivalencia de las estructuras iterativas y recursivas

En este apéndice vamos a utilizar el lenguaje de Bare Bones del Capítulo 11 como herramienta para responder a la cuestión planteada en el Capítulo 4 en relación con la potencia de las estructuras iterativas y recursivas. Recuerde que Bare Bones contiene solo tres sentencias de asignación (`clear`, `incr` y `decr`) y una estructura de control (construida a partir de una pareja de sentencias `while-end`). Este lenguaje tan simple tiene la misma potencia de computación que una máquina de Turing; por tanto, si aceptamos la tesis de Church-Turing, podemos concluir que todo problema con una solución algorítmica tendrá una solución expresable en Bare Bones.

El primer paso en la comparación de las estructuras iterativas y recursivas consiste en sustituir la estructura iterativa de Bare Bones por una estructura recursiva. Haremos esto eliminando las sentencias `while` y `end` del lenguaje y proporcionando, en su lugar, la capacidad de dividir un programa Bare Bones en diferentes unidades, con la posibilidad de invocar una de esas unidades desde otro punto del programa. De manera más precisa, lo que proponemos es que cada programa en el lenguaje modificado consista en una serie de unidades de programa sintácticamente disjuntas. Vamos a suponer que cada programa tiene que contener exactamente una unidad denominada `MAIN` que tenga la siguiente estructura sintáctica

```
MAIN: begin;  
      .  
      .  
      .  
      end;
```

(donde los puntos representan otras sentencias Bare Bones) y quizá otras unidades (semánticamente subordinadas a `MAIN`) que tendrán la estructura:


```

uni dad: begi n;
.
.
.
ret urn;

```

(donde *uni dad* representa el nombre de la unidad que tiene la misma sintaxis que los nombres de variable). La semántica de esta estructura particionada es que el programa siempre inicia su ejecución al principio de la unidad *MAIN* y se detiene al alcanzar la sentencia final de dicha unidad. Las otras unidades de programa distintas de *MAIN* pueden invocarse como procedimientos por medio de la sentencia condicional

```
if nombre not 0 perform uni dad;
```

(donde *nombre* representa cualquier nombre de variable y *uni dad* representa cualquiera de los nombres de unidades de programa diferentes de *MAIN*). Además, permitiremos que las otras unidades diferentes de *MAIN* se invoquen de manera recursiva.

Con estas características añadidas, podemos simular la estructura *while-end* incluida en el lenguaje Bare Bones original. Por ejemplo, un programa Bare Bones de la forma

```

while X not 0 do;
S;
end;

```

(donde *S* representa cualquier secuencia de sentencias Bare Bones) puede sustituirse por la siguiente estructura de unidad

```

MAIN: begi n;
      if X not 0 perform uni dadA;
      end;
uni dadA: begi n;
          S;
          if X not 0 perform uni dadA;
          ret urn;

```

En consecuencia, podemos concluir que el lenguaje modificado tiene todas las capacidades del lenguaje Bare Bones original.

También se puede demostrar que cualquier problema que pueda resolverse utilizando el lenguaje modificado, puede resolverse empleando Bare Bones. Un método de hacer esto consiste en mostrar cómo cualquier algoritmo expresado en el lenguaje modificado podría escribirse también en el lenguaje Bare Bones original. Sin embargo, esto implica una descripción explícita de cómo simular estructuras recursivas mediante la estructura *while-end* de Bare Bones.

Para nuestros propósitos, resulta más simple recurrir a la tesis de Church-Turing, tal como la hemos presentado en el Capítulo 11. En particular, la tesis de Church-Turing combinada con el hecho de que Bare Bones tiene la misma potencia que las máquinas de Turing, indica que no hay ningún lenguaje que pueda ser más potente que nuestro lenguaje Bare Bones original. Por tanto,

cualquier problema resoluble en nuestro lenguaje modificado puede también resolverse utilizando Bare Bones.

Concluimos así que la potencia del lenguaje modificado coincide con la del lenguaje Bare Bones original. La única diferencia entre los dos lenguajes es que uno de ellos proporciona una estructura de control iterativa y el otro proporciona un mecanismo de recursión. Por tanto, las dos estructuras de control son equivalentes, de hecho, en términos de potencia de computación.



a p é n d i c e

F

Respuestas a las cuestiones y ejercicios

Capítulo 1

Sección 1.1

1. Una y solo una de las dos entradas superiores tiene que ser 1 y la entrada inferior tiene que ser también 1.
2. El 1 en la entrada inferior es transformado en 0 por la puerta NOT, haciendo que la salida de la puerta AND sea 0. Por tanto, ambas entradas de la puerta OR son 0 (recuerde que la entrada superior del biestable se mantiene a 0), por lo que la salida de la puerta OR pasa a ser 0. Esto significa que la salida de la puerta AND continuará teniendo el valor 0 después de que la entrada inferior del biestable vuelva a 0.
3. La salida de la puerta OR superior estará a 1, haciendo que la puerta NOT superior genere una salida igual a 0. Esto hará que la puerta OR inferior genere un 0, haciendo que la puerta NOT inferior tenga una salida igual a 1. Este 1 es el que se ve como salida del biestable, además de ser realimentado hacia la puerta OR superior, donde mantiene la salida de dicha puerta en el nivel 1, incluso después de que la entrada del biestable haya vuelto a 0.
4. a. El circuito completo es equivalente a una única puerta OR.
b. Este circuito completo también es equivalente a una única puerta XOR.
5. a. 6AF2 b. E85517 c. 48
6. a. 01011111110110010111
b. 0110000100001010
c. 1010101111001101
d. 0000000100000000

Sección 1.2

1. En el primer caso, la celda de memoria número 6 termina conteniendo el valor 5. En el segundo caso, termina conteniendo el valor 8.

2. El Paso 1 borra el valor original contenido en la celda número 3, al escribirse allí el nuevo valor. En consecuencia, el Paso 2 no almacena el valor original de la celda número 3 en la celda número 2. El resultado es que ambas celdas terminan teniendo el valor que estaba originalmente almacenado en la celda número 2. Un procedimiento correcto sería el siguiente:

Paso 1. Desplazar el contenido de la celda número 2 a la celda número 1.

Paso 2. Desplazar el contenido de la celda número 3 a la celda número 2.

Paso 3. Desplazar el contenido de la celda número 1 a la celda número 3.

3. 32768 bits.

Sección 1.3

1. Una extracción más rápida de los datos y tasas de transferencia más altas.
2. El punto que hay que recordar es que la lentitud del movimiento mecánico, comparada con la velocidad de funcionamiento interno de la computadora, dicta que debemos minimizar el número de veces que haya que desplazar los cabezales de lectura/escritura. Si rellenamos una superficie completa antes de comenzar con la siguiente, debemos desplazar el cabezal de lectura/escritura cada vez que terminemos con una pista. El número de desplazamientos será, por tanto, aproximadamente igual al número de total de pistas de las dos superficies. Sin embargo, si alternamos entre una superficie y otra conmutando electrónicamente entre los cabezales de lectura/escritura, solo tendremos que desplazar los cabezales de lectura/escritura después de haber rellenado cada cilindro.
3. En esta aplicación, la información debe extraerse del almacenamiento masivo de forma aleatoria, lo que requeriría mucho tiempo si estamos utilizando el sistema en espiral que se emplea en los discos CD y DVD. Además, la tecnología actual no permite actualizar porciones individuales de los datos en un CD o un DVD.
4. El espacio de almacenamiento se asigna en unidades de sectores físicos (en realidad, en unidades de grupos de sectores, en la mayoría de los casos). Si el último sector físico no está lleno, puede añadirse texto adicional sin incrementar el espacio de almacenamiento asignado al archivo. Si el último sector físico está lleno, cualquier nueva adición al documento requerirá la asignación de sectores físicos adicionales.
5. Las unidades flash no requieren que se efectúe ningún movimiento físico, por lo que tienen tiempos de respuesta más cortos y no sufren desgaste físico.
6. Un buffer es un área de almacenamiento de datos utilizada para almacenar datos de manera temporal, usualmente como forma de absorber las incoherencias existentes entre el origen de los datos y su destino final.

Sección 1.4

1. Ciencias de la computación.

2. Los dos patrones son iguales, salvo por el hecho de que el sexto bit comenzando por el extremo de menor peso es siempre igual a 0 para las letras mayúsculas y a 1 para la minúsculas.

3. a.

00100010	01010011	01110100	01101111
01110000	00100001	00100010	00100000
01000011	01101000	01100101	01110010
01111001	01101100	00100000	01110011
01101000	01101111	01110101	01110100
01100101	01110100	00101110	
- b.

01000100	01101111	01100101	01110011
00100000	00110010	00100000	00101011
00100000	00110011	00100000	00111101
00100000	00110101	00111111	

4.



5. a. 5 b. 9 c. 11 d. 6 e. 16 f. 18
6. a. 110 b. 1101 c. 1011 d. 10010 e. 11011 f. 100
7. En 24 bits, podemos almacenar tres símbolos utilizando ASCII. Por tanto, podemos almacenar valores numéricos hasta 999. Sin embargo, si usamos los bits como dígitos binarios podemos almacenar valores hasta 16.777.215.
8. a. 15.15 b. 51.0.128 c. 10.160
9. Las representaciones geométricas permiten realizar más fácilmente los cambios de escala que las imágenes codificadas en forma de mapa de bits. Sin embargo, las representaciones geométricas no proporcionan normalmente la misma calidad fotográfica que los mapas de bits. De hecho, como se explica en la Sección 1.8, las representaciones JPEG de mapas de bits son muy populares en el campo de la fotografía.
10. Con una tasa de muestreo de 44.100 muestras por segundo, una hora de música estéreo requeriría 635.040.000 bytes de almacenamiento. Por tanto, se llenaría aproximadamente un CD, cuya capacidad es ligeramente mayor que 600MB.

Sección 1.5

1. a. 42 b. 33 c. 23 d. 6 e. 31
2. a. 100000 b. 1000000 c. 1100000 d. 1111 e. 11011
3. a. $3\frac{1}{4}$ b. $5\frac{7}{8}$ c. $2\frac{1}{2}$ d. $6\frac{3}{8}$ e. $\frac{5}{8}$
4. a. 100.1 b. 10.11 c. 1.001 d. 0.0101 e. 101.101
5. a. 100111 b. 1011.110 c. 100000 d. 1000.00

Sección 1.6

1. a. 3 b. 15 c. -4 d. -6 e. 0 f. -16
2. a. 00000110 b. 11111010 c. 11101111
d. 00001101 e. 11111111 f. 00000000
3. a. 11111111 b. 10101011 c. 00000100
d. 00000010 e. 00000000 f. 10000001
4. a. Con 4 bits el valor mayor es 7 y el más pequeño es -8.
b. Con 6 bits el valor mayor es 31 y el más pequeño es -32.
c. Con 8 bits el valor mayor es 127 y el más pequeño es -128.
5. a. 0111 ($5 + 2 = 7$) b. 0100 ($3 + 1 = 4$) c. 1111 ($5 + (-6) = -1$)
d. 0001 ($-2 + 3 = 1$) e. 1000 ($-6 + (-2) = -8$)
6. a. 0111 b. 1011 (desbordamiento) c. 0100 (desbordamiento)
d. 0001 e. 1000 (desbordamiento)
7. a.
$$\begin{array}{r} 0110 \\ + 0001 \\ \hline 0111 \end{array}$$
 b.
$$\begin{array}{r} 0011 \\ + 1110 \\ \hline 0001 \end{array}$$
 c.
$$\begin{array}{r} 0100 \\ + 1010 \\ \hline 1110 \end{array}$$
 d.
$$\begin{array}{r} 0010 \\ + 0100 \\ \hline 0110 \end{array}$$
 e.
$$\begin{array}{r} 0001 \\ + 1011 \\ \hline 1100 \end{array}$$
8. No. El desbordamiento tiene lugar cuando se intenta almacenar un número demasiado grande para el sistema que se está utilizando. Cuando se suma un valor positivo con un valor negativo, el resultado debe encontrarse entre los dos valores que se están sumando. Por tanto, el resultado será lo suficientemente pequeño como para poder almacenarlo sin ningún error.
9. a. 6 porque $1110 \rightarrow 14 - 8$
b. -1 porque $0111 \rightarrow 7 - 8$
c. 0 porque $1000 \rightarrow 8 - 8$
d. -6 porque $0010 \rightarrow 2 - 8$
e. -8 porque $0000 \rightarrow 0 - 8$
f. 1 porque $1001 \rightarrow 9 - 8$
10. a. 1101 porque $5 + 8 = 13 \rightarrow 1101$
b. 0011 porque $-5 + 8 = 3 \rightarrow 0011$
c. 1011 porque $3 + 8 = 11 \rightarrow 1011$
d. 1000 porque $0 + 8 = 8 \rightarrow 1000$
e. 1111 porque $7 + 8 = 15 \rightarrow 1111$
f. 0000 porque $-8 + 8 = 0 \rightarrow 0000$
11. No. El valor mayor que puede almacenarse en notación exceso ocho es 7, representado por 1111. Para representar un valor mayor, hay que emplear al menos notación exceso 16 (que utiliza patrones de 5 bits). De forma similar, el valor 6 no se puede representar en notación exceso cuatro (el valor mayor que se puede representar en notación exceso cuatro es 3).

Sección 1.7

1. a. $\frac{5}{6}$ b. $3\frac{1}{4}$ c. $\frac{9}{32}$ d. $-1\frac{1}{2}$ e. $-(\frac{1}{64})$
2. a. 01101011 b. 01111010 (error de truncamiento)
c. 01001100 d. 11101110 e. 11111000 (error de truncamiento)

3. 01001001 ($9/16$) es mayor que 00111101 ($13/32$). He aquí una forma simple de determinar cuál de los dos patrones representa un valor mayor:

Caso 1. Si los bits de signo son diferentes, el valor mayor será igual que el que tenga el bit de signo igual a 0.

Caso 2. Si los dos bits de signo son iguales a 0, analizamos la parte restante de los dos patrones de izquierda a derecha, hasta encontrar una posición de bit en la que los dos patrones difieran. El patrón que contenga un 1 en dicha posición representará el valor mayor.

Caso 3. Si los dos bits de signo son iguales a 1, analizamos la parte restante de los dos patrones de izquierda a derecha, hasta encontrar una posición de bit en la que los dos patrones difieran. El patrón que tenga un 0 en dicha posición representará el valor mayor.

La simplicidad de este proceso de comparación es una de las razones para representar el exponente en los sistemas de punto flotante mediante notación en exceso, en lugar de mediante notación en complemento a dos.

4. El valor mayor sería $7^{1/4}$, que está representado por el patrón 01111111. En cuanto al valor positivo más pequeño, podríamos decir que existen dos respuestas "correctas". En primer lugar, si nos ajustamos al proceso de codificación descrito en el texto, que requiere que el bit más significativo de la mantisa sea 1 (denominado forma normalizada), la respuesta es $1/32$, que está representado por el patrón 00001000. Sin embargo, la mayoría de las máquinas no imponen esta restricción para valores próximos a 0. En tales máquinas, la respuesta correcta sería $1/256$ que está representado por el valor 00000001.

Sección 1.8

1. Codificación por longitud de recorrido, codificación dependiente de la frecuencia, codificación relativa y codificación por diccionario.
2. 121321112343535
3. Los dibujos animados en color están compuestos por bloques de color homogéneo con bordes bien definidos. Además, el número de colores implicado es limitado.
4. No. Tanto GIF como JPEG son sistemas de compresión con pérdidas, lo que quiere decir que se perderán detalles de la imagen.
5. El estándar de línea base JPEG aprovecha el hecho de que el ojo humano no es tan sensible a los cambios de color como a los cambios de brillo. De este modo, lo que se hace es reducir el número de bits utilizado para representar la información de color, sin una pérdida de calidad apreciable en la imagen.
6. Enmascaramiento temporal y enmascaramiento de frecuencia.
7. Al codificar la información, se realizan aproximaciones. En el caso de datos numéricos, estas aproximaciones se van acumulando al realizar cálculos, lo que puede conducir a la obtención de resultados erróneos. Las aproximaciones no son tan críticas en el caso de las imágenes y del sonido, porque los datos codificados normalmente tan solo se almacenan, transfieren y

reproducen. Sin embargo, si las imágenes o el sonido se reprodujeran, regrabaran y luego se recodificaran de manera repetida, dichas aproximaciones podrían acumularse y terminar produciendo datos no utilizables.

Sección 1.9

1. b, c y e
2. Sí. Si se produce un número par de errores en un byte, la técnica de paridad no permite detectarlos.
3. En este caso, los errores se producen en los bytes a y d de la Cuestión 1. La respuesta a la Cuestión 2 sigue siendo la misma.
4.

a.	001010111 001101000 101100101
	101110010 101100101 000100000
	001100001 101110010 101100101
	000100000 001111001 101101111
	001110101 100111111
b.	100100010 101001000 101101111
	101110111 100111111 100100010
	000100000 001000011 001101000
	101100101 101110010 001111001
	101101100 000100000 001100001
	001110011 001101011 101100101
	001100100 100101110
c.	000110010 100101011 100110011
	000111101 100110101 100101110
5. a. BED b. CAB c. HEAD
6. Una solución es la siguiente:

A	0 0 0 0 0
B	1 1 1 0 0
C	0 1 1 1 1
D	1 0 0 1 1

Capítulo 2

Sección 2.1

1. En algunas máquinas, se trata de un proceso en dos pasos consistente en leer primero el contenido de la primera celda del registro y luego escribirlo desde el registro a la celda de destino. En la mayoría de las máquinas, esto se realiza de forma directa en un solo paso, sin utilizar un registro intermedio.
2. El valor que hay que escribir, la dirección de celda en la que hay que escribirlo y el comando necesario para escribir.
3. Los registros de propósito general se utilizan para almacenar los datos inmediatamente aplicables a la operación que se está llevando a cabo; la

memoria principal se emplea para almacenar datos que se necesitarán próximamente y el almacenamiento masivo se utiliza para almacenar datos que probablemente no vayan a ser necesarios en un futuro próximo.

Sección 2.2

1. El término *mover* tiene a menudo la connotación de eliminar algo de una ubicación y colocarlo en otra, dejando un hueco detrás. En la mayoría de los casos, en una computadora, dicha eliminación no tiene lugar. En lugar de ello, lo que se suele hacer con el objeto que se está moviendo es copiarlo (o clonarlo) en la nueva ubicación.
2. Una técnica común, denominada direccionamiento relativo, consiste en indicar a qué distancia hay que saltar, en lugar de indicar la dirección absoluta a la que hay que saltar. Por ejemplo, una instrucción podría decirle a la computadora que saltara hacia adelante tres instrucciones o que saltara hacia atrás dos instrucciones. Sin embargo, es preciso darse cuenta de que será necesario modificar dichas instrucciones si posteriormente se insertan instrucciones adicionales entre el origen y el destino del salto.
3. En este caso, podría argumentarse en favor de cualquiera de las dos respuestas. La instrucción se establece en forma de un salto condicional. Sin embargo, puesto que la condición de que 0 sea igual 0 siempre se satisface, el salto se llevará siempre a cabo como si no se hubiera establecido ninguna condición. A menudo podemos encontrarnos con máquinas que tienen este tipo de instrucciones en su repertorio, porque dichas instrucciones proporcionan un diseño eficiente. Por ejemplo, si una máquina está diseñada para ejecutar una instrucción con una estructura tal como "Si ... saltar a...", dicho tipo de instrucción puede emplearse tanto para expresar saltos condicionales como incondicionales.
4. 156C = 0001010101101100
166D = 0001011001101101
5056 = 0101000001010110
306E = 0011000001101110
C000 = 1100000000000000
5. a. Almacenar (STORE) el contenido del registro 6 en la celda de memoria número 8A.
b. Saltar (JUMP) a la posición DE si el contenido del registro A es igual al del registro 0.
c. Combinar mediante AND el contenido de los registros 3 y C, almacenando el resultado en el registro 0.
d. Mover (MOVE) el contenido del registro F al registro 4.
6. La instrucción 15AB requiere que el procesador solicite a la circuitería de memoria el contenido de la celda de memoria situada en la dirección AB. Este valor, después de ser obtenido de la memoria, se almacena en el registro 5. La instrucción 25AB no requiere que se extraiga el valor de la memoria; en lugar de ello el valor AB se almacena directamente en el registro 5.
7. a. 2356 b. A503 c. 80A5

Sección 2.3

1. 34 Hexadecimal
2. a. 0F b. C3
3. a. 00 b. 01 c. Cuatro veces
4. Se detiene. Es un ejemplo de lo que a menudo se denomina código auto-modificante. Es decir, el programa se modifica a sí mismo. Observe que las dos primeras instrucciones colocan el valor hexadecimal C0 en la posición de memoria F8, mientras que las dos siguientes instrucciones colocan el valor 00 en la posición F9. Por tanto, en el momento que la máquina alcanza la instrucción contenida en la posición F8, se encuentra con la instrucción de detención (C000) que se ha escrito allí.

Sección 2.4

1. a. 00001011 b. 10000000 c. 00101101
 d. 11101011 e. 11101111 f. 11111111
 g. 11100000 h. 01101111 i. 11010010
2. 00111100 con la operación AND.
3. 00111100 con la operación XOR.
4. a. El resultado final es 0 si la cadena contiene un número par de 1s. En caso contrario es 1.
 b. El resultado es el valor del bit de paridad, si trabajamos con paridad par.
5. La operación lógica XOR se asemeja a la suma, salvo en el caso de que ambos operandos sean 1, en cuyo caso XOR produce un 0, mientras que la suma es 10. (Por tanto, la operación XOR puede considerarse una operación de suma pero sin acarreo.)
6. Utilice AND con la máscara 11011111 para cambiar minúsculas a mayúsculas. Utilice OR con la máscara 00100000 para cambiar mayúsculas a minúsculas.
7. a. 01001101 b. 11100001 c. 11101111
8. a. 57 b. B8 c. 6F d. 6A
9. 5
10. 00110110 en complemento a dos; 01011110 en punto flotante. Lo importante aquí es que el procedimiento utilizado para sumar los valores es diferente dependiendo de la interpretación que le demos a los patrones de bits.
11. Una solución sería la siguiente:
 12A7 (Cargar, LOAD, el registro 2 con el contenido de la celda de memoria A7.)
 2380 (Cargar, LOAD, el registro 3 con el valor 80.)
 7023 (Combinar mediante OR los registros 2 y 3 almacenando el resultado en el registro 0.)
 30A7 (Almacenar, STORE, el contenido del registro 0 en la celda de memoria A7.)
 C000 (Parar, HALT.)

12. Una solución sería la siguiente:

15E0 (Cargar, LOAD, el registro 5 con el contenido de la celda de memoria E0.)
A502 (Rotar, ROTATE, 2 bits hacia la izquierda el contenido del registro 5.)
260F (Cargar, LOAD, el registro 6 con el valor 0F.)
8056 (Combinar mediante AND los registros 5 y 6, almacenando el resultado en el registro 0.)
30E1 (Almacenar, STORE, el contenido del registro 0 en la celda de memoria E1.)
C000 (Parar, HALT.)

Sección 2.5

1.
 - a. 37B5
 - b. Un millón de veces.
 - c. No. Una página de texto típica contiene menos de 4.000 caracteres. Por tanto, la capacidad de imprimir cinco páginas por minuto indica una velocidad de impresión de no más de 20.000 caracteres por minuto, que es muy inferior a un millón de caracteres por segundo. (Lo importante aquí es que una computadora puede enviar caracteres a una impresora mucho más rápido de lo que la impresora los puede imprimir; por tanto, la impresora necesita una forma de decirle a la computadora que espere.)
2. El disco efectuará 50 revoluciones en un segundo, lo que quiere decir que cada segundo pasarán 800 sectores bajo el cabezal de lectura/escritura. Puesto que cada sector contiene 1.024 bytes, los bits pasarán bajo el cabezal de lectura/escritura a aproximadamente 6,5 Mbps. Por tanto, la comunicación entre la controladora y la unidad de disco tendrá que tener una velocidad al menos igual a esta, para que la controladora no se quede rezagada a la hora de leer datos del disco.
3. Una novela de 300 páginas codificada mediante Unicode ocupa unos 2 MB o 16.000.000 de bits. Por tanto, se requerirán aproximadamente 0,3 segundos para transferir la novela completa a 54 Mbps.

Sección 2.6

1. La cadena de procesamiento contendrá las instrucciones B1B0 (que se está ejecutando), 5002 y quizá también B0AA. Si el valor del registro 1 es igual al valor del registro 0, se ejecutará el salto a la dirección B0 y todo el esfuerzo ya invertido en insertar las instrucciones en la cola de procesamiento se habrá desperdiciado. Por el contrario, lo que no se desperdicia es ningún tiempo, porque ese esfuerzo invertido en cargar dichas instrucciones no ha requerido tiempo adicional.
2. Si no se toma ninguna precaución, la información existente en las posiciones de memoria F8 y F9 se extraerá como si fuera una instrucción, antes de que la parte anterior del programa haya tenido la posibilidad de modificar el contenido de esas celdas.
3.
 - a. El procesador que está tratando de sumar 1 a la celda puede leer primero el valor de dicha celda. A continuación, el otro procesador lee el valor de

la celda. (Observe que en este punto ambos procesadores habrán extraído el mismo valor.) Si el primer procesador finaliza ahora la suma y escribe el resultado en la celda antes de que el segundo procesador finalice la resta y escriba su resultado, el valor final contenido en la celda reflejará únicamente la actividad realizada por el segundo procesador.

- b. Los procesadores pueden leer los datos de la celda como antes, pero esta vez el segundo procesador podría escribir su resultado antes que el primero. Por tanto, el valor final de la celda solo reflejará la actividad que ha llevado a cabo el primer procesador.

Capítulo 3

Sección 3.1

1. Un ejemplo tradicional sería la cola de personas que están esperando para adquirir las entradas para un determinado evento. En este caso, puede haber alguien que trate de "colarse", lo que violaría la estructura FIFO.
2. Opciones (b), (c) y (e)
3. Los sistemas empotrados se centran a menudo en tareas dedicadas, mientras que los PC son computadoras de propósito general. Los sistemas empotrados suelen tener recursos más limitados que los PC de una antigüedad comparable, pero pueden tener que enfrentarse a límites temporales muy estrictos, con una intervención humana mínima.
4. El tiempo compartido hace referencia a aquella situación en la que hay más de un usuario accediendo a una máquina simultáneamente. La multitarea hace referencia a aquellas situaciones en las que un mismo usuario está realizando más de una tarea al mismo tiempo.

Sección 3.2

1. *Shell*: se comunica con el entorno de la máquina.

Administrador de archivos: coordina el uso de los dispositivos de almacenamiento masivo de la máquina.

Controladores de dispositivo: gestionan las comunicaciones con los dispositivos periféricos de la máquina.

Gestor de memoria: coordina el uso de la memoria principal de la máquina.

Planificador de tareas: coordina los procesos del sistema.

Despachador: controla la asignación de tiempo de procesador a los procesos.

2. La línea es vaga y la distinción depende a menudo de quién esté emitiendo el juicio. Hablando en términos aproximados, el software de utilidad realiza tareas básicas y universales, mientras que el software de aplicación lleva a cabo tareas que son específicas de la aplicación a la que la máquina se dedica.
3. La memoria virtual es el espacio imaginario de memoria cuya presencia aparente se crea mediante el procedimiento de intercambiar datos y programas entre la memoria principal y el almacenamiento masivo.

4. Cuando se enciende la máquina, el procesador comienza ejecutando el programa de arranque, que reside en memoria ROM. Este proceso de arranque hace que el procesador transfiera el sistema operativo desde el almacenamiento masivo al área volátil de la memoria principal. Una vez completada esta transferencia, el proceso de arranque hace que el procesador salte al punto de inicio del sistema operativo.

Sección 3.3

1. Un programa es un conjunto de instrucciones. Un proceso es la acción de seguir dichas instrucciones.
2. El procesador completa su ciclo de máquina actual, guarda el estado del proceso actual y configura su contador de programa con un valor predeterminado (que es la posición de la rutina de tratamiento de la interrupción). De este modo, la siguiente instrucción que se ejecute será la primera instrucción de la rutina de tratamiento de interrupciones.
3. Se les podrían dar prioridades más altas, para que el despachador les otorgara un trato preferente. Otra opción sería asignar a los procesos de mayor prioridad unas franjas temporales más largas.
4. Si cada proceso consumiera su franja temporal completa, la máquina podría proporcionar una franja completa a casi 20 procesos en un segundo. Si los procesos no consumen toda su franja temporal, este valor podría ser mucho más alto, pero entonces el tiempo requerido para realizar cada cambio de contexto podría ser más significativo (véase la Cuestión 5).
5. Un total de $\frac{3000}{3001}$ del tiempo de la máquina se invertiría en ejecutar procesos. Sin embargo, cuando un proceso solicita una actividad de E/S, su franja temporal se termina mientras la controladora responde a la solicitud. Por tanto, si cada proceso realizara una de esas solicitudes después de un solo microsegundo de su franja temporal, la eficiencia de la máquina caería a $\frac{1}{2}$. Es decir, la máquina invertiría tanto tiempo realizando cambios de contexto como el que invertiría en ejecutar procesos.

Sección 3.4

1. Este sistema garantiza que el recurso no sea usado por más de un proceso al mismo tiempo. Sin embargo, obliga a que el recurso sea asignado en forma estrictamente alternativa. Una vez que un proceso ha utilizado y liberado un recurso, debe esperar a que otro proceso lo utilice antes de poder acceder de nuevo a él. Esto incluso si el primer proceso necesita el recurso de forma inmediata y el otro proceso no va a necesitarlo durante algún tiempo.
2. Si dos vehículos entran por los extremos opuestos del túnel al mismo tiempo, no serán conscientes de la presencia del otro. El proceso de entrar y encender las señales luminosas es otro ejemplo de región crítica, o en este caso podríamos denominarlo proceso crítico. Con esta terminología, podríamos corregir el fallo diciendo que los vehículos situados en extremos opuestos del túnel podrían llegar a ejecutar el proceso crítico al mismo tiempo.

3.
 - a. Esto garantiza que el recurso no compartible no sea solicitado y asignado de manera parcial; es decir, a cada vehículo se le concede el uso del puente completo o no se le concede.
 - b. Esto quiere decir que la asignación del recurso no compartible puede ser cancelada.
 - c. Esto hace compartible el recurso no compartible, lo que elimina la competencia.
4. Una secuencia de flechas que forma un bucle cerrado en el grafo dirigido. Es precisamente basándose en esto que se han desarrollado técnicas que permiten a algunos sistemas operativos reconocer la existencia de un interbloqueo y tomar en consecuencia las acciones correctivas oportunas.

Sección 3.5

1. Los nombres y fechas se consideran inadecuados porque son elecciones bastante comunes y no resisten por tanto los intentos de los que se dedican a tratar de adivinar contraseñas. El uso de palabras completas también se considera inadecuado, porque un atacante podría escribir fácilmente un programa para ir probando las palabras contenidas en un diccionario. Además no se recomienda utilizar contraseñas que solo contengan caracteres ya que se construyen a partir de un conjunto de caracteres limitado.
2. Cuatro es el número de patrones de bits distintos que pueden formarse utilizando 2 bits. Si hicieran falta más niveles de privilegio, los diseñadores necesitarían al menos 3 bits para representar los distintos niveles, con lo cual probablemente elegirían usar un total de 8 niveles. De forma similar, la elección natural si queremos menos de 4 niveles de privilegio sería 2, que es el número de patrones que puede representarse con 1 bit.
3. El proceso podría alterar el propio sistema operativo, de modo que el despachador concediera todas las franjas temporales a dicho proceso.

Capítulo 4

Sección 4.1

1. Una red abierta es aquella cuyas especificaciones y protocolos son públicos, lo que permite a los distintos fabricantes producir productos compatibles.
2. Ambos permiten conectar dos buses para formar una red de bus de mayor tamaño. Sin embargo, un puente solo reenvía aquellos mensajes que están destinados al otro lado del puente, mientras que un conmutador tiene múltiples conexiones, cada una de las cuales puede actuar como un puente.
3. Un router es un dispositivo que dirige los mensajes para su transmisión entre las redes que forman una interred.
4. Por ejemplo, una empresa de venta por correo y sus clientes, un cajero automático y los clientes del banco o una farmacia y sus clientes.
5. Hay numerosos protocolos que regulan el flujo de tráfico en las ciudades, la comunicación verbal por vía telefónica y las normas de etiqueta.

6. La computación en cluster implica normalmente la utilización de múltiples computadoras dedicadas, con el fin de proporcionar una computación distribuida de alta disponibilidad o con equilibrado de carga. La computación en retícula tiene un acoplamiento mayor que la computación en cluster y puede implicar el uso de máquinas que se unen a la tarea de computación distribuida solo cuando no están haciendo otra cosa.

Sección 4.2

1. Los ISP de nivel 1 y nivel 2 proporcionan el "núcleo" de las comunicaciones por Internet, mientras que los ISP de acceso proporcionan a sus clientes el acceso a dicho núcleo.
2. El Sistema de nombres de dominio (DNS) es el conjunto de servidores de nombres en Internet que permite traducir las direcciones mnemónicas a direcciones IP (y que permite también realizar la traducción inversa).
3. La expresión 3.6.9 representa el patrón de tres bytes 000000110000 011000001001. El patrón de bits 0001010100011100 se representaría como 21.28 en notación decimal con puntos.
4. Podría haber varias respuestas a esta cuestión. Una sería que ambas notaciones van de lo más específico a lo más general. Las direcciones Internet con el formato mnemónico comienzan con el nombre de una máquina concreta y continúan hasta llegar al nombre del TLD. Las direcciones postales comienzan con el nombre de una persona y van describiendo regiones progresivamente mayores como la ciudad, la provincia y el país. Este orden está invertido en las direcciones IP, que comienzan por el patrón de bits que identifica el dominio.
5. Los servidores de nombres ayudan a traducir direcciones mnemónicas a direcciones IP. Los servidores de correo envían, reciben y almacenan mensajes de correo electrónico. Los servidores FTP proporcionan un servicio de transferencia de archivos.
6. SSH proporciona funciones de cifrado y autenticación.
7. Descargan al servidor inicial de la tarea de enviar mensajes individuales a cada cliente. La técnica P2P traslada esta responsabilidad a los propios clientes (*peers*), mientras que la multidifusión traslada esta tarea a los routers de Internet.
8. Entre los criterios que podríamos considerar se incluyen el coste, la portabilidad, hasta qué punto resulta práctico utilizar nuestra computadora como teléfono, la necesidad de mantener los teléfonos analógicos existentes, la necesidad de emplear servicios telefónicos de emergencia y la fiabilidad y las áreas de servicio de los diversos proveedores implicados.

Sección 4.3

1. Una dirección URL es básicamente la dirección de un documento en la World Wide Web. Un explorador es un programa que ayuda al usuario a acceder a información de hipertexto.
2. Un lenguaje de composición es un sistema para insertar información explicativa en un documento.

3. HTML es un lenguaje de composición concreto. XML es un estándar para el diseño de lenguajes de composición.
4.
 - a. `<html>` indica el principio de un documento HTML.
 - b. `<head>` indica el principio de la cabecera de un documento.
 - c. `</p>` indica el final de un párrafo.
 - d. `</a>` indica el final de un elemento que está vinculado a otro documento.
5. *Lado de cliente* y *lado de servidor* son términos utilizados para identificar si una actividad se realiza en la computadora del cliente o en la computadora del servidor.

Sección 4.4

1. La capa de enlace recibe el mensaje y se lo entrega a la capa de red. La capa de red determina la dirección en la que hay que reenviar el mensaje y devuelve el mensaje a la capa de enlace para que lo reenvíe. Las capas superiores no son requeridas para el encaminamiento de los mensajes, aunque los routers avanzados pueden emplear las capas de transporte o de aplicación para proporcionar servicios tales como un filtrado selectivo o una calidad de servicio por capas.
2. A diferencia de TCP, UDP es un protocolo no orientado a conexión que no confirma que el mensaje ha sido recibido en el destino.
3. La capa de transporte utiliza los números de puerto del protocolo de transporte para determinar qué unidad dentro de la capa de aplicación debe recibir un mensaje entrante.
4. Realmente no hay nada que lo impida. Un programador en cualquier máquina podría modificar el software que se ejecuta en dicha máquina con el fin de almacenar todos los mensajes que pasen a su través. Esta es la razón por la que los datos confidenciales deben cifrarse.

Sección 4.5

1. El *phishing* es una técnica para obtener información confidencial preguntando a los usuarios sus contraseñas, números de tarjeta de crédito, etc. por correo electrónico haciéndose pasar por una entidad legítima, como por ejemplo el banco del usuario o el departamento de informática del campus. Las computadoras no están protegidas frente al *phishing*; los usuarios deben confiar en su propio sentido común a la hora de decidir si hay que revelar datos confidenciales a otras personas sin la apropiada verificación.
2. La pasarela de una zona es un router que simplemente se limita a reenviar paquetes (partes de mensajes) a medida que estos pasan a su través. Por tanto, un cortafuegos situado en una pasarela no puede filtrar el tráfico según su contenido, sino simplemente según la información de direccionamiento.
3. El uso de contraseñas protege los datos (y por tanto también la información). El uso del cifrado protege a la información.
4. En el caso de un sistema de cifrado de clave pública, el conocer cómo están cifrados los mensajes no permite descifrarlos.

5. Los problemas son internacionales por su propia naturaleza y, por tanto, no están sujetos a las leyes de un único gobierno. Además, las soluciones legales simplemente proporcionan a aquellos que han sufrido algún tipo de daño la posibilidad de recurrir a los tribunales, en lugar de prevenir que esos daños lleguen a producirse.

Capítulo 5

Sección 5.1

1. Un proceso es la actividad de ejecutar un algoritmo. Un programa es una representación de un algoritmo.
2. En el capítulo de introducción hemos hablado de algoritmos para reproducir música, para hacer funcionar a las lavadoras, para construir modelos y para realizar trucos de magia, así como el algoritmo de Euclides. Muchos de los "algoritmos" que llevamos a cabo en nuestra vida cotidiana no son algoritmos porque no se ajustan a nuestra definición formal de algoritmo, como por ejemplo el algoritmo de división de enteros que se ha citado en el texto, o el algoritmo ejecutado por un reloj que continúa haciendo avanzar sus manecillas y dando las horas día tras día.
3. La definición informal falla al no requerir que los pasos estén ordenados y no sean ambiguos. Simplemente enuncia los requisitos de que los pasos sean ejecutables y tengan un determinado fin.
4. Hay que señalar dos aspectos. El primero es que las instrucciones definen un proceso que no termina. En realidad, sin embargo, el proceso terminará por alcanzar un estado en el que ya no habrá más monedas en el bolsillo. De hecho, este podría ser perfectamente el estado inicial. Llegados a este punto, el problema que surge es el de la ambigüedad. El algoritmo, tal como está representado, no nos dice qué es lo que hay que hacer en esta situación.

Sección 5.2

1. Un ejemplo sería la composición de la materia. A un cierto nivel, las moléculas podrían ser las primitivas, pero estas partículas realmente están compuestas por átomos, que a su vez están formados por electrones, protones y neutrones. Además, actualmente sabemos que incluso estas "primitivas" son compuestas.
2. Una vez que un procedimiento está construido correctamente, puede utilizarse como un bloque componente en estructuras de programa más grandes sin tener que considerar de nuevo la composición interna de dicho procedimiento.
3. $X \leftarrow$ el valor más grande;
 $Y \leftarrow$ el valor más pequeño;
while (Y distinto de cero) do
 ($\text{Resto} \leftarrow$ resto después de dividir X entre Y ;
 $X \leftarrow Y$;
 $Y \leftarrow \text{Resto}$);
 $\text{MCD} \leftarrow X$

4. Todos los colores de la luz pueden generarse combinando el rojo, el azul y el verde. Por ello, el tubo de rayos catódicos de una televisión está diseñado para generar estos tres colores básicos.

Sección 5.3

1. a. **if** ($n = 1$ o $n = 2$)
 then (la respuesta es la lista que contiene solo el propio valor n)
 else (Dividir n entre 3, obteniendo un cociente q y un resto r .
 if ($r = 0$)
 then (la respuesta es la lista que contiene q treses)
 if ($r = 1$)
 then (la respuesta es la lista que contiene $(q - 1)$ treses
 y dos doses;)
 if ($r = 2$)
 then (la respuesta es la lista que contiene q treses y
 un 2)
)
 b. El resultado sería la lista que contiene 667 treses.
 c. Probablemente haya experimentado con valores de entrada pequeños hasta empezar a visualizar un patrón.
2. a. Sí. *Sugerencia:* coloque la primera ficha en el centro para evitar el cuadrante que contiene el agujero al mismo tiempo que cubre un cuadrado de cada uno de los otros cuadrantes. Cada cuadrante representa entonces una versión más pequeña del problema original.
 b. El tablero con un solo agujero contiene $2^{2n} - 1$ cuadrados y cada ficha cubre exactamente tres cuadrados.
 c. Los apartados (a) y (b) de esta cuestión proporcionan un excelente ejemplo de cómo conocer la solución de un problema ayuda a resolver otro. Véase la cuarta fase de Polya.
3. Dice, "Esta es la respuesta correcta."
4. Intentar simplemente ensamblar las piezas sería una técnica abajo-arriba. Sin embargo, examinar la caja del puzzle para ver qué aspecto tiene la imagen añade una componente arriba-abajo a nuestro intento de solución.

Sección 5.4

1. Cambie la prueba en la sentencia **while** por la siguiente "el valor objetivo no es igual a la entrada actual y todavía quedan entradas por considerar".
2. $Z \leftarrow 0$;
 $X \leftarrow 1$;
 repeat ($Z \leftarrow Z + X$;
 $X \leftarrow X + 1$)
 until ($X = 6$)
3. Esto ha demostrado ser un problema en el lenguaje C. Cuando las palabras clave **do** y **while** están separadas por varias líneas, los lectores de un programa suelen dudar acerca de la interpretación apropiada de una cláusula **while**. En particular, el **while** situado al final de una sentencia **do** suele interpretarse como una sentencia **while**. Por tanto, la experiencia dice que

es mejor utilizar diferentes palabras clave para representar las estructuras de bucle de precomprobación y postcomprobación.

4. Carmen Alicia Alicia
Genaro Carmen Beatriz
Alicia Genaro Carmen
Beatriz Beatriz Genaro
5. Es una pérdida de tiempo insistir en colocar el pivote por encima de una entrada idéntica de la lista. Por ejemplo, haga el cambio propuesto y luego pruebe el nuevo programa con una lista en la que todas las entradas sean iguales.

6. **procedure** Ordenar (Lista)

```

N ← 1;
while (N sea menor que la longitud de Lista) do
  (J ← N + 1;
   while (J no sea mayor que la longitud de la Lista) do
     (if (la entrada de la posición J es menor que la entrada
        de la posición N)
       then (intercambiar las dos entradas);
      J ← J + 1)
  N ← N + 1)

```

7. La siguiente es una solución ineficiente. ¿Puede hacerla más eficiente?

procedure Ordenar (Lista)

```

N ← la longitud de la Lista;
while (N sea mayor que 1) do
  (J ← la longitud de la Lista;
   while (J sea mayor que 1) do
     (if (la entrada de la posición J es menor que la entrada
        de la posición J - 1)
       then (intercambiar las dos entradas);
      J ← J - 1)
  N ← N - 1)

```

Sección 5.5

1. El primer nombre considerado sería Hector, el siguiente sería Lara y el último Jose.
2. 8, 17
3. 1, 2, 3, 3, 2, 1
4. La condición de terminación es "N es mayor o igual que 3" (o "N no es menor que 3"). Esta es la condición para la que no se crea ninguna activación adicional.

Sección 5.6

1. Si la máquina puede ordenar 100 nombres en un segundo, puede realizar $\frac{1}{4}$ (10.000 - 100) comparaciones en un segundo. Esto significa que cada comparación tarda aproximadamente 0,0004 segundos. En consecuencia, orde-

nar 1000 nombres [lo que requiere una media de $\frac{1}{4}$ (1.000.000 – 1000) comparaciones] requerirá unos 100 segundos o $1\frac{2}{3}$ minutos.

2. La búsqueda binaria pertenece a la clase $\Theta(\lg n)$, la búsqueda secuencial a la clase $\Theta(n)$ y la ordenación por inserción pertenece a la clase $\Theta(n^2)$.
3. La clase $\Theta(\lg n)$ es la más eficiente, seguida de $\Theta(n)$, $\Theta(n^2)$ y $\Theta(n^3)$.
4. No. La respuesta no es correcta aunque pueda parecerlo. Lo cierto es que dos de las tres cartas son iguales por ambos lados. Por tanto, la probabilidad de elegir una de esas cartas es dos tercios.
5. No. Si el dividendo es menor que el divisor, como por ejemplo en $\frac{3}{7}$, la respuesta dada es 1, aunque debería ser 0.
6. No. Si el valor de X es cero y el valor de Y es distinto de cero, la respuesta proporcionada no será correcta.
7. Cada vez que se lleva a cabo la prueba de terminación, el enunciado "Suma = $1 + 2 + \dots + K$ y K menor o igual que N " es verdadero. Combinando esto con la condición de terminación " K mayor o igual que N " se obtiene la conclusión deseada "Suma = $1 + 2 + \dots + N$ ". Puesto que K se inicializa con el valor cero y se incrementa en una unidad cada vez que se pasa por el bucle, su valor terminará por alcanzar el de N .
8. Lamentablemente, no. Una serie de problemas están fuera del control del diseño hardware y software, como por ejemplo un mal funcionamiento mecánico o los problemas eléctricos pueden afectar a los cálculos.

Capítulo 6

Sección 6.1

1. Un programa de tercera generación es independiente de la máquina en el sentido de que sus pasos no se enuncian en términos de los atributos de la máquina, como por ejemplo los registros y las direcciones de las celdas de memoria. Por otro lado, es dependiente de la máquina en el sentido de que seguirán produciéndose errores de truncamiento y de desbordamiento aritmético.
2. La principal diferencia es que un ensamblador traduce cada instrucción del programa fuente en una única instrucción del lenguaje máquina, mientras que un compilador suele generar muchas instrucciones en lenguaje máquina para obtener el equivalente a una única sentencia del programa fuente.
3. El paradigma declarativo se basa en desarrollar una descripción del problema que hay que resolver. El paradigma funcional fuerza al programador a describir el problema en términos de las soluciones a otros problemas más pequeños. El paradigma orientado a objetos pone el énfasis en describir los componentes que podemos encontrar en el entorno del problema.
4. Los lenguajes de tercera generación permiten expresar el programa más en términos del entorno del problema y menos en función de las características de la computadora, como hacían los lenguajes de las anteriores generaciones.

Sección 6.2

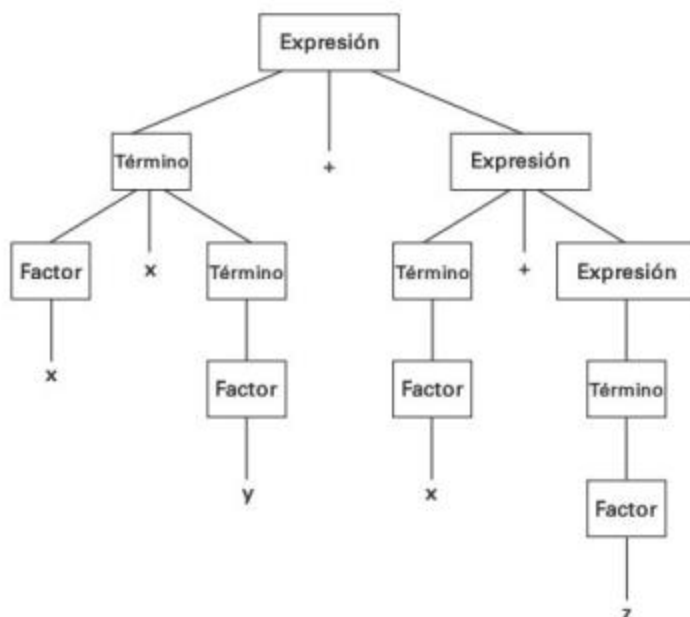
1. La utilización de una constante descriptiva puede mejorar la accesibilidad del programa.
2. Una sentencia declarativa describe la terminología; una sentencia imperativa describe pasos dentro de un algoritmo.
3. Entero, real, carácter y booleano.
4. Las estructuras if-then-else y el bucle while son muy comunes.
5. Todos los componentes de una matriz tienen el mismo tipo.

Sección 6.3

1. El ámbito de una variable es el rango del programa en el que la variable es accesible.
2. Una función es un procedimiento que devuelve un valor asociado al nombre de la función.
3. Porque eso es lo que son. Las operaciones de E/S son en realidad llamadas a rutinas que se encuentran dentro del sistema operativo de la máquina.
4. Un parámetro formal es un identificador dentro de un procedimiento. Sirve como contenedor para el valor, el parámetro real, que se pasa al procedimiento en el momento de invocar este.
5. Un procedimiento está diseñado para realizar una acción, mientras que una función está diseñada para generar un valor. Por tanto, el programa es más legible si el nombre de un procedimiento refleja la acción que realiza y el nombre de una función refleja el valor que devuelve.

Sección 6.4

1. *Análisis léxico*: el proceso de identificar los lexemas.
Análisis sintáctico: el proceso de reconocer la estructura gramatical del programa.
Generación de código: el proceso de generar las instrucciones que definen el programa objeto.
2. Una tabla de símbolos es el registro de la información que el analizador sintáctico ha obtenido a partir de las sentencias declarativas del programa.
3. En los diagramas sintácticos, los términos que aparecen dentro de los óvalos son terminales. Los términos que requieren una descripción ulterior se incluyen en rectángulos y se denominan "no terminales".
4. Véase la figura de la página siguiente.
5. Las cadenas que ajustan a la estructura Chachacha están compuestas por una más de las siguientes subcadenas:
adelante atrás cha cha cha
atrás adelante cha cha cha
balanceo derecha cha cha cha
balanceo izquierda cha cha cha



Sección 6.5

1. Una clase es la descripción de un objeto.
2. Una sería probablemente la clase `ClaseMeteorito` a partir de la cual se construirían los distintos meteoritos. Dentro de la clase `ClaseLaser` podríamos encontrar una variable de instancia denominada `Direccion` que indicaría la dirección hacia la que apunta el láser. Esta variable podría ser utilizada por los métodos `disparar`, `moverDcha` y `moverIzq`.
3. La clase `Empleado` puede contener características relativas al nombre, dirección, antigüedad, etc. del empleado. La clase `EmpleadoTiempoCompleto` podría contener características relativas a planes de pensiones. La clase `EmpleadoTiempoParcial` puede contener características relativas al número de horas de trabajo por semana, el salario por hora, etc.
4. Un constructor es un método especial dentro de una clase que se ejecuta en el momento de crear una instancia de esa clase.
5. Algunos elementos de una clase se designan como privados para evitar que otras unidades de programa obtengan un acceso directo a dichos elementos. Si un elemento es privado, entonces las repercusiones de modificar dicho elemento deberían estar restringidas al interior de la clase.

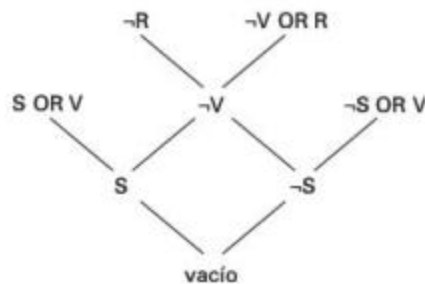
Sección 6.6

1. La lista incluiría técnicas para iniciar la ejecución de procesos concurrentes y técnicas para implementar la comunicación entre procesos.
2. Una sería asignar la responsabilidad a los procesos y otra sería asignar la responsabilidad a los datos. Esta última presenta la ventaja de concentrar la tarea en un único punto del programa.

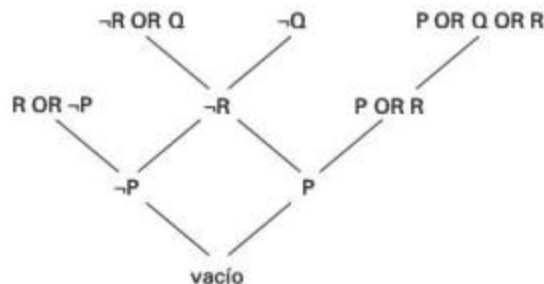
- Entre ellas se incluyen la predicción meteorológica, el control del tráfico aéreo, la simulación de sistemas complejos (desde reacciones nucleares a control del tráfico de peatones), las redes de computadoras y el mantenimiento de bases de datos.

Sección 6.7

- R, T y V. Por ejemplo, podemos demostrar que R es una consecuencia añadiendo su negación al conjunto y demostrando que el principio de resolución puede llevarnos al enunciado vacío, como se muestra a continuación:



- No. El conjunto es incoherente, porque el principio de resolución puede conducir al enunciado vacío, como aquí se muestra:



- madre(X, Y) :- progenitor(X, Y), hembra(X).
padre(X, Y) :- progenitor(X, Y), varon(X).
- Prolog concluirá que carolina es su propia hermana. Para resolver este problema, la regla tiene que incluir el hecho de que X no puede ser igual a Y, lo que en Prolog se escribe $X \neq Y$. Así, una versión mejorada de la regla sería

hermano (X, Y) :- $X \neq Y$, progenitor(Z, X), progenitor(Z, Y).

lo que quiere decir que X es hermano de Y si X e Y no son iguales y tienen un progenitor en común. La siguiente versión, por el contrario, nos dice que X e Y son hermanos solo si tienen en común ambos progenitores:

hermano (X, Y) :- $X \neq Y$, $Z \neq W$
progenitor (Z, X), progenitor (Z, Y),
progenitor (W, X), progenitor (W, Y).

Capítulo 7

Sección 7.1

1. Una secuencia larga de sentencias de asignación no es tan compleja en el contexto del diseño de un programa como unas cuantas sentencias más complicadas.
2. ¿Qué tal el número de errores que se encuentren después de un cierto periodo de uso fijo? Un problema con esta solución es que ese valor no puede medirse por adelantado.
3. Lo importante aquí es pensar en cómo pueden medirse las propiedades del software. Una solución para tratar de estimar el número de errores en un segmento de software es colocar intencionadamente algunos errores en el software en el momento de diseñarlo. Luego, después de que el software haya sido supuestamente depurado, comprobamos cuántos de los errores originales conocidos siguen estando presentes. Por ejemplo, si colocamos de forma intencionada siete errores en el software y nos encontramos con que cinco de ellos han sido eliminados después de la depuración, podemos hacer la conjetura de que solo se han eliminado $\frac{5}{7}$ del número total de errores que el software contiene.
4. Entre las posibles respuestas se incluyen el descubrimiento de métricas, el desarrollo de componentes prefabricados, el desarrollo de herramientas CASE o la tendencia hacia la adopción de estándares. Otro, del que trataremos posteriormente en la Sección 7.5, sería el desarrollo de sistemas de notación y modelado como UML.

Sección 7.2

1. Los pequeños esfuerzos realizados durante el desarrollo pueden reportar enormes dividendos durante el mantenimiento.
2. La fase de análisis de requisitos se concentra en lo que el sistema propuesto debe realizar. La fase de diseño se centra en cómo consigue sus objetivos ese sistema. La fase de implementación se concentra en la construcción real del sistema. La fase de pruebas se concentra en asegurarse de que el sistema hace lo que se pretende que haga.
3. Una especificación de requisitos del software es un acuerdo escrito entre un cliente y una empresa de ingeniería de software, en el que se indican los requisitos y las especificaciones del software que hay que desarrollar.

Sección 7.3

1. La técnica tradicional en cascada dicta que las fases de análisis de requisitos, diseño, implementación y de pruebas se realicen de forma lineal. Los modelos más recientes permiten un enfoque más relajado de prueba y error.
2. ¿Qué tal el modelo incremental, el modelo iterativo y XP?
3. El prototipado evolutivo tradicional se realiza dentro de la organización que está desarrollando el software, mientras que el desarrollo de código fuente

abierto no está restringido a una organización. En el caso del desarrollo de código fuente abierto, la persona que controla el desarrollo no determina necesariamente qué mejoras hay que realizar, mientras que en el caso de prototipado evolutivo tradicional es la persona que gestiona el desarrollo del software quien asigna personal a tareas específicas de mejora.

4. Esta es una cuestión que merece la pena considerar con cuidado. Si usted fuera un administrador en una empresa de desarrollo software, ¿le permitirían adoptar la metodología de código fuente abierto para el desarrollo del software que su empresa vaya a comercializar?

Sección 7.4

1. Los capítulos de una novela dependen de los anteriores capítulos, mientras que las secciones de una enciclopedia son en buena medida independientes. Por tanto, una novela tiene un mayor grado de acoplamiento entre sus capítulos que una enciclopedia entre sus secciones. Sin embargo, las secciones de una enciclopedia probablemente tengan un mayor grado de cohesión que los capítulos de una novela.
2. El marcador acumulado sería un ejemplo de acoplamiento de datos. Otros "acoplamientos" que puedan existir incluirían la fatiga, el estado de ánimo, el conocimiento obtenido acerca de la estrategia del oponente y quizá la autoconfianza. En muchos deportes, la cohesión de las unidades se incrementa terminando la acción y reiniciando la siguiente unidad desde el principio. Por ejemplo, en el béisbol cada periodo comienza sin ningún jugador en las bases aún cuando el equipo puede haber finalizado el periodo anterior con ellos. En otros casos, el marcador de cada unidad se controla por separado, como sucede en el tenis donde cada set se pierde o se gana independientemente de lo que haya sucedido en los otros sets.
3. Esta es una cuestión difícil. Desde un cierto punto de vista, podríamos comenzar incluyendo todo en único módulo. Esto daría como resultado muy poca cohesión y ningún acoplamiento. Si entonces empezamos a dividir este único módulo en otros más pequeños, el resultado será un incremento en el grado de acoplamiento. Podemos por tanto concluir que un aumento de la cohesión tiende a incrementar el acoplamiento.

Por el contrario, suponga que el problema que tenemos entre manos se puede dividir de manera natural en tres módulos de muy alta cohesión, a los que denominaremos A, B y C. Si nuestro diseño original no observara esta división natural (por ejemplo, si agrupáramos la mitad de las tareas de A con la mitad de las tareas de B, etc.), cabría esperar que la cohesión fuera baja y que el acoplamiento fuera alto. En este caso, rediseñar el sistema aislando A, B y C en módulos separados permitiría muy probablemente reducir el acoplamiento entre módulos e incrementar al mismo tiempo la cohesión intermodular.

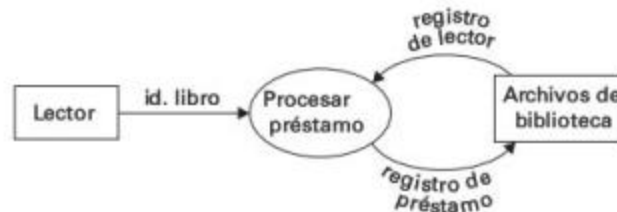
4. El acoplamiento es el grado de enlazamiento entre módulos. La cohesión es el grado de conexión dentro de un módulo. El ocultamiento de información es la restricción que se impone a la compartición de información.
5. Deberíamos añadir una flecha para indicar que Control Juego debe informar a Actualizar Marcador de quién ha ganado y otra flecha en la otra

dirección que indique que `ActualizarMarcador` informará del estado actual (como por ejemplo "fin del set" o "fin del juego") cuando devuelva el control a `Control Juego`.

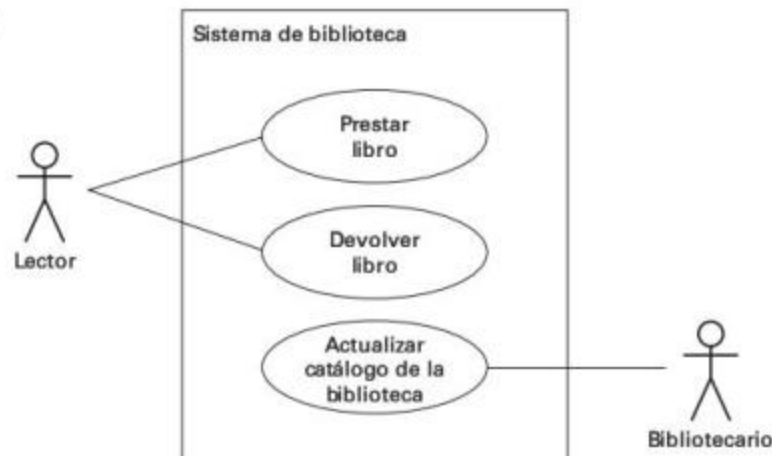
6. Borre todas las flechas horizontales de la Figura 7.5 excepto la primera y la última. Es decir, el juez debe evaluar el servicio del `JugadorA` y enviar directamente el mensaje `ActualizarMarcador` al `Marcador`. (Por supuesto, esto ignora la posibilidad de que exista un segundo servicio. ¿Cómo modificaría el diseño del programa para permitir las faltas dobles?)
7. Un programador tradicional escribe sus programas utilizando sentencias como las que hemos presentado en el Capítulo 6. Un ensamblador de componentes construye los programas enlazando bloques prefabricados denominados componentes.
8. Hay muchas respuestas a esta cuestión. Una combinación sería que el calendario fijara automáticamente una alarma en un reloj para notificar al usuario una cita próxima. Además, la aplicación de calendario podría utilizar los componentes de una aplicación de mapas para proporcionar las indicaciones acerca de cómo llegar a la dirección donde tendrá lugar la cita.

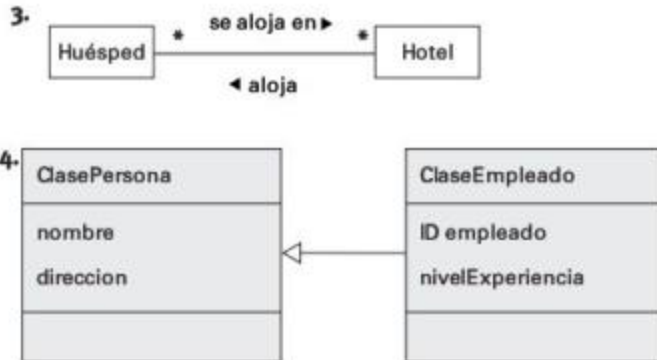
Sección 7.5

1. Asegúrese de que su diagrama trate con el flujo de datos (no con el movimiento de los libros). El siguiente diagrama indica que las identificaciones de los libros (suministradas por los lectores) y los registros de lectores (extraídos de los archivos de la biblioteca) se combinan para formar los registros de préstamos que se almacenan en los archivos de la biblioteca.



2.





5. Simplemente dibuje un rectángulo alrededor de la figura y añada una etiqueta "sd" en la esquina superior izquierda, como en la Figura 7.13.
6. Los patrones de diseño proporcionan soluciones estandarizadas y bien desarrolladas para la implementación de temas software recurrentes.

Sección 7.6

1. El grupo para el aseguramiento de la calidad del software (SQA) se encarga de controlar e imponer los sistemas de control de calidad adoptados por la organización.
2. Los seres humanos tenemos la tendencia a no anotar los pasos (decisiones, acciones, etc.) que damos durante un proyecto. (También existen problemas relativos a conflictos de personalidades, celos y choques de egos.)
3. Mantenimiento de registros y revisión.
4. El propósito de probar el software es detectar los errores. En un cierto sentido, por tanto, una prueba que no revele ningún error se puede considerar fallida.
5. Una sería considerar el grado de ramificación en los módulos. Por ejemplo, un módulo procedimental que contenga numerosos bucles y sentencias `if-then-el` se probablemente será más proclive a errores que un módulo que tenga una estructura lógica simple.
6. El análisis de valores límite sugeriría que probemos el software con una lista de 100 entradas, así como con una lista que no contenga ninguna entrada. También podríamos realizar una prueba con una lista que esté en el orden correcto.

Sección 7.7

1. La documentación toma la forma de documentación de usuario, documentación del sistema y documentación técnica. Puede aparecer en los manuales de acompañamiento, dentro del programa fuente (en forma de comentarios y código bien escrito), en mensajes interactivos que el propio programa escriba en un terminal, en diccionarios de datos y en forma de documentos de diseño, tales como diagramas de estructura, diagramas de clases, diagramas de flujo de datos y diagramas de entidad-relación.

2. Tanto en las fases de desarrollo como de modificación. Lo importante es que las modificaciones deben documentarse tan exhaustivamente como el programa original. (También es cierto que el software se documenta durante la fase de uso. Por ejemplo, un usuario del sistema puede descubrir problemas que, en lugar de ser corregidos, simplemente se comentan en futuras ediciones del manual de usuario del sistema. Además, a menudo se publican libros de carácter práctico después de que el software haya estado utilizándose durante un amplio periodo.)
3. Las diferentes personas tendrán diferentes opiniones sobre esta cuestión. Algunos argumentarán que el programa es el verdadero objetivo de todo el proyecto y que es, por tanto, naturalmente lo más importante. Otros argumentarán que un programa no vale nada si no está documentado, porque si no puede comprenderse, no se podrá ni utilizar ni modificar. Además, con una buena documentación, la tarea de crear el programa puede acometerse de nuevo "fácilmente".

Sección 7.8

1. a. ¿Qué tal la capacidad de ajustar la inclinación de una pantalla o la forma de un ratón? En los teléfonos inteligentes, ¿qué tal el uso de pantallas táctiles en lugar de un ratón, o la técnica de inclinar el teléfono con el fin de proporcionar indicaciones de entrada al sistema?
- b. ¿Qué tal la disposición de una ventana en la pantalla, incluyendo el diseño de las barras de herramientas, de las barras de desplazamiento y los menús desplegables? En un teléfono inteligente, ¿acaso no es el inclinar la cámara para apuntar a los elementos de interés algo bastante similar a la forma en la que los seres humanos pensamos?
2. a. Sería poco práctico e incómodo utilizar un ratón (o incluso un lápiz óptico) en un teléfono inteligente. Además, el tamaño reducido de la pantalla requiere que los elementos no esenciales de la visualización se restrinjan a un espacio limitado. Por esta razón, las barras de desplazamiento suelen omitirse, y si existen, se muestran en forma de líneas finas.
- b. Un toque deslizante en la pantalla es un gesto natural adaptado a la forma en la que pensamos. Podemos mover papeles u otros elementos deslizando sobre una mesa. Algunas personas podrían sostener que esto es bastante más natural que el uso de barras de desplazamiento en una computadora de sobremesa. Aunque las barras de desplazamiento se mueven, en efecto, de la manera esperada, el área que se está desplazando se mueve en la dirección opuesta. Para un usuario que nunca haya usado una computadora, este comportamiento puede parecer anti-intuitivo.
3. Podríamos responder que la diferencia radica en "el papel de las características humanas". Otra buena respuesta sería que el diseño de interfaces se centra en las características externas de un sistema software, más que en las características internas.
4. Las tres que se comentan en el texto son la formación de hábitos, lo estrecho que es nuestro campo de atención y las limitadas capacidades de multi-

procesamiento. ¿Puede imaginarse algunas otras? ¿Qué tal la tendencia a hacer suposiciones?

Sección 7.9

1. El aviso de copyright enuncia quién es el propietario de ese trabajo e identifica al personal autorizado a usar ese trabajo. Todos los trabajos, incluyendo la especificación de requisitos, los documentos de diseño, el código fuente y el producto final, suelen requerir una inversión considerable. Cualquier persona o empresa debe tomar las medidas necesarias para garantizar que se preserven sus derechos de propiedad intelectual y que esa propiedad intelectual no sea utilizada por personas no deseadas.
2. Las leyes de la propiedad intelectual y de patentes benefician a la sociedad porque animan a los creadores de nuevos productos a ponerlos a disposición del público. Sin ese tipo de protección, las empresas dudarían en realizar grandes inversiones en el desarrollo de nuevos productos.
3. Una declaración de renuncia de responsabilidad no protege a una empresa frente a la negligencia.

Capítulo 8

Sección 8.1

1. Lista: un listado de los miembros de un equipo deportivo.
Pila: la pila de bandejas en una cafetería.
Cola: la cola de clientes en una cafetería.
Árbol: el organigrama de muchos gobiernos.
2. Las pilas y colas pueden considerarse como tipos especiales de listas. En el caso de una lista general, las entradas se pueden insertar y eliminar de cualquier posición. En el caso de una pila, las entradas se pueden insertar y eliminar solo por el principio de la pila. En el caso de una cola, las entradas solo pueden insertarse por el final y solo pueden eliminarse por el principio.
3. Las letras de la pila de arriba abajo serían E, D, B y A. Si extrajéramos una letra de la pila, sería la letra E.
4. Las letras de la cola desde el principio al final serían B, C, D y E. Si extrajéramos una letra de la cola, sería la letra B.
5. Los nodos hoja (o terminales) son D y C. B tiene que ser el nodo raíz porque todos los demás nodos tienen padre.

Sección 8.2

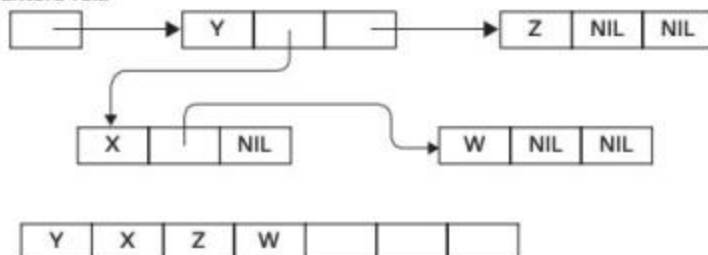
1. Los datos dentro de la memoria principal de una computadora están realmente almacenados en celdas de memoria individualmente direccionables. Las estructuras como matrices, listas y árboles se simulan, para hacer los datos más accesibles a los usuarios de esos datos.
2. Si fuéramos a escribir un programa para jugar a las damas, la estructura de datos que representa el tablero realmente sería una estructura estática por-

que el tamaño del tablero no cambia durante el juego. Sin embargo, si fuéramos a escribir un programa para jugar al dominó, la estructura de datos que representa el patrón de fichas construido sobre la mesa probablemente fuera una estructura dinámica, porque este patrón varía de tamaño y no puede ser predeterminado.

3. Un listín telefónico es básicamente una colección de punteros (números de teléfono) a personas. Las pistas dejadas en la escena de un crimen son punteros (quizá cifrados) que apuntan al perpetrador.

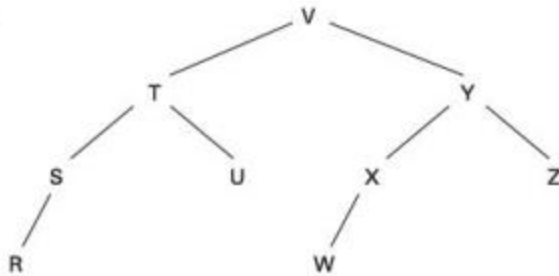
Sección 8.3

1. 5 3 7 4 2 8 1 9 6
2. Si R es el número de filas de la matriz, la fórmula es $R(J - 1) + (I - 1)$.
3. $(c \times i) + j$
4. El puntero inicial contiene el valor NIL.
5. Último $\leftarrow$ el último nombre que imprimir
Terminado $\leftarrow$ false
Puntero Actual $\leftarrow$ el puntero inicial;
while (Puntero Actual not NIL and Terminado = false) do
 (imprimir la entrada a la que apunta Puntero Actual,
 if (nombre que se acaba de imprimir = Último)
 then (Terminado $\leftarrow$ true)
 Puntero Actual $\leftarrow$ el valor en la celda de puntero
 correspondiente a la entrada a la que apunta
 Puntero Actual)
6. El puntero de pila apunta a la celda que se encuentra inmediatamente debajo de la pila.
7. Represente la pila como una matriz unidimensional y el puntero de pila como una variable de tipo entero. A continuación, utilice este puntero de pila para mantener un registro de la posición de la cima de la pila dentro de la matriz, en lugar de la dirección exacta de memoria.
8. Tanto la condición de cola llena como la condición de cola vacía están indicadas por el hecho de que los punteros inicial y final son iguales. Por tanto, hace falta información adicional para distinguir entre esas dos condiciones.
9. Puntero raíz

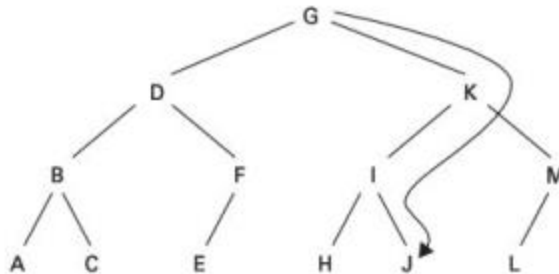


Sección 8.4

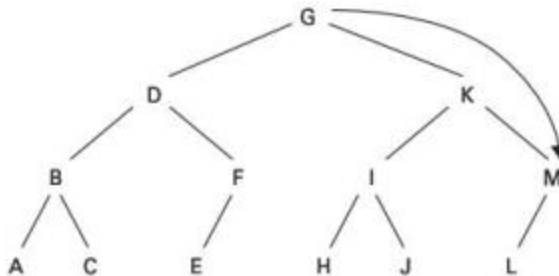
1.



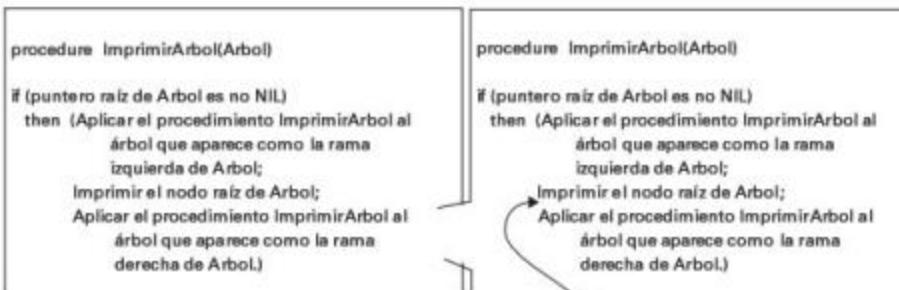
2. Al buscar J:



Al buscar P:



3.



Aquí, en el momento
de imprimir K

4. En cada nodo, cada puntero de hijo podría utilizarse para representar una letra distinta del alfabeto. Una palabra podría representarse mediante una ruta descendente por el árbol, junto con la secuencia de punteros que representa la ortografía de la palabra. Un nodo podría marcarse de forma especial si representara el final de una palabra correctamente escrita.

Sección 8.5

1. Un tipo es una plantilla; una instancia de ese tipo es una entidad real construida a partir de esa plantilla. Como analogía, un perro es un tipo de animal, mientras que Lassie y Rex son instancias de dicho tipo.
2. Un tipo de datos definido por el usuario es una descripción de una organización de datos, mientras que un tipo abstracto de datos incluye operaciones para manipular los datos.
3. Un aspecto que hay que resaltar es que podemos elegir entre implementar la lista como una lista contigua o como una lista enlazada. La elección que hagamos afectará a la estructura de los procedimientos que definamos para insertar nuevas entradas, para borrar entradas ya existentes y para encontrar las entradas que busquemos. Sin embargo, esta elección no debe ser visible para el usuario de una instancia de ese tipo abstracto de datos.
4. El tipo abstracto de datos contendrá al menos una descripción de una estructura de datos para almacenar el saldo de la cuenta y procedimientos para efectuar un depósito y para extraer fondos mediante un cheque.

Sección 8.6

1. Tanto los tipos abstractos de datos como las clases son plantillas para construir instancias de un tipo. Sin embargo, las clases son más generales, en el sentido de que están asociadas con el mecanismo de herencia y de que pueden describir también simples conjuntos de procedimientos.
2. Una clase es una plantilla a partir de la cual se construyen objetos.
3. La clase puede contener una cola circular junto con procedimientos para añadir entradas, eliminar entradas, comprobar si la cola está llena y comprobar si la cola está vacía.

Sección 8.7

1. a. A5 b. A5 c. CA
2. D50F, 2EFF, 5FFE
3. 2EA0, 2FB0, 2101, 20B5, D50E, E50F, 5EE1, 5FF1, BF14, B008, C000
4. Al recorrer una lista enlazada en la que cada entrada está compuesta por dos celdas de memoria (una celda de datos seguida de un puntero a la siguiente entrada), una instrucción de la forma DR0S podría utilizarse para extraer el dato, mientras que DR1S podría utilizarse para extraer el puntero a la siguiente entrada. Si se empleara la forma DRTS, entonces podría ajustarse la celda de memoria exacta a la que se está haciendo referencia modificando el valor contenido en el registro T.

Capítulo 9

Sección 9.1

1. El departamento de compras estaría interesado en los registros de inventario para realizar pedidos de materias primas, mientras que el departamento de contabilidad necesitaría la información para llevar las cuentas de la empresa.
2. Un modelo de base de datos proporciona una perspectiva organizativa de una base de datos que es más compatible con las aplicaciones que la organización real. Por tanto, definir un modelo de base de datos es el primer paso para poder utilizar una base de datos como herramienta abstracta.
3. El software de aplicación traduce las solicitudes de los usuarios de la terminología de la aplicación a otra terminología compatible con el modelo de base de datos soportado por el sistema de gestión de base de datos. El DBMS a su vez convierte dichas solicitudes en acciones sobre la base de datos real.

Sección 9.2

1. **a.** Gervasio Salas **b.** Charo Garrido **c.** S26Z

2. Una solución es

```
TEMP ← SELECT from PUESTO
        where Dept = "PERSONAL"
LIST ← PROJECT TitPuesto from TEMP
```

En algunos sistemas esto da como resultado una lista con títulos de puesto repetidos, dependiendo de cuántas veces aparezca cada puesto dentro del departamento de personal. Es decir, nuestra lista podría contener numerosas apariciones del título Secretaria. Es más común, sin embargo, diseñar la operación PROJECT de modo que se eliminen las tuplas duplicadas de la relación resultante.

3. Una solución es

```
TEMP1 ← JOIN PUESTO and ASIGNACION
        where PUESTO.IdPuesto = ASIGNACION.IdPuesto
TEMP2 ← SELECT from TEMP1
        where FechaFin = "*"
TEMP3 ← JOIN EMPLEADO and TEMP2
        where EMPLEADO.IdEmpl = TEMP2.IdEmpl
RESULT ← PROJECT Nombre, Dept from TEMP3
```

4. select TitPuesto

```
    from PUESTO
    where Dept = "PERSONAL"
```

```
select EMPLEADO.Nombre, PUESTO.Dept
from PUESTO, ASIGNACION, and EMPLEADO
where (PUESTO.IdPuesto = ASIGNACION.IdPuesto) and
      ( ASIGNACION.IdEmpl = EMPLEADO.IdEmpl )
      and ( ASIGNACION.FechaFin = "*" )
```


5. El propio modelo no proporciona independencia de datos. Esa es una propiedad del sistema de gestión de la base de datos. La independencia de datos se consigue proporcionando al sistema de gestión de los datos la posibilidad de presentar una organización relacional coherente al software de aplicación, aún cuando la organización real de los datos pueda cambiar.
6. Mediante atributos comunes. Por ejemplo, la relación EMPLEADO de esta sección está enlazada con la relación ASIGNACION mediante el atributo IdEmpl y la relación ASIGNACION está enlazada con la relación PUESTO mediante el atributo IdPuesto. Los atributos como estos utilizados para conectar la relación se denominan en ocasiones atributos de conexión.

Sección 9.3

1. Puede haber métodos para asignar y extraer FechaInicio, así como FechaFin. También podría proporcionarse otro método para informar del tiempo total de servicio.
2. Un objeto persistente es un objeto que se almacena de manera indefinida.
3. Un enfoque sería establecer un objeto para cada tipo de producto del inventario. Cada uno de estos objetos podría mantener el inventario total de su producto correspondiente, el coste del producto y enlaces con las órdenes pendientes relativas a dicho producto.
4. Como se indica al principio de esta sección, las bases de datos orientadas a objetos parecen manejar los tipos de datos compuestos más fácilmente que las bases de datos relacionales. Además, el hecho de que los objetos puedan contener métodos que adopten papeles activos a la hora de responder cuestiones promete dar a las bases de datos orientadas a objetos una ventaja con respecto a las bases de datos relacionales, cuyas relaciones simplemente almacenan los datos.

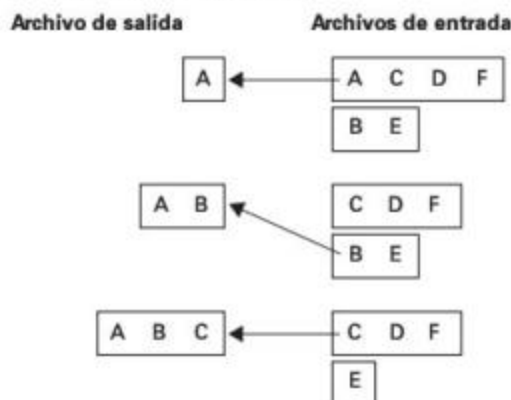
Sección 9.4

1. Una vez que una transacción ha alcanzado su punto de confirmación, el sistema de gestión de la base de datos acepta la responsabilidad de que la transacción completa se realice en la base de datos física. Una transacción que no haya alcanzado su punto de confirmación no dispondrá de tal garantía. Si surgen problemas puede tener que volver a realizarse la transacción.
2. Una solución sería impedir el entremezclado de transacciones durante un instante, para que todas las transacciones actuales puedan completarse. Esto establecería un punto en el que las futuras anulaciones en cascada se detendrían.
3. Se obtendría un saldo de 100 euros si las transacciones se ejecutaran una a continuación de otra. Se obtendría un saldo de 200 euros si se ejecutara la primera transacción después de que la segunda transacción hubiera extraído el saldo original y antes de que almacenara su nuevo saldo. Se obtendría un saldo de 300 euros si la segunda transacción se ejecutara después de que la primera extrajera el saldo original y antes de que la primera almacenara su nuevo saldo.

4. a. Si ninguna otra transacción tiene acceso exclusivo, se concederá acceso compartido.
b. Si otra transacción ya tiene algún tipo de acceso, el sistema de gestión de la base de datos normalmente hará que la nueva transacción espere, o bien podría anular las otras transacciones y proporcionar acceso a la nueva transacción.
5. Se producirá un interbloqueo si dos transacciones obtuvieran acceso exclusivo a diferentes elementos y luego solicitaran acceso al elemento para el que la otra tiene acceso exclusivo.
6. El interbloqueo anterior podría eliminarse anulando una de las transacciones (utilizando el registro) y proporcionando a la otra transacción acceso al elemento de datos que la primera transacción tenía bloqueado.

Sección 9.5

1. Los pasos iniciales serían los siguientes:



2. La idea es dividir en primer lugar el archivo que hay que almacenar en muchos archivos separados, cada uno de los cuales contendrá un registro. A continuación, agrupamos los archivos de un solo registro en parejas y aplicamos el algoritmo de combinación a cada pareja. Esto nos da como resultado la mitad de los archivos, cada uno de ellos con dos registros. Además, cada uno de estos archivos de dos registros estará ordenado. Podemos agruparlos en parejas y aplicar de nuevo el algoritmo de combinación. Con ello, tendremos un conjunto menor de archivos, cada uno de ellos de un tamaño mayor, y además esos archivos estarán ordenados. Continuando de esta forma, terminaremos por obtener un único archivo, que estará formado por todos los registros originales, pero ordenados. (Si en cualquier paso del proceso nos encontráramos con un número impar de archivos, basta con dejar uno de los archivos aparte y emparejarlo con otro de los archivos de mayor tamaño en la siguiente etapa.)
3. Si el archivo está almacenado en cinta o en CD, lo más probable es que su organización física sea secuencial. Sin embargo, si el archivo está almacenado en disco magnético, lo más probable es que esté disperso entre varios sectores del disco y la naturaleza secuencial del archivo será una propiedad

conceptual apoyada en un sistema de punteros o en algún tipo de lista, en la que se anotarán los sectores en los que está almacenado el archivo.

4. En primer lugar, localice el valor clave dentro del índice del archivo. A partir de ahí, obtenga la ubicación del registro objetivo. Después, extraiga el registro almacenado en dicha ubicación.
5. Un algoritmo hash mal elegido provoca un agrupamiento mayor de lo normal y, por tanto, un mayor número de desbordamientos. Dado que el desbordamiento de cada fragmento del almacenamiento masivo está organizado como una lista enlazada, las búsquedas en los registros de desbordamiento son, esencialmente, búsquedas secuenciales.
6. Las asignaciones de fragmentos son las siguientes:
 a. 0 b. 0 c. 3 d. 0 e. 3
 f. 3 g. 3 h. 3 i. 3 j. 0

Por tanto, todos los registros hash se asignan a los fragmentos 0 y 3, dejando los fragmentos 1, 2, 4 y 5 vacíos. El problema es que el número de fragmentos que se está utilizando (6) y los valores de clave tienen el 3 como factor común. (Trate de volver a asignar estos valores clave utilizando siete fragmentos y compruebe la mejora que se obtiene.)

7. Lo importante aquí es que estamos aplicando esencialmente un algoritmo hash para dividir a las personas del grupo en 365 categorías distintas. El algoritmo de hash es, por supuesto, el cálculo del cumpleaños de cada uno. Lo sorprendente es que solo hacen falta 23 personas antes de que la probabilidad de que dos de los cumpleaños coincidan sea superior al 50 por ciento. En términos de un archivo hash, esto indica que a la hora de distribuir los registros entre 365 fragmentos disponibles de almacenamiento masivo, el fenómeno del agrupamiento empezará a hacerse notar después de introducir tan solo 23 registros.

Sección 9.6

1. El buscar patrones en un conjunto dinámico de datos es problemático.
2. Descripción de clases: identificación de las características de los subscriptores a una determinada revista.
 Discriminación de clases: identificación de las características que permiten distinguir entre los subscriptores de dos revistas.
 Análisis de agrupamientos: identificación de las revistas que tienden a atraer a subscriptores similares.
 Análisis de asociaciones: identificación de las relaciones entre subscriptores a diversas revistas y entre los diferentes hábitos de compra.
 Análisis de excepciones: identificación de los subscriptores a una revista que no se ajustan al perfil de los subscriptores normales.
 Análisis de patrones secuenciales: identificación de tendencias en la subscripción a revistas.
3. El cubo de datos puede permitir ver los datos de ventas como ventas mensuales, ventas por región geográfica, ventas por clase de producto, etc.
4. Las consultas tradicionales a bases de datos extraen hechos almacenados en la base de datos. La minería de datos busca patrones entre esos hechos.

Sección 9.7

1. Lo importante aquí es comparar su respuesta a esta cuestión con la de la siguiente. Las dos suscitan esencialmente la misma cuestión pero en diferentes contextos.
2. Véase el problema anterior.
3. Podría recibir anuncios publicitarios que de otro modo no habría recibido, pero también podría verse asediado por ofertas indeseadas o convertirse en objetivo de algún grupo de criminales.
4. Lo importante aquí es que una prensa libre puede alertar a los ciudadanos acerca de los abusos o de los abusos potenciales, haciendo así que entre en juego la opinión pública. En la mayoría de los casos citados en el texto, fue la prensa libre la que inició las acciones correctivas al alertar al público acerca de lo que estaba sucediendo.

Capítulo 10

Sección 10.1

1. El procesamiento de imágenes se ocupa del análisis de imágenes bidimensionales, los gráficos 2D tratan con la conversión de formas bidimensionales en imágenes y los gráficos 3D se ocupan de la conversión de escenas tridimensionales en imágenes.
2. La fotografía tradicional produce imágenes de escenas reales, mientras que los gráficos 3D generan imágenes de escenas virtuales.
3. El primero es “construir” la escena virtual. El segundo es capturar la imagen.

Sección 10.2

1. Los pasos son el modelado (construcción de la escena), la reproducción o generación (de una imagen) y la visualización (mostrar la imagen en la pantalla).
2. La ventana de imagen es la parte del plano de proyección que constituye la imagen.
3. Un buffer de imagen es un área de memoria que contiene una versión codificada de una imagen.

Sección 10.3

1. Es un rombo (un cuadrado deformado).
2. Un modelo procedimental es un segmento de programa que controla la construcción de un objeto.
3. La lista podría incluir el terreno cubierto de hierba, un paseo empedrado, los árboles, las nubes, el sol y los actores. Lo importante aquí es darse cuenta de cuál es el ámbito de un grafo de escena (puede contener una gran cantidad de detalles).

4. Representar todos los objetos mediante mallas poligonales proporciona un mecanismo uniforme para el proceso de generación. (En la mayoría de los casos, la generación se implementa como una tarea de generación de parches planos en lugar de generar objetos.)
5. La aplicación de texturas es un medio de asociar una imagen bidimensional con la superficie de un objeto.

Sección 10.4

1. La luz especular es aquella que se refleja "directamente" en una superficie. La luz difusa es luz que se "dispersa" a partir de una superficie. La luz ambiente es aquella que no tiene una fuente definida.
2. El recorte es el proceso de descartar aquellos objetos (y partes de objetos) que no caen dentro del volumen de visualización.
3. Suponga que en mitad de un parche debe aparecer un resalte luminoso. Dicho resalte es provocado por la orientación específica de la superficie en dicho punto del parche. Puesto que el sombreado de Gouraud solo toma en consideración las orientaciones de la superficie a lo largo de las aristas del parche, no será capaz de mostrar ese resalte. Sin embargo, como el sombreado de Phong trata de determinar las orientaciones de la superficie en el interior del parche, sí que podría detectar ese resalte luminoso.
4. La cadena de generación proporciona una técnica estandarizada para generar, lo que permite obtener sistemas de generación más eficientes. En particular, la cadena de generación puede implementarse en firmware, lo que quiere decir que el proceso de generación puede realizarse más rápidamente que si implementara dicha tarea mediante software tradicional.
5. El propósito de esta cuestión es hacer pensar al lector sobre las diferencias entre los modelos de iluminación local y global, más que esperar que proporcione una respuesta concreta. Entre las soluciones potenciales que el lector podría proponer se incluyen el colocar copias adecuadamente modificadas de los objetos que hay que reflejar detrás del espejo y considerar el espejo transparente, o tratar de manejar las imágenes que se reflejan en el espejo como si fueran sombras de objetos.

Sección 10.5

1. Solo nos interesan los rayos que terminan por alcanzar la ventana de imagen. Si comenzamos en la fuente luminosa, no sabríamos qué rayos seguir.
2. El trazado de rayos distribuido trata de evitar la apariiencia inherentemente brillante generada por el trazado de rayos tradicional. Para ello, lo que hace es trazar múltiples rayos.
3. La radiosidad requiere mucho tiempo de cálculo y no consigue reflejar bien los efectos especulares.
4. Tanto el trazado de rayos como la radiosidad implementan un modelo de iluminación global y ambos hacen un uso muy intenso de la computación. Sin embargo, el trazado de rayos tiende a producir superficies con apa-

riencia brillante, mientras que la radiosidad genera superficies de apariencia apagada.

Sección 10.6

1. Esta pregunta no tiene una respuesta exacta. Si las imágenes perduran durante 200 milisegundos y proyectáramos cinco fotogramas por segundo, cada fotograma se habría desvanecido de nuestros circuitos de percepción en el momento de proyectar el siguiente fotograma. Esto haría, probablemente, que percibiéramos una imagen parpadeante, que resultaría incómoda de contemplar durante un periodo de tiempo prolongado, aunque hay que reconocer que seguiría percibiéndose el efecto de animación. (De hecho, incluso velocidades más bajas permiten generar animaciones burdas.) Observe que la velocidad de cinco fotogramas por segundo está muy por debajo del estándar en el cine, que es de 24 fotogramas por segundo.
2. Un *storyboard* es un “esbozo pictórico” de la secuencia de animación deseada.
3. El proceso de interpolación de fotogramas consiste en crear fotogramas que rellenen los huecos entre fotogramas clave.
4. La Dinámica es la rama de la Mecánica que analiza los movimientos que son consecuencia de la aplicación de fuerzas. La Cinemática es la rama de la Mecánica que analiza el movimiento sin tener en cuenta las fuerzas que lo provocan.

Capítulo 11

Sección 11.1

1. Las presentadas en el capítulo incluyen las acciones reflejas, las acciones basadas en el conocimiento del mundo real, las acciones de búsqueda de objetivos, el aprendizaje y la percepción.
2. Nuestro propósito no es dar una respuesta concluyente a esta cuestión, sino usarla para mostrar lo delicado que es realmente el argumento acerca de la existencia de inteligencia.
3. Aunque la mayoría de nosotros diría probablemente que no, sin embargo, si fuera un ser humano el que dispensara los mismos productos en una atmósfera similar probablemente todos diríamos que existe una consciencia, aunque no fuéramos capaces de explicar la diferencia.
4. No existe una respuesta correcta o incorrecta a esta cuestión. La mayoría de los expertos estarían de acuerdo en que la máquina parece al menos ser inteligente.
5. No existe una respuesta correcta o incorrecta a esta cuestión. Es preciso señalar que los bots para mensajería, que son programas diseñados para emular el comportamiento de una persona en un sistema de mensajería, tienen dificultades a la hora de mantener una conversación con sentido incluso durante breves periodos de tiempo. Esos bots para programas de mensajería y salas de chat son fácilmente identificables como máquinas.

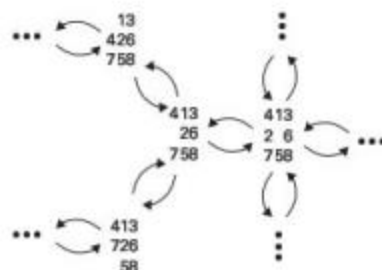
Sección 11.2

1. En el caso del control remoto, el sistema solo necesita reenviar la imagen, mientras que para usar la imagen con el fin de decidir las maniobras que hay que realizar, el robot debe ser capaz de "comprender" el significado de la imagen.
2. Las posibles interpretaciones para una sección del dibujo no se corresponden con ninguna de las interpretaciones de otra sección. Para incluir este tipo de comprensión dentro un programa, podrían aislarse las interpretaciones permitidas para las diversas uniones de líneas y luego escribir un programa que tratara de encontrar un conjunto de interpretaciones compatibles (una para cada unión). De hecho, si nos paramos a pensar en ello, esto es lo que probablemente nuestros propios sentidos hacen a la hora de tratar de evaluar el dibujo. Probablemente haya detectado usted mismo cómo sus ojos saltaban adelante y atrás entre los dos extremos del dibujo, a medida que sus sentidos intentaban encajar las posibles interpretaciones. (Si le interesa este tema, pruebe a leer acerca de las investigaciones de personas como D. A. Huffman, M. B. Clowes y D. Waltz.)
3. Hay cuatro bloques en la pila, pero solo tres de ellos son visibles. Lo importante en esta cuestión es comprender que este concepto aparentemente simple requiere una cantidad significativa de "inteligencia".
4. ¿Verdad que es interesante? Este tipo de sutiles distinciones en el significado presentan problemas muy complejos en el campo de la comprensión del lenguaje natural.
5. No se sabe si han colocado en la plaza un nuevo banco en el que sentarse o si han abierto una nueva entidad bancaria.
6. El proceso de análisis sintáctico produce estructuras idénticas, pero el análisis semántico reconoce que la cláusula preposicional de la primera frase nos dice dónde se ha construido la valla, mientras que la cláusula de la segunda frase nos dice cuándo se construyó.
7. Son hermanos.

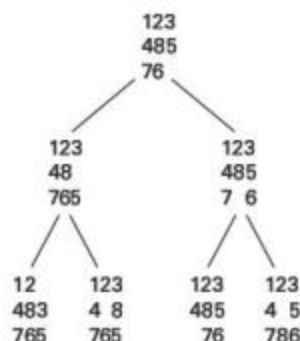
Sección 11.3

1. Los sistemas de producción proporcionan una solución uniforme a un amplio rango de problemas. Es decir, aunque aparentemente sean distintos en su formulación original, todos los problemas reformulados en términos de sistemas de producción se reducen al problema de encontrar una ruta a través de un grafo de estados.

2.



- El árbol tiene una profundidad de cuatro movimientos. La parte superior tendrá el aspecto siguiente:

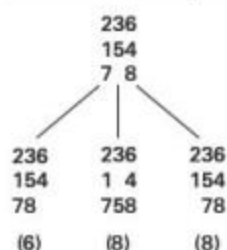


- La tarea requiere demasiado papel y demasiado tiempo.
- Nuestro sistema heurístico para la resolución del puzzle de ocho piezas está basado en un análisis de la situación inmediata, al igual que en el caso del escalador. Esta actitud miope es la que permitía a nuestro algoritmo avanzar inicialmente por una ruta incorrecta en el ejemplo de esta sección, al igual que el escalador puede verse en apuros si siempre decide su camino basándose únicamente en el terreno local. (Esta analogía hace que a los sistemas heurísticos basados en información local o inmediata a menudo se les denomine sistemas de escalada de montañas.)
- El sistema hace girar las piezas 5, 6 y 8 en el sentido de las agujas del reloj o en el sentido contrario a las agujas del reloj hasta alcanzar el estado objetivo.
- El problema aquí es que nuestro esquema heurístico ignora el valor de mantener el hueco adyacente a las piezas que están descolocadas. Si el hueco está rodeado por piezas que se encuentran en su posición correcta, será necesario mover algunas de esas piezas antes de poder mover las otras que todavía no se encuentran en su lugar adecuado. Por tanto, es erróneo considerar que son correctas todas las piezas que rodean al hueco. Para solventar este problema, podemos primero observar que una pieza que se encuentre en su posición correcta pero bloqueando el hueco a otras piezas en posición incorrecta tendrá que ser desplazada de su posición correcta y luego vuelta a colocar. Así, cada pieza correctamente colocada pero que se encuentre en la ruta entre el hueco y la pieza incorrectamente colocada más próxima aportará al menos dos movimientos a la solución restante. Por tanto, podemos modificar nuestro cálculo del coste estimado del modo siguiente:

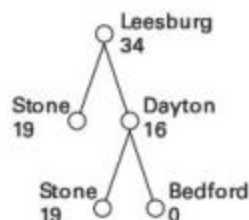
En primer lugar, calculamos el coste estimado como antes. Sin embargo, si el hueco está totalmente aislado de las piezas incorrectamente colocadas, determinaremos la ruta más corta entre el hueco y una pieza mal colocada, multiplicaremos el número de piezas de esta ruta por dos y sumaremos el valor resultante al coste previamente estimado.

Con este sistema, los nodos hoja de la Figura 11.11 tendrán un coste estimado de 6, 6 y 4 (de izquierda a derecha), con lo que inicialmente se comenzará a explorar la rama correcta.

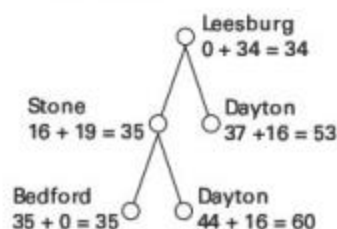
Nuestro nuevo sistema no está libre de errores. Por ejemplo, considere la siguiente configuración. La solución consiste en deslizar la pieza 5 hacia abajo, girar en el sentido de las agujas del reloj las dos filas superiores hasta colocar correctamente esas piezas, mover de nuevo la pieza 5 hacia arriba y finalmente mover la pieza 8 a su posición correcta. Sin embargo, nuestro nuevo sistema heurístico nos diría que comencemos moviendo la pieza 8 porque el estado obtenido por ese movimiento inicial tiene un coste estimado de solo 6, comparado con las otras opciones que tienen un coste de 8.



8. La solución encontrada por el algoritmo de búsqueda de elección del mejor es la ruta que va desde Leesburg a Dayton y luego a Bedford. Esta ruta no es la más corta.



9. La solución encontrada es la ruta que va desde Leesburg a Stone, y luego a Bedford. Esta ruta es la más corta.



Sección 11.4

1. El conocimiento del mundo real es la información acerca del entorno que un ser humano utiliza para comprender y razonar. Desarrollar métodos para representar, almacenar y recuperar esa información es uno de los principales objetivos de la inteligencia artificial.
2. Utiliza la suposición del mundo cerrado.
3. El problema del marco es el problema de actualizar correctamente el almacén de conocimientos de una máquina a medida que se van produciendo

sucesos. La tarea es complicada por el hecho de que muchos sucesos tienen consecuencias indirectas.

4. Imitación, entrenamiento supervisado y refuerzo. El refuerzo no implica una intervención humana directa.
5. Las técnicas tradicionales construyen un único sistema de cómputo. Las técnicas evolutivas implican la construcción de múltiples generaciones de sistemas de prueba, a partir de las cuales puede descubrirse un sistema "correcto".

Sección 11.5

1. Todos los patrones generan una salida igual a 0, excepto el patrón 1, 0, que genera una salida igual a 1.
2. Asigne un peso de 1 a cada entrada y asigne a la neurona un valor de umbral de 1,5.
3. Uno de los problemas principales identificado en el texto es que el proceso de entrenamiento puede oscilar, repitiendo el mismo ajuste una y otra vez.
4. La red derivará hacia la configuración en la que la neurona central está excitada y todas las restantes están inhibidas.

Sección 11.6

1. En lugar de desarrollar un plan de acción completo, el enfoque reactivo consiste en esperar y tomar decisiones a medida que van surgiendo las opciones.
2. Lo importante aquí es pensar en lo amplio que es el campo de la robótica. Abarca todo el campo de la inteligencia artificial, así como numerosos temas de otros campos. El objetivo es desarrollar máquinas verdaderamente autónomas que puedan moverse e interactuar inteligentemente con su entorno.
3. Control interno y estructura física.

Sección 11.7

1. No existe una respuesta correcta o incorrecta.
2. No existe una respuesta correcta o incorrecta.
3. No existe una respuesta correcta o incorrecta.

Capítulo 12

Sección 12.1

1. Por ejemplo, las operaciones booleanas AND, OR y XOR. De hecho, nosotros hemos utilizado tablas en el Capítulo 1 al presentar estas funciones.
2. El cálculo del pago correspondiente a un préstamo, el área de un círculo o el consumo de gasolina de un automóvil.
3. A tales funciones los matemáticos las denominan funciones trascendentales. Como ejemplos podemos citar las funciones logarítmicas y trigonomé-

tricas. Estos ejemplos concretos pueden calcularse, pero no por medios algebraicos. Por ejemplo, las funciones trigonométricas pueden calcularse dibujando el correspondiente triángulo, midiendo sus lados y solo después recurriendo a la operación algebraica de división.

- Un ejemplo sería el problema de dividir un ángulo en tres partes iguales. Es decir, no fueron capaces de construir un ángulo que tuviera un tercio del tamaño de otro ángulo dado. Lo importante aquí es que el sistema computacional de los griegos basado en una regla y un compás es otro ejemplo de sistema que presenta limitaciones.

Sección 12.2

- El resultado es el siguiente diagrama:



-

Estado actual	Contenido de la celda actual	Valor que hay que escribir	Dirección del movimiento	Estado al que pasar
START	*	*	izquierda	ESTADO 1
ESTADO 1	0	0	izquierda	ESTADO 2
ESTADO 1	1	0	izquierda	ESTADO 2
ESTADO 1	*	0	izquierda	ESTADO 2
ESTADO 2	0	*	derecha	ESTADO 3
ESTADO 2	1	*	derecha	ESTADO 3
ESTADO 2	*	*	derecha	ESTADO 3
ESTADO 3	0	0	derecha	HALT
ESTADO 3	1	0	derecha	HALT

-

Estado actual	Contenido de la celda actual	Valor que hay que escribir	Dirección del movimiento	Estado al que pasar
START	*	*	izquierda	SUBTRACT
SUBTRACT	0	1	izquierda	BORROW
SUBTRACT	1	0	izquierda	NO BORROW
BORROW	0	1	izquierda	BORROW
BORROW	1	0	izquierda	NO BORROW
BORROW	*	*	derecha	ZERO
NO BORROW	0	0	izquierda	NO BORROW
NO BORROW	1	1	izquierda	NO BORROW
NO BORROW	*	*	derecha	RETURN
ZERO	0	0	derecha	ZERO
ZERO	1	0	derecha	ZERO
ZERO	*	*	sin movimiento	HALT
RETURN	0	0	derecha	RETURN
RETURN	1	1	derecha	RETURN
RETURN	*	*	sin movimiento	HALT

4. Lo importante aquí es que se supone que el concepto de máquina de Turing captura el significado del verbo "computar". Es decir, en cualquier momento en que se produzca una situación en la que esté teniendo lugar una computación deberían estar presentes los componentes y actividades de una máquina de Turing. Por ejemplo, una persona que esté calculando los impuestos que debe pagar está realizando un cierto grado de computación. La máquina de computación es la persona y la cinta está representada por el papel en el que se anotan los valores.
5. La máquina descrita por la siguiente tabla se detiene si se la inicia con una entrada par, pero nunca se detiene si se inicia con una entrada impar.

Estado actual	Contenido de la celda actual	Valor que hay que escribir	Dirección del movimiento	Estado al que pasar
START	*	*	izquierda	ESTADO 1
ESTADO 1	0	0	derecha	HALT
ESTADO 1	1	1	sin movimiento	ESTADO 1
ESTADO 1	*	*	sin movimiento.	ESTADO 1

Sección 12.3

1. `clear AUX;`
`incr AUX;`
`while X not 0 do;`
 `clear X;`
 `clear AUX;`
`end;`
`while AUX not 0 do;`
 `incr X;`
 `clear AUX;`
`end;`
2. `while X not 0 do;`
 `decr X;`
`end;`
3. `copy X to AUX;`
`while AUX not 0 do;`
 S1
 `clear AUX;`
`end;`
`copy X to AUX;`
`invert AUX; (Véase la Cuestión 1)`
`while AUX not 0 do;`
 S2
 `clear AUX;`
`end;`
`while X not 0 do;`
 `clear AUX;`
 `clear X;`
`end;`

4. Si suponemos que X hace referencia a la celda de memoria en la dirección 40 y que cada segmento de programa empieza en la posición 00, tendremos la siguiente tabla de conversión:

	Dirección	Contenido
clear X;	00	20
	01	00
	02	30
	03	40
	Dirección	Contenido
incr X;	00	11
	01	40
	02	20
	03	01
	04	50
	05	01
	06	30
	07	40
	Dirección	Contenido
decr X;	00	20
	01	00
	02	23
	03	00
	04	11
	05	40
	06	22
	07	01
	08	B1
	09	10
	0A	40
	0B	03
	0C	50
	0D	02
	0E	B1
	0F	06
	Dirección	Contenido
while X not 0 do;	00	20
	01	00
	02	11
	03	40
	04	B1
end;	05	WZ
	.	.
	.	.
	.	.
	WX	B0
	WY	00

5. Al igual que sucede en una máquina real se podrían manejar los números negativos mediante un sistema de codificación. Por ejemplo, el bit situado más a la derecha de cada cadena puede utilizarse como signo, empleándose los bits restantes para representar el módulo del valor.
6. La función es una multiplicación por 2.

Sección 12.4

1. Sí. De hecho, este programa se detiene independientemente del valor inicial de sus variables y por tanto debe detenerse si iniciamos sus variables con la representación codificada del programa.
2. El programa solo se detiene si el valor inicial de X termina en 1. Dado que la representación ASCII de un símbolo de punto y coma es 00111011, la versión codificada del programa debe terminar con un 1. Por tanto, el programa es auto-terminante.
3. Lo importante aquí es que la lógica es la misma que en nuestro argumento relativo a que el problema de la detención no tiene una solución algorítmica. Si el pintor pinta su propia casa, entonces no puede hacerlo y viceversa.

Sección 12.5

1. Solo podríamos concluir que el problema tiene una complejidad de orden $\Theta(2^n)$. Si pudiéramos demostrar que el "mejor algoritmo" para resolver el problema pertenece a $\Theta(2^n)$, podríamos concluir que el problema pertenece a $\Theta(2^n)$.
2. No. Como regla general, el algoritmo de orden $\Theta(n^2)$ será mejor que $\Theta(2^n)$, pero para valores de entrada pequeños un algoritmo exponencial suele ser mejor que un algoritmo polinómico. De hecho, es cierto que en ocasiones se prefieren los algoritmos exponenciales a los polinómicos, cuando la aplicación solo utiliza valores de entrada pequeños.
3. Lo importante es que el número de subcomités crece exponencialmente, así que a partir de este punto, la tarea de enumerar todas las posibilidades se hace muy laboriosa.
4. Dentro de la clase de los problemas polinómicos se encuentra el problema de ordenación que puede resolverse mediante algoritmos polinómicos como el de ordenación por inserción.

Dentro de la clase de problemas no polinómicos tendríamos la tarea de enumerar todos los subcomités que podrían formarse a partir de un determinado comité.

Cualquier problema polinómico es un problema NP. El problema del viajante es un ejemplo de problema NP que no se ha podido demostrar que sea un problema polinómico.

5. No. Nuestro uso del término *complejidad* hace referencia al tiempo requerido para ejecutar un algoritmo, no a lo difícil que pueda ser entender dicho algoritmo.

Sección 12.6

1. $211 \times 313 = 66043$
2. El mensaje 101 es la representación binaria de 5. $5^e = 5^5 = 15625$. $15625 \pmod{91} = 64$, que es 1000000 en notación binaria. Por tanto, 1000000 es la versión cifrada del mensaje.
3. El mensaje 10 es la representación binaria de 2. $2^d = 2^{29} = 536870912$. $536870912 \pmod{91} = 32$, que es 100000 en notación binaria. Por tanto, 100000 es la versión descifrada del mensaje.
4. $n = p \times q = 7 \times 19 = 133$. Para determinar d necesitamos un valor entero positivo k tal que $k(p-1)(q-1) + 1 = k(6 \times 18) + 1 = 108k + 1$ sea divisible por $e = 5$. Los valores $k = 1$ y $k = 2$ no cumplen esta condición, pero $k = 3$ produce $108k + 1 = 325$, que es divisible por 5. El cociente 65 es el valor de d .

Índice

- :- (símbolo Prolog), 339
- * (asterisco), 301
- /* (comentario), 306
- ** (exponenciación), 301
- //(comentario), 306
- + (concatenación), 301
- : = (operador de asignación), 301
- /(barra inclinada), 301
- (resta), 301
- 2D, gráficos, 494
- 3D, animación, 525
- 3D, gráficos, 495-498
 - animación, 524-528
 - modelado, 499-508
 - paradigma, 497
 - renderización, 508-520
- 3D, televisión, 507
- 3G, red de telefonía, 188
- 4G, red de telefonía, 188

- A*, algoritmo, 558
- Ábaco, 4
- Abstracción, 12-13, 27, 365, 404
- Acceso directo a memoria (DMA), 114-115
- Acceso telefónico, 181
- Access, sistema de base de datos de Microsoft, 454
- ACM (*Association for Computing Machinery*), 352, 379
- Acoplamiento de control, 364
- Acoplamiento intermodular, 363-364
- Actividades del lado del cliente, 197-199
- Actividades del lado del servidor, 197-199
- Actores, 372

- Ada, lenguaje, 271, 301, 332, 641
- Adaptador gráfico, 518
- Adaptador telefónico analógico, 187
- Adapter, patrón adaptador, 377
- ADD, instrucciones, 97
- Adleman, Leonard, 212
- Administrador, 156
 - de archivos, 141-142, 154
 - de sistemas, 135
 - de tareas, 151
 - de ventanas, 140
- Agentes inteligentes, 534-536
- Agregado, tipo, 298-299
- Agrupación, 478
- Aiken, Howard, 7
- Alexander, Christopher, 378
- Alfa, pruebas, 382
- Algoritmo de división, 2
- Algoritmo del pintor, 515
- Algoritmos
 - A*, 558
 - búsqueda binaria, 254-261, 266
 - búsqueda secuencial, 242-244
 - ciencia de los, 11-12
 - complejidad de, 611-621
 - concepto, 224-227
 - definición formal, 225-226
 - determinista, 621
 - diseño, 230
 - eficiencia de los, 262-272, 611
 - estructuras iterativas, 242-253
 - genéticos, 565-566
 - naturaleza abstracta, 226-227
 - no determinista, 618, 621
 - ordenación por combinación, 613-615
 - ordenación por inserción, 248-252, 264-266

- papel de los, 2-4
- proceso de descubrimiento, 235-242
- representación, 228-235
- RSA, 621
- verificación de, 267-272
- Aliasing, 513
- Almacén de datos, 480
- Almacenamiento
 - de árboles binarios, 417-420
 - de bits, 29
 - de enteros, 56-62
 - de fracciones, 63-68
 - de listas, 411-412, 423-428
 - de pilas y colas, 414-417
- ALVINN (*Autonomous Land Vehicle in a Neural Net*), 563, 571-572
- Amazon, 447
- Ambiente, luz, 510
- Ámbito de una variable, 308
- Análisis contextual, 544
- Análisis de agrupamiento, 481
- Análisis de algoritmos, 264-267
- Análisis de asociaciones, 481
- Análisis de excepciones, 481
- Análisis de imágenes, 541
- Análisis de requisitos, 354
- Análisis de valores límite, 382
- Análisis semántico, 543
- Análisis sintáctico, 543
- Analista de sistemas, 357
- Analista de software, 357
- Analizador léxico, 315, 323
- Analizador sintáctico, 315
- Ancho de banda, 117
- AND, 24, 25, 26, 107-108
- Anfitriones, 180
- Ángulo de incidencia, 509

- Animación, 524–529
- Anisotropía, superficie, 510
- ANSI (*American National Standards Institute*), 42, 44, 288
- Antivirus, software, 211–212
- Anulación en cascada, 468
- Anular, 468
- Aplicación, capa de, 201, 202, 204
- Aplicación de protuberancias, 518
- Aplicación de texturas, 504
- Aplicación, software de, 137, 138, 449–450
- Apple Computer, 9
- Aprendizaje, 562–565
- Árbol sintáctico, 317–318, 319
- Árboles, 402–403
 - binarios, 403, 417–420, 426–428
 - de búsqueda, 551–553
- Archivos, 40
 - almacenamiento y extracción de, 40–41
 - de texto, 44, 471–472
 - estructuras de, 471–480
 - hash, 476–479
 - indexados, 474–475
 - secuenciales, 471–474
 - y bases de datos, 447
- Aristóteles, 18
- Aritmético/lógicas, instrucciones, 106–111
- ARM (*Advanced Risk Machines*), 93
- ARM holdings, 93, 129
- Arquitectura
 - alternativas, 118–120
 - de componentes, 368
 - de computadoras, 88–91
 - de Internet, 179–181
 - sistema operativo, 137–146
 - von Neumann, 89, 92, 115
- Arrays, 298, 299, 400, 407–409
 - almacenamiento, 407–411
 - tipos agregados, 298–299, 400, 410–411
- ASCII (*American Standard Code for Information Interchange*), 42–43
- Aseguramiento de la calidad, 379–383
- Aserciones, 270
- Asignación de recursos, 150–155
- Asociaciones, 373–374
- ASP (*Active Server Pages*), 199
- AT&T, 272
- Atanasoff, John, 8
- Atanasoff-Berry, máquina de, 8
- Atributos, 452
- Audio, compresión de, 73
- Auditoría, software de, 157
- Autenticación, 214, 476
- Auto-referencia, 606
- Autoridades de certificación, 214
- Auto-terminante, 606, 607
- Avars, 527
- Axioma, 269
- Babbage, Charles, 5–6, 7
- Banda ancha, 117
- Bardeen, John, 8
- Base de la pila, 401
- Base diez, sistema en, 50
- Base dos, sistema en, 50
- Base, caso, 261
- Bases de datos, 445–492
 - capas conceptuales, 449
 - definición, 446
 - diseño relacional, 452–455
 - distribuidas, 449, 450
 - esquemas, 448
 - fundamentos, 446–451
 - impacto social de la tecnología de, 483–485
 - mantenimiento de la integridad, 466–470
 - orientadas a objetos, 463–466
 - relacionales, 452–463
 - y archivos, 447
- BD (disco Blu-ray), 39
- Berners-Lee, Tim, 9, 191
- Berry, Clifford, 8
- Beta, pruebas, 382
- Bezier, curvas de, 500–501
- Bezier, superficies, 502
- Biestables, 25–29
- Bifurcación (*forking*), 153
- Binario, árbol, 403, 417–420, 426–428
- Bioinformática, 482
- BIOS (*Basic Input/Output System*), 145
- Bit de signo, 56
- Bit más significativo, 31
- Bit menos significativo, 32
- Bits, 24
 - de paridad, 75–77
- por segundo (bps), 74, 116
- representación de la información mediante, 42–50
- Bloqueo, 468–470
- Bloqueo compartido, 469
- Blu-ray, discos, 39
- BOINC (*Berkeley's Open Infrastructure for Network Computing*), 178
- Boole, George, 24
- Boolean, tipo de datos, 297
- Booleana, expresión, 317
- Bourne, shell, 140
- Brattain, Walter, 8
- Brillo, componente, 46
- Bucle, 244–248
 - controlado a la salida, 248
 - controlado a la entrada, 247
- Buffer, 41, 402
 - z, 515
- Bus serie universal (USB), 112, 114, 116
- Bus, topología, 89, 169, 173
- Búsqueda, árboles de, 551–553
- Búsqueda binaria, algoritmo, 254–261, 266
- Búsqueda con prioridad para los mejores, 553
- Búsqueda en anchura, 553
- Búsqueda en profundidad, 553
- Búsqueda, proceso, 254–261
- Búsqueda secuencial, algoritmo, 242–244
- Byron, Augusta Ada (Ada Lovelace), 5, 7, 641
- Byte, 31–32
- Byte de comprobación, 76–77
- Bytecode, 321
- C#, lenguaje, 293, 294, 301, 303, 304, 305, 321, 642
- C, lenguaje, 294, 301, 303, 304, 309, 314, 641
- C, shell, 140
- C++, lenguaje, 293–294, 301, 303, 304, 305, 367–368, 642
- C++, *Standard Template Library*, 368
- Caballo de Troya, 208
- Cabecera de procedimiento, 307–308
- Cabezales de lectura/escritura, 35–36

- Caché, memoria, 90
- CAD (*Computer-Aided Design*), 47
- Cadena de generación, 512–515, 518–520
- Caja negra, 367
- CALEA (*Communications Assistance for Law Enforcement Act*), 216
- Cámara digital, 494
- Cambio de contexto, 147
- Camel casing, 233
- Campo clave, 41
- Campes, 41, 299
- Captura de movimiento, 527–528
- Cargador de arranque, 144
- Carnivore, 216
- Carpeta, 141
- CASE, herramientas, 352–353
- Case, instrucción, 304
- CASE. Véase Ingeniería del software asistida por computadora (CASE)
- Caso peor, análisis del, 264–267
- Caso promedio, análisis del, 264
- Casos de uso, diagrama, 372
- Cauce segmentado, 118
- CD-DA (*Compact disk-digital audio*), 38, 78
- Celda de memoria, 31–32
- Centro de proyección, 497
- CERN, 191
- CERT (*Computer Emergency Response Team*), 208
- Certificado, 214
- CFE (*Common Firmware Environment*), 145
- CGI (*Common Gateway Interface*), 199
- Character, tipo de datos, 296
- Chips, 9, 29
- Church, Alonzo, 598
- Church-Turing, tesis de, 598, 604
- Ciclo de máquina, 99
- Ciclo de vida del software, 353–358
 - análisis de requisitos, 354
 - diseño, 356–357
 - implementación, 357
 - pruebas, 357
- Cifrado, 212–214, 621–625
- Cilindro, 35
- Cima de la pila, 401
- Cinemática, 526–528
- Cinta magnética, 37
- Circuito integrado, 8
- Circuitos electrónicos, 37
- CISC (*Complex Instruction Set Computer*), 92–93
- Clases, 293, 324–327, 432–434
 - asociaciones entre, 373
 - con constructor, 329
 - de equivalencia, 382
- Cláusula, forma de, 336
- Clave, 41
- Claves de cifrado, 622
- Claves de descifrado, 622
- Claves privadas, 213, 622
- Claves públicas, 212, 622
- Ciente, 175
- Ciente/servidor, modelo, 175–177
- COBOL, 286
- Codificación audio, 48
- Codificación dependiente de la frecuencia, 68
- Codificación diferencial, 69
- Codificación LZW, 70–71
- Codificación por diccionario, 69
 - adaptativa, 70
 - dinámica, 70
- Codificación por longitud de secuencia, 68
- Codificación relativa, 69
- Código de operación, 95, 97, 109, 435
- Código estándar americano para el intercambio de información. Véase ASCII
- Código fuente de una página web, 193
- Códigos de corrección de errores, 77–79
- Códigos de redundancia cíclica (CRC), 77
- Coerción, 321
- Cognética, 385
- Cohesión, 365–366
 - funcional, 365
 - lógica, 365
- Cola, 133, 400–402, 414–417
- Cola circular, 417
- Cola de trabajos, 133
- Colisión, 478
- Colossus, 8
- Comentarios, 294, 305–306
- Comisión Internacional Electrotécnica (IEC), 379
- Commodore, 9
- Compilación just-in-time, 321
- Compiladores, 286
- Complejidad
 - de los problemas, 611–621
 - espacial, 616
 - temporal, 612
- Complemento, 56
- Complemento a dos, notación, 46
- Componentes, 367–369, 400
- Compresión con pérdidas, 68
- Compresión de vídeo, 73
- Compresión sin pérdidas, 68
- Comprobación y configuración, instrucción de, 152
- Computabilidad de las funciones, 592–594
- Computable, función, 592–594
- Computación
 - en cluster, 177
 - en nube, 178
 - en retícula, 178
 - historia de la, 4–11
- Computación, ciencias de la, 2, 11–12
 - repercusiones sociales, 16–18
- Computadora de conjunto
 - complejo de instrucciones (CISC), 92–93
- Computadora de conjunto reducido de instrucciones (RISC), 92–93
- Computadoras de sobremesa, 9
- Computadoras, arquitectura de, 88–91
- Comunicación
 - entre dispositivos, 112–118
 - entre procesos, 175–177
 - errores de, 75–79
 - medios de, 115
 - paralelo, 116
 - serie, 116
 - velocidades de, 116–117
- Concentrador, 169
- Condición de finalización, 245–246, 252
- Conmutador, 173
- Conocimiento
 - declarativo, 535
 - del mundo real, 560–561
 - procedimental, 535

- Conocimiento (*cont.*)
 - representación y manipulación, 560–562
- Constantes, 299–300
- Constructores, 327–329
- Contador de programa, 99
- Contador de saltos, 206
- Contraseña, 158
- Control de congestión, 206
- Control de flujo, 206
- Controlador de dispositivo, 142
- Controladora, 112
- Conversión de barrido, 513–515
- Copyright, 389
- Correo
 - basura (*spam*), 209
 - electrónico (email), 185–186
 - web, 198
- Cortafuegos, 210
- CPU. *Véase* Procesador
- CRC (Códigos de redundancia cíclica), 77
- CRC, tarjetas, 377
- Criptografía de clave pública, 212–214, 621–625
- Crominancia, 46
- Cromosoma, 565
- CSMA/CA (*Carrier Sense, Multiple Access with Collision Avoidance*), 171
- CSMA/CD (*Carrier Sense, Multiple Access with Collision Detection*), 170
- Cubos de datos, 482
- Cuerpo de un bucle, 244

- Darwin, Charles, 579
- Datos
 - acoplamiento de, 363–365
 - compresión de, 68–74
 - globales, 365
 - y programas, 101
- DBMS (*Database Management System*), 449–450, 460, 466
- Declarativa, sentencia, 294, 319, 327
- Declarativo, paradigma, 290
- Decorator, patrón, 378
- Deducción lógica, 335–338
- Define type, sentencia, 430–432
- Degenerado, caso, 261
- Denegación de servicio (DoS), ataques por, 209
- Depuración, 284
- Derecho a la intimidad, 215
- Desarrollo de código fuente
 - abierto, 359
- Desbordamiento, 59
- Descomposición sin pérdidas, 455
- Descripción de clases, 481
- Desenfoco, 526
- Despachador, 143, 147–148
- Desplazamiento
 - aritmético, 109
 - circular, 109
 - lógico, 109
 - operaciones de, 109
- Diagrama de clases, 373, 374, 375
- Diagrama de estructura, 361
- Diagrama de secuencia, 376–377
- Diagramas de flujo, 230, 247
- Diagramas de interacción, 376–377
- Diagramas sintácticos, 316
- Diccionario, 69
- Diccionario de datos, 371
- Difusa, luz, 509
- Digital, tecnología, 54
- Digitalización, 501
- Dijkstra, E. W., 163
- Dinámica, 526–528
- Dirección de celda de memoria, 32
- Direccionamiento, tipos de, 436
- Direct3D, 519
- Directorio, 141
- Disco compacto (CD), 38–39, 78
- Disco versátil digital (DVD), 39
- Disco, formateo, 36
- Discriminación de clases, 481
- Diseño asistido por computadora (CAD), 47
- Diseño relacional, 452–455
- Diseño, etapa del ciclo de vida del software, 356–357
- Dispositivo de Internet móvil (MID), 88
- DMA (*Direct Memory Access*), 114–115
- DNS (*Domain Name System*), 183
- DNS, búsqueda, 183
- Doble núcleo, procesadores de, 119
- DOCTOR, 538
- Documentación, 383–384
 - del sistema, 383
 - del usuario, 383
- técnica, 384
- Dominio de nivel superior (TLD), 183
- Dominio, 182
 - nombre de, 183
- DRAM, 33
- DSL (*Digital Subscriber Line*), 116, 181
- DVD (*Digital Versatile Disk*), 39

- E/S mapeada en memoria, 113
- E/S, instrucciones de, 94
- E/S, solicitudes de, 149
- eBay, 447
- Eckert, J. Presper, 8, 92
- ECPA (*Electronic Communication Privacy Act*), 215
- Edison, Thomas, 92, 356
- Editor de texto, 44
- EFI (*Extensible Firmware Interface*), 145
- Eficiencia de los algoritmos, 262–272
- Ejecución de programas, 99–106
- Eliminación de caras ocultas, 514–515
- Eliminación de caras traseras, 515
- ELIZA, 538
- En línea, 34
- Encaminador, 174–175
- Encaminamiento, 206
- Encapsulación, 330
- ENIAC, 8
- Enlace, capa de, 201, 202, 203
- Enmascaramiento, 107
 - de frecuencia, 73
 - temporal, 73
- Ensamblador de componentes, 368
- Ensamblador, lenguaje, 285
- Ensambladores, 285
- Entorno integrado de desarrollo. *Véase* IDE
- Entrada efectiva de una neurona, 567
- Entrada/Salida (E/S), 94
- Entrenamiento supervisado, 563
- Entrenamiento, conjunto de, 563
- Enunciados incoherentes, 336
- EOF (*End-of-file*), 472
- Equilibrado de carga, 136, 178
- Ergonomía, 385
- Error de truncamiento, 65–67

- Error de redondeo, 65–67
- Escalado, 136
- Escena, 499
- Espacio del problema, 548
- Especificación de requisitos del software, 355
- Especular, luz, 509
- Esquemas, 448
- Estado inicial, 547
- Estado objetivo, 547
- Estructuras de datos, 298–299
 - árboles, 402–403
 - estáticas y dinámicas, 405
 - implementación, 407–420
 - listas, pilas y colas, 400–402
 - manipulación de, 420–422
 - matrices (arrays), 298–299, 400
- Estructuras iterativas, 242–253
- Estructuras recursivas, 254–262
- Ethernet, 169, 172
- Ética
 - basada en el carácter, 17
 - basada en el deber, 18
 - basada en las consecuencias, 17
 - basada en los contratos, 17
 - de la virtud, 18
- Etiqueta en un lenguaje de composición, 192
- Euclides, 2
- Evitación de colisiones,
 - protocolos de, 170–171
- Exclusión mutua, 152
- Exclusivo, bloqueo, 469
- Explorador, 191
- Exploradores de la superficie de Marte, 137
- Exponente, campo, 63
- Extensiones multipropósito de correo Internet (MIME), 185
- Extracción de información, 544
- Extracción, operación de pila, 401
- Extremo de mayor orden, 32
- Extremo de menor orden, 32
- Factor de carga, 479
- Factores de forma, 523
- Fase de desarrollo tradicional del ciclo de vida del software, 354–357
- FIFO (*First-in, first-out*), 133, 402
- Filtros de correo basura, 210
- Fin de archivo, 472
- FireWire, 112, 114, 116–117
- Firma digital, 214
- Firmware, 145
- Flash, memoria, 39
- FlashROM, 145
- Float, 295
- Flowers, Tommy, 8
- Flujo de datos, 29
 - diagrama, 371
- Flujos de audio, 188
- For, estructura de bucle, 304, 305
- Forma normalizada, 64
- Forma, modelado, 500–505
- Formatear, 36
- Formato de intercambio de gráficos (GIF), 71
- FORTRAN, 286, 289, 294, 642
- Fotograma clave, 525
- Fotogramas, 524–525
- Fractales, 503
- Fragmentos de interacción, 376
- Fragmentos hash, 476
- Franja temporal, 147
- FTP (*File Transfer Protocol*), 186
- FTP anónimos, 186
- FTPS, 212
- Fuera de línea, 34
- Funcional, paradigma, 290–292
- Funciones, 312–315, 592–594
 - computable, 592–594
 - no computable, 605–611
- Gandhi, Mahatma, 579
- GB (gigabyte), 33
- Gbps, 74, 116
- Gen, 565
- Generación (*rendering*), 497, 508–520
- Generación del código, 321
- Generaciones de los lenguajes de programación, 284–287
- Generador de código, 315
- Genéticos, algoritmos, 565–566
- Gestor de memoria, 142–143
- GIF (*Graphic Interchange Format*), 71
- Gödel, Kurt, 4
- GOMS, modelo, 387
- Google, 9, 10, 447
- Google Goggles, 546
- Goto, sentencias, 302, 414
- Gouraud, sombreado de, 517
- GPS (*Global Positioning System*), 9
- Grabación de bits por zonas, 35
- Gráficos por computadora, 493–532
 - ámbito, 494–496
 - animación, 524–529
 - gráficos 3D, 495–498
 - iluminación global, 519–523
 - modelado, 499–508
 - renderización, 508–520
- Grafo de escena, 506–508, 512, 519
- Grafo de estados, 548
- Grafo dirigido, 548
- Gramática, 316
- Gramática ambigua, 318
- Grupo de expertos de imágenes en movimiento (MPEG), 73
- Grupos de aseguramiento de la calidad del software (SQA), 379
- GUI (*Graphical User Interface*), 139–140, 388
- Guión (*storyboard*), 525
- Gusano, 208
- Hamming, distancia de, 77–78
- Handshaking, 115
- Hardware, 2
- Hash, archivos, 476–479
- Hash, función, 476
- Hash, tabla, 476
- Hashing, 476
- Heathkit, 9
- Hebra, 332–334
- Herencia, 329–330, 375
- Hermanos, 403
- Herramienta abstracta, 13, 27, 365
- Heurística, 553–560
- Hexadecimal, notación, 29–30
- Hijos, árboles, 403
- Hipermedia, 190
- Hipertexto, 190
- Hipervínculos, 190
- Hollerith, Herman, 7
- Hopper, Grace, 287, 543
- HTML (*Hypertext Markup Language*), 192–195
- HTTP (*Hypertext Transfer Protocol*), 191
- HTTPS, 212
- Huffman, códigos de, 69
- Huffman, David A., 69
- IBM, 7, 8, 9

- ICANN (*Internet Corporation for Assigned Names and Numbers*), 182
- IDE (*Integrated Development Environment*), 352
- Identificadores, 285
- IEC (*International Electrotechnical Commission*), 379
- IEEE (*Institute of Electrical and Electronics Engineers*), 352, 356, 379
- IEEE 1028, estándar, 381
- IEEE 802, estándar, 172
- If-then-else, sentencia, 302–304, 316–317
- Iluminación, modelos de, 519–523
- Imágenes
- compresión de, 71–73
 - en páginas web, 194–195
 - reconocimiento de, 540–543
 - representación de, 46–47
- IMAP (*Internet Mail Access Protocol*), 185
- Imitación, 563
- Imperativa, sentencia, 294, 319
- Imperativo, paradigma, 289, 290, 291–292, 371
- Implementación de un lenguaje, 315–324
- Implementación, etapa del ciclo de vida del software, 357
- Incompletitud, teorema de, 4, 11
- Independencia de los datos, 450
- Indexados, archivos, 474–475
- Índices, 298
- Inferencia, reglas de, 335, 550
- Información errónea, 483–484
- Ingeniería del software, 349–353
- aseguramiento de la calidad, 379–383
 - asistida por computadora (CASE), 352
 - documentación, 383–384
 - en el mundo real, 368
 - estándares, 353, 355
 - herramientas, 370–379
 - metodologías, 358–360
 - modularidad, 360–370
- Inicio de sesión, proceso de, 156
- Instancia de un tipo de datos, 430
- Instancia de una clase, 293, 327
- Instancia, variable de, 325
- Instituto Americano de Ingenieros Eléctricos, 356
- Instituto de Ingenieros de Radio, 356
- Instituto de Ingenieros Eléctricos y Electrónicos. Véase IEEE
- Instituto Nacional Estadounidense de Normalización. Véase ANSI
- Instrucción de nivel máquina, 91–99
- Instrucción privilegiada, 158
- Instrucciones
- de asignación, 231, 301–302, 326
 - de control, 302–305
 - de entrada/salida, 94, 113
 - de longitud variable, 93
- Integer, 295
- almacenamiento de enteros, 56–62
- Integración a muy gran escala (VLSI), 29
- Intel, 92, 93, 101, 114
- Inteligencia artificial, 533–589
- agentes inteligentes, 534–536
 - basada en el comportamiento, 554
 - consecuencias, 578–581
 - en teléfonos inteligentes, 546
 - fuerte y débil, 542
 - investigaciones, 560–566
 - metodologías de investigación, 536–538
 - orígenes de, 538
 - percepción, 540–547
 - procesamiento del lenguaje, 543–545
 - razonamiento, 547–560
 - redes neuronales, 566–575
 - robótica, 575–578
 - test de Turing, 538–539
- Inteligencia basada en el comportamiento, 554
- Interacción entre la luz y las superficies, 509–511
- Interbloqueo, 152–154
- Interconexión de sistemas
- abiertos (OSI), 205
- Interfaces, diseño de, 385–388
- Interfaz de programación de aplicaciones (API) de Java, 368
- Interfaz digital para instrumentos musicales (MIDI), 48
- Interfaz gráfica de usuario (GUI), 139–140, 388
- Intermodular, acoplamiento, 363–364
- Internet, 9, 168, 174, 179–190
- aplicaciones, 184–190
 - arquitectura, 179–181
 - direccionamiento, 181–184
 - protocolos, 200–207
 - software, 200–204
- Internet2, 182
- Interplataforma, software, 287
- Interpolación, 525
- Intérprete, 286
- Interrupciones, 147, 148
- Intranet, 180
- Intratables, problemas, 618
- Introducción, operación de pila, 401
- Invariante de bucle, 270
- IP (*Internet Protocol*), 205–207
- IP, dirección, 182
- IPv4, 207
- IPv6, 207
- ISO (*International Organization for Standardization*), 43, 45, 288, 352, 379
- ISO 9000, estándares, 379
- ISO/OEC 15504, 379
- Isotrópica, superficie, 510
- ISP (*Internet Service Provider*), 179–180
- ISP de acceso, 180
- ISP de nivel 1, 179
- ISP de nivel 2, 179
- Jacquard, Joseph, 6
- Jacquard, telar de, 6
- Java, 293, 294, 301, 303, 304, 334, 368, 643
- implementación, 321
 - punteros en, 414
- JCL (*Job Control Language*), 133
- Jobs, Steve, 9
- JOIN, operación, 458–463
- Joint Photographic Experts Group, 71
- JPEG, 71
- JSP (*JavaServer Pages*), 199
- JUMP, instrucciones, 95, 100
- Just-in-time, compilación, 321
- KB (kilobyte), 33
- Kbps, 74, 116

- Kilby, Jack, 8
- Kilobyte, 33
- Kineografía, 524
- Korn shell, 140
- LAN (*Local Area Network*), 168
- Lector óptico de caracteres, 541
- Legal, enfoque
 - para conjuntos de datos, 484
 - seguridad de red, 214–217
- Leibniz, Gottfried Wilhelm, 5
- Lempel, Abraham, 70
- Lempel-Ziv-Welsh (LZW),
 - codificación, 70–71
- Lenguaje de composición de hipertexto (HTML), 192–195
- Lenguaje de composición extensible (XML). *Véase* XML
- Lenguaje de consulta estructurado. *Véase* SQL
- Lenguaje de control de trabajos (JCL), 133
- Lenguaje de programación universal, 599–604
- Lenguaje formal, 287
- Lenguaje máquina, 91–99, 284–285
 - punteros en, 434–436
 - universal, 321
- Lenguaje unificado de modelado. *Véase* UML
- Lenguajes
 - de composición, 197
 - de formato fijo, 316
 - de formato libre, 316
 - de programación de tercera generación, 286–294
 - fuertemente tipados, 321
 - naturales, 287, 536–537
- Leonardo da Vinci, 92
- Lexema, 315
- Ley anticiberocupas de protección del consumidor, 216
- Ley de fraude y abusos informáticos, 215
- Ley de Intimidad, 484
- Licencia de software, 389–390
- LIFO (*Last-In, First-Out*), 401
- Línea de vida, 376
- Línea digital de abonado (DSL), 116, 181
- Lingüística, 534, 537
- Linux, 132, 140, 359
- Listas, 400–402
- almacenamiento de, 411–414, 423–428
- búsqueda, 254–261
- contiguas, 410, 412
- enlazadas, 412–414, 421
- ordenación de, 248–252
- Literales, 299–300
- LOAD, instrucción, 93–94, 113
- Localizador uniforme de recursos (URL), 191–192
- Lógica formal, 290
- Luminancia, 47
- Luz
 - ambiente, 510
 - difusa, 509
 - especular, 509
 - reflejada, 509–511
 - refractada, 511
- LZW, codificación, 70–71
- Mac OS, 132
- Magnético, disco, 35
- Malla poligonal, 499, 500, 502–503
- Malware (software malicioso), 207
- MAN (*Metropolitan Area Network*), 168
- Mantisa, campo, 63
- Mapa de bits, 46–47, 107
- Máquina analítica, 5–7
- Máquina diferencial, 5, 6, 7
- Máquina electromecánica, 7
- Máquinas multiprocesador, 119
- Marco, 376
- Mark I, 7–8, 270
- Máscara, 107
- Masivo, almacenamiento, 34–42
- Matar un proceso, 153
- Matrices. *Véase* Arrays
- Mauchly, John, 8
- MB (megabyte), 33
- Mbps, 74, 116
- Memoria
 - asociativa, 572–575
 - caché, 90
 - capacidad, 33
 - celdas de, 31–32
 - de acceso aleatorio (RAM), 33
 - de solo lectura (ROM), 144
 - dinámica, 33
 - división de valores almacenados en, 94
 - DRAM, 33
 - flash, 39
 - organización de la, 31–33
 - principal, 31–34
 - RAM, 33
 - ROM, 144
 - SDRAM, 33
 - virtual, 143
- Memoria de fotograma, 498
- Meta-razonamiento, 561
- Metodología abajo-arriba, 240
- Metodología arriba-abajo, 240
- Métodos, 292, 325
- Métodos ágiles, 360
- Métricas, 351
- Microprocesador, 88
- Microsoft, 9, 132, 136, 137, 311, 321
- Microsoft Access, 454
- Microsoft Windows, 132, 137, 142, 151, 311
- MID (*Mobile Internet Devices*), 88
- MIDI (*Musical Instrument Digital Interface*), 48
- Miller, George A., 387
- MIMD, arquitectura, 119
- MIME (*Multipurpose Internet Mail Extensions*), 185
- Minería de datos, 480–483
- Miniaturización, 9
- Modelado, 499–508
- Modelo de base de datos relacional, 452–463
- Modelo de iluminación global, 519–523
- Modelo de iluminación local, 519
- Modelo en cascada, 358
- Modelo incremental, 358
- Modelo iterativo, 358
- Modelo procedimental, 501–503
- Módem, 116, 181
- Modularidad, 360–370
 - acoplamiento, 363–365
 - cohesión, 365–366
 - componentes, 367–369
 - ocultamiento de la información, 366
- Módulos, 361
- Mondrian, Piet, 259
- Monitor, 334
- Morfismo, 524
- Motor de búsqueda, 9, 197
- Movimiento en gráficos 3D, 526–528
- MP3, 73

- MPEG (*Motion Pictures Experts Group*), 73
- MS-DOS, 140
- Muchos-a-muchos, relaciones, 373
- Multidifusión, 189
- Multinúcleo, procesador, 119
- Multinúcleo, sistemas operativos, 154
- Multiplexación, 117
- Multiprogramación, 135, 149
- Multitarea, 135
- NASA, robot, 576
- Neurona, 554, 567–568, 572–575
- NIL, puntero, 413
- No computable, función, 605–611
- No determinista, algoritmo, 618, 621
- No polinómicos, problemas, 615–618
- No terminal, 317
- Nodo hoja, 403
- Nodo raíz, 403
- Nodo terminal, 403
- Normal, 509
- Normal, vector, 517
- NOT, 26, 27
- Notación
 - binaria, 44–45, 50–52
 - decimal con puntos, 182
 - en exceso 16, 60–61
 - en punto flotante, 46, 63–65
 - modular, 622–623
- Novell Inc., 168
- NP, problemas, 618–620
- NP-completos, problemas, 619
- NPT Inc., 390
- Núcleo, 141
- Números fraccionarios
 - almacenamiento de, 63–67
 - en binario, 54–55
- N-unidifusión, 188
- O mayúscula, notación, 613
- Objeto transparente, 521–522
- Objetos, 293, 324–327, 432–434
 - modelado, 499–506
 - persistente, 465
 - renderización, 508–520
- Ocultamiento de la información, 366
- OOP. Véase Programación orientada a objetos
- Open Firmware, 145
- OpenGL (*Open Graphics Library*), 519
- Operaciones
 - aritméticas, 110, 301
 - booleanas, 24–25
 - lógicas, 107–108
 - relacionales, 455–460
- Operadores, precedencia de, 301
- Operando, campo, 95–97
- Optimización de código, 322
- OR, 24–27, 107–108
- Oracle, 294, 378
- Ordenación
 - por columnas, 408
 - por combinación, algoritmo de, 613–615
 - por filas, 408
 - por inserción, algoritmo, 248–252, 264–266
- Organización Internacional de Estandarización. Véase ISO
- Orientadas a objetos, bases de datos, 463–466
- Orientado a objetos, paradigma, 292, 362
- OSI (*Open System Interconnection*), 205
- P2P, modelo, 176–177
- Página de memoria, 143
- Paginación, 143
- Paint, Microsoft, 494
- Palabra de estado, 115
- Palabras clave, 316
- Palabras reservadas, 316
- Palm OS, 137
- Paquetes, 202
- Parámetros, 234, 308–312
 - formales, 309
 - paso por referencia, 311, 313
 - paso por valor, 310, 312
 - reales, 309, 310
- Parche plano, 499
- Paréntesis, 301
- Pareto, principio de, 381
- Pareto, Vilfredo, 381
- Paridad impar, 75
- Paridad par, 76
- Pasarela (*gateway*), 175
- Pascal casing, 233
- Pascal, Blaise, 5
- Paseos estructurados, 377
- Paso por referencia, 311, 313
- Paso por valor, 310, 312
- Patentes, 389
- Patrones de diseño, 377–378
- PC (*Personal Computer*), 9, 454
- Peer-to-peer (P2P), modelo, 176–177
- Pentium, microprocesador, 270
- Percepción, 540–547
- Pérdidas de memoria, 425
- Perl, 296
- Persona-máquina, interfaz, 385–388
- Peso en una neurona artificial, 567
- Phishing, 209
- Phong, sombreado, 517
- PHP, 199, 296
- Pila, 400–402, 414–417
- Pista, 35
- Pixel, 46
- Placa madre, 88
- Planificador, 143
- Plano de proyección, 497
- Plantilla, 367–368, 378
- Platón, 18
- Pocket PC, 137
- Polimorfismo, 330
- Polinómicos, problemas, 615–618
- Polinomio de dirección, 409
- Polya, G., 236
- POP3 (*Post Office Protocol version 3*), 185
- Post, Emil, 595
- Post, sistema de producción de, 595
- Postcondiciones, 270
- PostScript, 47
- Potencia de cálculo, 101
- Precedencia de los operadores, 301
- Precondiciones en demostraciones de corrección, 269
- Predicados, 338–339
- Pretty Good Privacy, 212
- Primero en entrar, primero en salir. Véase FIFO
- Primitivas, 228–230
- Primitivos, tipos de datos, 297
- Privilegio, niveles de, 158
- Problema
 - de la actualización perdida, 469

- de la detención, 605–607
- de la mochila, 629
- de totalización incorrecta, 468
- del marco, 562
- del terminal oculto, 171, 172
- del viajante, 618–619
- polinómico no determinista, problema NP, 619
- Problemas, complejidad de, 611–621
- Problemas irresolubles, 610
- Procedimental, paradigma, 289, 290
- Procedimental, unidad, 307–315
- Procedimientos, 307–308, 370
 - cabecera de, 307–308
 - invocación de, 307
- Procesador, 88–89, 99, 100, 143
 - basado en ARM, 93, 101, 129
 - de doble núcleo, 119
 - Intel, 92
 - multinúcleo, 119
- Procesador de textos, 44
- Procesamiento
 - concurrente, 332–334
 - de imágenes, 494, 541–542
 - del lenguaje, 543–545
 - en cadena, 118
 - en tiempo real, 134
 - interactivo, 134
 - paralelo, 119, 332–334
 - por lotes, 133
- Proceso de arranque, 143
- Proceso unificado racional (RUP), 359
- Procesos, 146
 - conmutación de, 147
 - estado del proceso, 146
 - gestión de la competencia entre, 150–155
 - inicio/detención, 147–148
 - matar (*kill*), 153
 - tabla de, 147
- Producción, 547
- Profundidad de un árbol, 403
- Programa, 2, 227
 - almacenado, concepto de, 89–91
 - fuentes, 315
 - objeto, 315
 - verificación de, 267–272
 - y datos, 100–101
- Programación
 - declarativa, 335–341
 - estructurada, 303
 - evolutiva, 565
 - extrema (XP), 360
 - lógica, 290, 338–341, 564
 - modular, 361–363
 - paradigmas, 288–294
- Programación, conceptos, 294–307
 - comentarios, 294, 305–306
 - constantes, 299–300
 - estructuras de datos, 298–299
 - literales, 299–300
 - sentencias de asignación, 301–302
 - sentencias de control, 302–305
 - tipos de datos, 295–298
 - variables, 295–297
- Programación, lenguajes de, 229
 - culturas, 303
 - historia, 284–294
 - implementación, 315–324
 - lenguajes de script, 296
 - primeras generaciones de, 284–287
 - procesamiento concurrente y, 332–334
 - programación declarativa y, 335–341
 - sintaxis, 316–317
 - universal, 599–604
- Programación orientada a objetos (OOP), 292–293, 324–331
 - clases, 324–327
 - constructores, 327–329
 - encapsulación, 330
 - estructura de programa, 328
 - herencia, 329–330, 375
 - objetos, 324–327
 - polimorfismo, 330
- Programador, 357
- PROJECT, operación, 458, 460–462
- Prolog, 338–341
- Propiedad intelectual, 16, 389
- Protocolo de acceso a correo Internet (IMAP), 185
- Protocolo de confirmación/anulación, 467–468
- Protocolo de control de transmisión. Véase TCP
- Protocolo de datagramas de usuario (UDP), 205–207
- Protocolo de espera y desalojo, 470
- Protocolo de transferencia de archivos (FTP), 186
- Protocolo de transferencia de hipertexto (HTTP), 191
- Protocolo simple de transferencia de correo (SMTP), 185
- Protocolos, 170–172
 - Internet, 200–207
- Prototipado descartable, 359
- Prototipado evolutivo, 359
- Prototipado rápido, 359
- Proveedor de servicios de Internet, 179–180
- Proyección en perspectiva, 497
- Proyección paralela, 497
- Proyectores, 497
- Pruebas
 - del camino básico, 381
 - de caja de cristal, 382
 - de caja negra, 382
 - de rendimiento, 101
 - de software, 381–382
 - etapa del ciclo de vida del software, 357
- Pseudocódigo, 230–235
- Puente, 173
- Puertas lógicas, 25–29
- Puerto, 112
 - número de, 204
- Puntero
 - al hijo derecho, 418
 - al hijo izquierdo, 418
 - de cabecera, 412, 415
 - de instrucciones, 406
 - de pila, 415–416
 - final, 415
 - nulo, 413
 - raíz, 418
- Punteros, 405–406, 414, 418, 434–436
- Punto caliente, 181
- Punto de acceso (AP), 169
- Punto de confirmación, 468
- Punto de separación, 54
- Punto de vista, 497
- Punto flotante de doble precisión, 66
- Punto flotante de simple precisión, 66
- Puntos de control, 501, 525
- Puzzle de ocho piezas, 536, 540–541, 547–549, 551–557
- Python, 317

- Radio Internet, 188
- Radio Shack, 9
- Radiosidad, 522–523
- RAM (*Random Access Memory*), 33
- Rama, 403
- Rasterización, 513
- Razonamiento, 547–560
- Real, tipo de datos, 295
- Realce de contornos, 542
- Realismo, 505–506
- Recolección de basura, 425
- Reconocimiento de caracteres, 540–541
- Recorte, 511
- Recuperación de información, 544
- Recurción, 261
- Red
 - abierta, 168
 - cerrada, 168
 - de área extensa (WAN), 168
 - de área local (LAN), 168
 - de área metropolitana (MAN), 168
 - propietaria, 168
 - semántica, 545
 - capa de, 201, 202, 203
- Redes, 136, 167
 - clasificaciones, 168–169
 - combinación de, 172–175
 - comunicación, 175–177
 - fundamentos, 168–179
 - neuronales, 566–575
 - protocolos, 170–172
 - seguridad, 207–217
 - topología, 169
- Reenvío, 206
 - tabla de, 175
- Refinamiento sucesivo, 240
- Reflexión, 509–511
- Refracción, 511
- Refuerzo, 563
- Región crítica, 152
- Registradoras, 183
- Registro, 298
- Registro de instrucciones, 95
- Registro federal, 484
- Registro físico, 40
- Registro lógico, 40
- Registros de uso especial, 88–89
- Registros de uso general, 88–89
- Relaciones, 452
- Reloj, 101
- Repeat, bucle, 247, 248
- Repetidor, 173
- Representación de la información, 42–50
- Research in Motion* (RIM), 390
- Resolución de problemas, 236–238
- Resolución, 335–338
- Resolvente, 336
- Responsabilidad legal, 389
- Revisiones en el desarrollo del software, 380
- RGB, codificación, 46
- RISC (*Reduced Instruction Set Computer*), 91–92
- Risks Forum, 380
- Ritchie, Dennis, 641
- Rivest, Ron, 212, 621
- Robótica, 575–578
- ROM (*Read-Only Memory*), 144
- Rossum, Guido von, 317
- Rotación, 109
- RSA, algoritmo, 621, 623–625
- RUP. Véase Proceso unificado racional
- Ruta de directorio, 142
- Rutina de tratamiento de instrucciones, 148–149
- Salto condicional, 95
- Salto incondicional, 95
- Sangrado, 316
- Sangrado de color, 523
- Script, lenguajes de, 296
- SD, tarjetas de memoria, 40
- SDHC, tarjeta de memoria, 40
- SDRAM, 33
- SDXC, tarjeta de memoria, 40
- Sector, 35
- Secuenciales, archivos, 471–474
- Seguridad
 - de red, 207–217
 - sistemas operativos, 156–159
- SELECT, operación, 457, 460–462
- Semáforo, 150–152
- Semántica, 229
- Semántica web, 197
- Serie, comunicación, 116
- Servidor, 175
 - de archivos, 176
 - de correo, 185
 - de impresión, 175
 - de nombres, 183
 - proxy, 210
- Shamir, Adi, 212, 621
- Shell de comandos, 140
- Shockley, William, 8
- SIMD, arquitectura, 120
- Sintaxis, 229, 316–317
- SISD, arquitectura, 119
- Sistema binario, 50–53
- Sistema de almacenamiento en disco, 35–37
- Sistema de control, 548
- Sistema de gestión de bases de datos. Véase DBMS
- Sistema de nombres de dominio (DNS), 183
- Sistema de partículas, 503
- Sistema de posicionamiento global (GPS), 9
- Sistema de producción, 547–550
- Sistema en un chip, 119
- Sistema operativo, 131–165
 - arquitectura, 137–146
 - asignación de recursos, 150–155
 - componentes, 139–143
 - coordinación de las actividades, 146–150
 - definición, 132
 - historia, 132–137
 - inicio del, 143–146
 - multinúcleo, 154
 - seguridad, 156–159
- Sistemas de alta disponibilidad, 178
- Sistemas de bases de datos, 446–448, 454
- Sistemas distribuidos, 177–178
- Sistemas empotrados, 136
- Sistemas expertos, 550
- Sistemas ópticos, 38–39
- Sistemas software dirigidos por eventos, 314
- Sistemas terminales, 180
- SMTP (*Simple Mail Transfer Protocol*), 185
- Sobrecarga, 302
- Software, 2
 - clasificación, 137–138
 - comercial, 354
 - de aplicación, 138, 139
 - de dibujo, 47, 501
 - de red, 200–204
 - de utilidad, 138–139
 - del sistema, 138

- dirigido por eventos, 314
- espía (*sniffing*), 157, 208–209
- multiplataforma, 287
- paquetes de desarrollo, 322
- pruebas, 381–382
- teléfono inteligente, 334
- verificación, 267–272
- Sombras proyectadas, 519
- Sombreado, 516–518
- Sombreado plano, 516
- Sonido, representación del, 47
- SPARK, 271
- Spooling*, 154
- Spyware*, 208–209
- SQL, 460–463
- SSH (*Secure Shell*), 186
- SSL (*Secure Sockets Layer*), 212
- Stibitz, George, 7
- STL, librería, 433
- STORE, código de operación, 93, 113
- Structured Query Language* (SQL), 460–463
- Suavizado, 542
- Subárboles, 403
- Subdominio, 183
- Subesquema, 448
- Subprograma, 232
- Subrutina, 232
- Suma binaria, 52–53
- Suma en notación en complemento a dos, 58
- Sumas de comprobación, 77
- Sun Microsystems, 294, 321
- Superusuario, 156
- Suplantación (*spoofing*), 210
- Suposición del mundo cerrado, 561
- Switch, instrucción, 304
- Tabla de símbolos, 319
- Tarjeta gráfica, 518
- Tarjetas de memoria SD, 40
- Tarjetas de memoria SDHC, 40
- Tarjetas de memoria SDXC, 40
- Tarjetas perforadas, 7
- Tasa de procesamiento, 118
- Tasa de transferencia, 36 TB (terabyte), 33
- TCP (*Transmission Control Protocol*), 205–207
- TCP/IP, protocolos, 168, 205–207
- Tecnologías analógica y digital, 54
- Teléfono inteligente, 10, 136, 187–188, 334, 352, 369, 546
- Teléfono móvil, 188
- Teléfono software, 187
- Telnet, 186
- Televisión 3D, 507
- Teoría de funciones recursivas, 592
- Terminal, en un diagrama sintáctico, 317
- Test de inteligencia, 579–580
- Test de Turing, 538–539
- Texto, representación de, 42–44
- Therac-25, 380
- Tiempo compartido, 135
- Tiempo de acceso, 36
- Tiempo de búsqueda, 36
- Tiempo de latencia, 36
- Tiempo de rotación, 36
- TIFF (*Tagged Image File Format*), 72–73
- Tipos de datos, 295–297
 - definidos por el usuario, 429–430
 - personalizados, 428–432
 - tipo abstracto de datos, 430–432
- Tipos, conversión explícita de, 321
- Tipos, promoción de, 321
- TLD (*Top-Level Domain*), 183
- TLD de código de país, 183
- Topología en estrella, 169
- Torvalds, Linus, 140, 359
- Traducción, proceso de, 315–322
- Traductor, 286
- Transferencia de datos, 93–94
- Transistor, 8
- Transporte, capa de, 201–204
- Trazado de rayos, 520–522
 - distribuido, 521
 - recursivo, 522
- Tribunal Penal Internacional, 214
- TrueType, 47
- Tubo de vacío, 8
- Tupla, en una relación, 452
- Turing, Alan, 538, 595
- Turing, computable según, 598
- Turing, máquinas de, 594–599
- UDP (*User Datagram Protocol*), 205–207
- Umbral, valor, 567
- UML (*Unified Modeling Language*), 371–377
- Unicode, 43
- Unidad aritmético/lógica, 88, 94, 101
- Unidad central de proceso (CPU). Véase Procesador
- Unidad de control, 88, 89, 94–95
- Unidad de registros, 88, 89
- Unificación, 338
- UNIX, 132, 140
- Uno-a-muchos, relaciones, 373–374
- Uno-a-uno, relaciones, 373–374
- Urban Challenge*, 576
- URL (*Uniform Resource Locator*), 191–192
- USA PATRIOT, Ley, 216
- USB (*Universal Serial Bus*), 112, 114, 116
- Usuario, interfaz de, 139–140, 385–388
- Utilidad, software de, 138–139
- Utilitarismo, 17
- Valores numéricos, 44–46
- Variables
 - ámbito de, 308
 - asignación, 326
 - de instancia, 325
 - globales, 308
 - locales, 308
- VBScript, 296
- Vector normal, 517
- Velocidades de comunicación, 116–117
- Velocidades de reloj, 101
- Ventana de imagen, 497
- Verificación del software, 267–272
- Videojuegos, 498, 519, 525
- Virtual, memoria, 143
- Virus, 207
- Visual Basic, 296, 311
- VLSI (*Very Large-Scale Integration*), 29
- VoIP (Voz sobre IP), 187–188
- Volumen de visualización, 512
- Von Koch, copo de nieve, 503
- Von Neumann,
 - arquitectura de, 115
 - cuello de botella de, 115
- Von Neumann, John, 92
- VxWORKS, 137
- W3. Véase World Wide Web

- W3C. *Véase* World Wide Web Consortium (W3C)
- WAN (*Wide Area Network*), 168
- Web, páginas, 191, 192–195
- Web, servidor, 191, 198
- Web, sitios, 191
- Weizenbaum, Joseph, 580
- Welsh, Terry, 70
- While, bucle, 247, 270
- While, sentencia, 244, 245, 251, 303–304
- WiFi, 171
- Windows CE, 137
- Windows, 132, 137, 142, 151, 311
- World Wide Web, 9, 190–200
- World Wide Web Consortium (W3C), 191
- Wozniak, Stephen, 9
- WWW. *Véase* World Wide Web
- X11, 140
- XML (*eXtensible Markup Language*), 196–197
- XOR (OR exclusiva), 24, 25, 26, 107–108
- XP (*eXtreme Programming*), 360
- Yahoo, 9
- Zeta-mayúscula, notación, 267, 613
- Ziv, Jacob, 70

Este texto está dirigido a estudiantes de Ciencias de la computación, así como a estudiantes de otras disciplinas. La mayoría de los estudiantes de Ciencias de la computación inicia sus estudios con la ilusión de que esta disciplina tiene que ver fundamentalmente con la programación, las páginas web y la compartición de archivos a través de Internet, ya que eso es básicamente lo que han visto hasta el momento. Sin embargo, el campo de las Ciencias de la computación va mucho más allá. Por ello, los estudiantes que comienzan en este campo necesitan conocer la amplitud de la materia en la que pretenden titularse. Tratar de comunicar esa amplitud es, precisamente, el objetivo de este libro. En él se proporciona a los estudiantes una panorámica de las Ciencias de la computación, una base que les permita apreciar la importancia y las interrelaciones de las futuras asignaturas que vayan cursando.

Asimismo, este amplio repaso es lo que necesitan los estudiantes de otras disciplinas que tengan que relacionarse con la sociedad técnica en la que viven. Un curso sobre Ciencias de la computación para este tipo de audiencia debería proporcionar una comprensión práctica y realista del campo de estudio completo, en lugar de simplemente una introducción a la utilización de Internet o una formación en el uso de algunos paquetes software populares. Por supuesto, existe también un lugar apropiado para este tipo de formación, pero este texto trata acerca de la educación. Por ello, a la hora de escribir el libro, uno de los objetivos principales era que fuese accesible para los estudiantes no técnicos. El resultado es que las ediciones anteriores se han empleado con éxito en cursos para estudiantes de un amplio rango de disciplinas y de niveles educativos, que van desde el Bachillerato a los cursos de máster. Esta undécima edición está diseñada para continuar con esa tradición.

Esta edición de *Introducción a la Computación* incluye seis meses de acceso gratuito al sitio web en inglés www.pearsonhighered.com/brookshear. Esta suscripción gratuita proporciona acceso completo a todas las áreas de soporte al estudiante.

PEARSON

www.pearson.es

ISBN: 978-84-7829-139-7



9 788478 291397